

微构体中填充导电介质的仿真优化

摘要

本文研究边长 10000 nm 的微构体填充直圆柱介质 A 与球形介质 B 后的导通问题。四问共用一个判定内核：按题给边界截断规则把越界介质折回微构体，两两计算表面最短距离，不超过 1.8 nm 即连边，再用并查集判断两个带电面是否落入同一连通分量。介质 A 建模为球柱体，使表面距离精确等于轴线段最短距离减 $2r$ ，判定完全向量化。内核经 5 项独立校验，其中用附件数据做的“还原—重截断”往返测试对 334 根介质逐坐标一致。

建模前的数据审计改变了模型。附件的一行不是一根介质 A，而是截断后的碎片（组 1/2/3 的母介质数为 7/28/354）；组 1、组 2 的周期盒是 $10000 \times 1000 \times 1000$ nm 而非立方体。组 3 的方向也不是各向同性，而是以带电面法向为极轴的极角均匀分布（KS 检验： $p = 0.73$ 不被拒绝，各向同性以 $p = 6.7 \times 10^{-19}$ 被拒绝）。但题面对问题二至四只写“方向随机”、未指定分布，故按最少假设取**球面均匀为主口径**，附件标定分布作为**灵敏度口径**全程并列——两者之差是全题最大的不确定来源。

问题一按各组实际几何直接判定：组 1 **不导通**、组 2 **导通**、组 3 **导通**。补回丢弃的短碎片、把判据阈值在 0.9–2.7 nm 间扰动，结论均不变；改用题面的逐字口径，三个判定也不变。**问题二**用蒙特卡洛频率估计、Wilson 区间给区间估计，每档 8000 次试验，得四档 $\varphi = 0.50\%/0.60\%/0.70\%/1.00\%$ 的 $P = 0.0874/0.2325/0.4911/0.9940$ ；排除体积判据的独立估计与仿真跃变中点一致到 9%。

问题三先用 logit 拟合粗定位跃变点，再在 0.01% 网格上逐点复算，球柱体口径给出 0.86%（608 根，12000 次独立复核 $P = 0.9062$ ）。球柱体只是真实圆柱的外界，用内界重算得 $\varphi_{\min} \in [0.86\%, 0.89\%]$ ，**可证达标的最小值为 0.89%（630 根）**。

问题四中导通概率对 N_A 、 N_B 单调不减，最优解必落在可行边界上，二维整数搜索降为一维。用代理模型缩小范围、候选点大样本验证，得 $N_A = 608$ 、 $N_B = 0$ 、成本 9.0252 元；再用单调性做预算线审计， $N_A \leq 480$ 的更便宜点被角点判据严格排除。但最优解附近是**退化**的——(604, 0)、(606, 17)、(607, 8)、(592, 141) 四个更便宜的配比即便做到 $T = 20000$ ，其导通概率与 0.90 的差距仍小于可分辨的尺度，既不能采纳也不能排除。故本文不主张“更便宜的点已全部排除”，而给出：**推荐配比** (608, 0)、**成本 9.0252 元**（唯一可行性被确认的最低成本方案），**最低总成本报到 9.02 元这一精度**。

关键词：连续渗流，蒙特卡洛仿真，周期边界截断，球柱体最短距离，仿真优化

一、问题重述与问题分析

1.1 问题背景与研究对象

导电体系的宏观导电性由填充其中的导电介质能否搭出一条贯穿电极的通路决定。题目把这一体系理想化为边长 10000 nm 的立方体“微构体”：内部充满绝缘溶剂与若干导电介质，选取垂直于 X 轴的两个面 ($x = \pm 5000$ nm) 为带电面，其余四面绝缘。两个导体之间、或导体与带电面之间，当表面最短距离不超过 1.8 nm 时视为导通；若两带电面之间存在一条由导电介质接力构成的通路，则判定微构体导通。

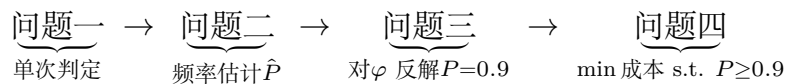
导电介质有两种：介质 A 为高 5000 nm、底面半径 30 nm 的直圆柱；介质 B 为半径 200 nm 的正球体。介质位置与方向随机，互不碰撞，允许相互贯穿；越出微构体边界的部分按**边界截断规则**（沿越界方向的反方向平移一个边长，必要时重复）折回微构体内。

1.2 需要回答的问题

1. 附件给出三个微构体中介质 A 的坐标，分别判定是否导通；
2. 只填介质 A，体积分数为 0.50%、0.60%、0.70%、1.00% 时的导通概率；
3. 只填介质 A，使导通概率不低于 90% 的最低体积分数（精确到 0.01%）；
4. A、B 混填，在导通概率不低于 90% 的前提下使填充总成本最低的填充量。成本为 A 1.05 元/ μm^3 、B 0.05 元/ μm^3 。

1.3 总体思路

四问共用同一个判定内核：**给定一组介质的位姿，判断微构体是否导通**。内核由三步组成——边界截断把每根介质化为若干位于盒内的碎片；两两计算表面最短距离，不超过 1.8 nm 则连边；并查集求连通分量，判断是否存在同时连到两个带电面的分量。四个问题只是内核的不同用法：



从数学结构看，问题二、三是**连续渗流**（continuum percolation）中贯穿概率的估计与反解——这正是碳纳米管、炭黑等导电填料复合材料里“填到多少才导电”这一问题的数学形式 [5]；问题四是以蒙特卡洛估计量为约束的**整数规划**，即题目标题所说的“仿真优化”。

整体求解路径如图 1（数学建模的一般流程与常用算法可参见 [1][2]）：先由附件数据审计定下建模口径（第 2 节），据此实现并校验判定内核（第 3 节），四个问题再分别以单次判定、频率估计、阈值反解和带约束优化的方式调用同一内核；每一环都

配有独立校验，结论不依赖任何未经检查的中间结果。

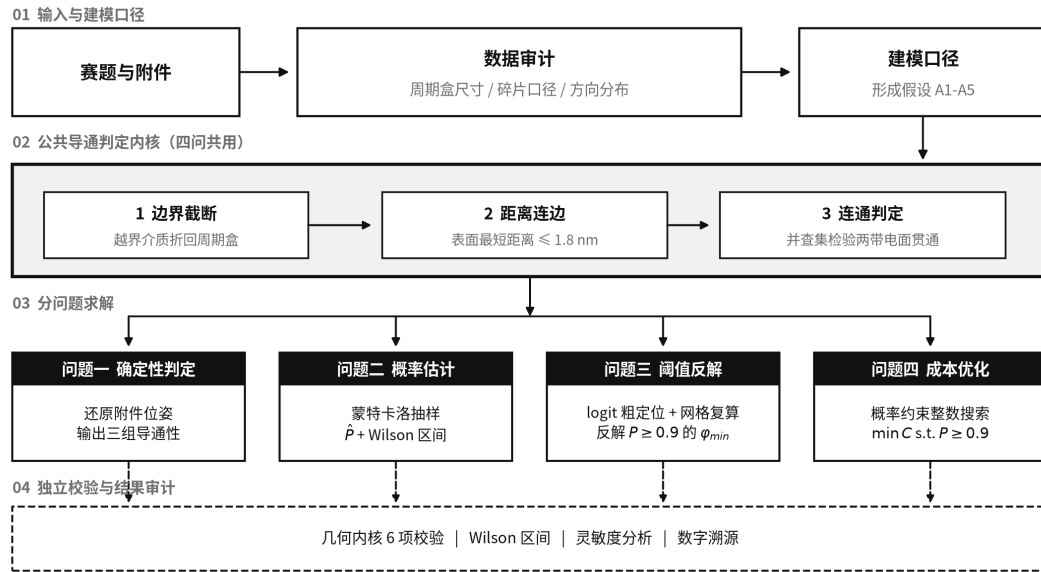


图 1 技术路线：四问共用同一个导通判定内核

1.4 问题一的分析

问题一表面上是”代入判定”，但附件的组织方式与题面并不直接对应：附件的一行并不是一根介质 A。因此问题一实际上先是一个数据还原问题，其结论同时决定了问题二至四随机生成器的口径。本文把这一步单独作为第 2 节。

1.5 问题二、三的分析

导通与否是随机配置的函数，导通概率没有解析表达式，只能用蒙特卡洛频率估计，并给出置信区间。问题三要求精确到 0.01%，对应介质 A 根数相差约 7 根，需要先用 logit 拟合定位跃变点，再在 0.01% 网格上直接复算，避免把结论压在拟合模型的形式上。

1.6 问题四的分析

介质 B 单颗成本只有介质 A 的 1/8.9，但半径 200 nm 的球跨接能力远不如长 5000 nm 的棒；球的作用是充当棒与棒之间的”桥”。由于导通概率对 N_A 、 N_B 都单调不减，最优解必落在可行域边界上，二维搜索可降为沿边界的一维搜索。

二、附件数据审计与建模口径的确定

先看数据再建模。附件里有四件事与题面的字面读法不同，每一件都改变了后面的模型。本节的全部结论由 `src/audit_attachment.py` 产生，可复现。

2.1 附件的一行是碎片，不是一根介质

先把冲突摆出来。附件说明的原文是：

每一行表示一个介质 A，前三列表示一个顶点（顶点指介质 A 的轴与介质 A 底面的交点）的三维坐标，后三列表示另外一个顶点的三维坐标。

本节要论证的结论与这句话相反，因此证据必须足够硬。三条如下：

1. **行长与题面的几何规格矛盾**。题面规定介质 A 是高 5000 nm 的直圆柱，轴长恒为 5000 nm；但附件中绝大多数行的两端点距离都小于 5000 nm（组 3 中仅 155/535 行等于 5000，最短的一行只有 526 nm）。若”一行一根”，则附件里绝大多数介质 A 都不合题面规格。
2. **同向行的轴长之和恰好补满 5000 nm**。把方向向量完全相同的行归为一组后，组数为 7 / 28 / 354，且每组轴长之和都不超过 5000 nm、绝大多数恰好等于 5000 nm。
3. **还原后重新截断能逐坐标复现附件**。第 8.1 节的 T3 把碎片接成母介质轴线再按题面规则重新截断，334 根完整介质与附件**逐坐标一致**。若”一行一根”，这个往返测试无从谈起。

故：一行是边界截断后的一个碎片，同向的若干行合起来才是一根母介质。组 3 还原出的 354 根母介质对应体积分数 0.5005%，而问题二第一档 0.50% 按题目的四舍五入规则恰好给出 $N_A = \text{round}(353.68) = 354$ 根——**根数分毫不差**，说明组 3 就是按问题二的口径生成的。（需要说明的是，仅凭”体积分数约等于 0.50% “并不能区分两种读法：若按各行的实际长度在 10000^3 盒里折算，组 3 也有 0.4965%，同样约等于 0.50%。真正有区分力的是上面的第 3 条与” 354 根分毫不差”这两点，见第 4.3 节表 6 的三口径对照。）

2.2 组 1、组 2 的周期盒不是立方体

碎片在边界处成对出现（前一碎片终点在 $+h$ 、后一碎片起点在 $-h$ ，其余两坐标完全相同），因此只要统计碎片端点**精确落在**某个平面上的次数，就能直接读出回绕面的位置。坐标是浮点数，随机落到某个整值平面上的概率为零，所以这种精确命中只能来自截断本身：

表 1 碎片端点落在候选周期面上的次数（判据： $||x_j| - h_j| < 10^{-6} \text{ nm}$ ）

微构体	X 面 ± 5000	Y 面 ± 500	Z 面 ± 500	Y 面 ± 5000	Z 面 ± 5000
组 1	7	2	4	0	0
组 2	22	13	11	0	0
组 3	181	—	—	118	111

结论很直接：组 3 的三个方向都在 ± 5000 环绕，与题面的 10000^3 立方体一致；而组 1、组 2 的 x 在 ± 5000 环绕、 y 与 z 却在 ± 500 环绕，截面只有 1000×1000 nm。两组的体积分数因此是 0.99% 与 3.96%，而非按立方体折算的 0.0099% 与 0.0396%——差了整整 100 倍。问题一必须按各自的实际几何判定。

2.3 附件丢弃了短碎片

把每根母介质的碎片首尾相接，长度应恰为 5000 nm，接不满的部分就是被附件丢弃的碎片。三组都有：组 1 有 2 根接不满（缺口 174 与 577 nm），组 2 有 4 根（缺口 48–454 nm），组 3 有 49 根（46 根缺口小于 500 nm，另 3 根为 636/732/870 nm）。而附件中保留的最短碎片是 526 nm。

这些数字与”按长度阈值筛除短碎片”的假设一致：组 3 的 46 个小缺口都在阈值以下；3 个较大的缺口若各由两个碎片构成，则单个长度仍在 500 nm 以下。需要讲清楚的是，“各缺两个碎片”是与该假设相容的解释，不是独立证据——若把它当成推论，就成了用阈值假设去证明阈值假设。组 1 那个 577 nm 的缺口同理：它单独看已超过 526 nm，只有在”由两个碎片构成”时才与阈值假设相容。本文只主张：丢弃的痕迹与长度阈值筛除相容，不主张已证明阈值的具体数值。

图 2 把保留碎片与反推出的缺失碎片画在同一坐标上，两者在 500 nm 处几乎不重叠。被丢弃的碎片有可能正好是一座桥，因此问题一另按”补回缺失碎片”的口径复判一次，结论不变（第 4.3 节）——结论不依赖上面这段关于阈值的解释是否正确。

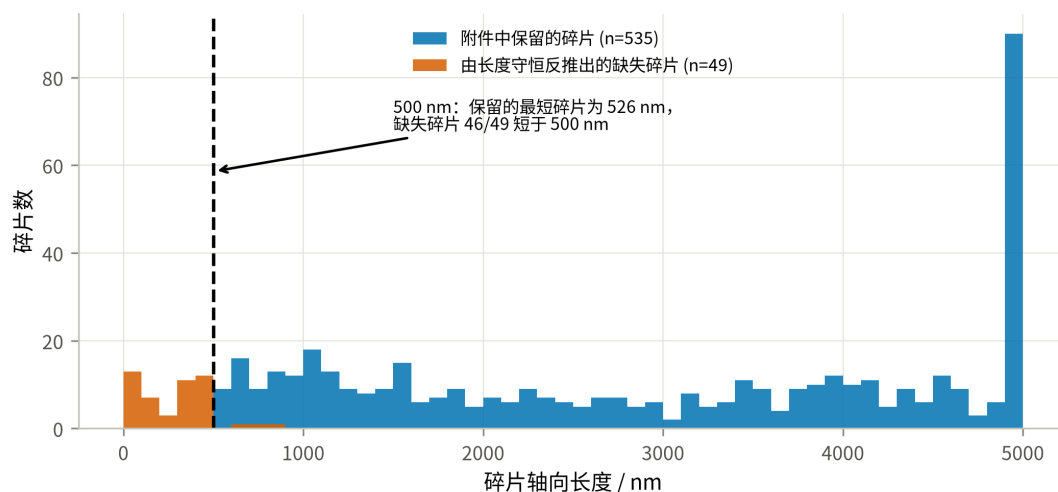


图 2 附件保留碎片与反推出的缺失碎片的长度分布

2.4 附件的方向分布不是各向同性——一条不进入主口径的发现

组 3 中 354 根母介质方向余弦绝对值的均值为 $E|u| = (0.656, 0.390, 0.394)$ 。各向同性应为 $(0.5, 0.5, 0.5)$ ；而以 X 轴（带电面法向）为极轴、 $\theta \sim U(0, \pi)$ 、 $\phi \sim U(0, 2\pi)$ 的

分布, 理论值为 $(2/\pi, (2/\pi)^2, (2/\pi)^2) = (0.637, 0.405, 0.405)$ 。对 $|u_x|$ 作 Kolmogorov–Smirnov 检验 (分布拟合优度检验见 [4]):

表 2 组 3 介质方向分布的三项检验

检验对象	假设	统计量	p 值	结论
$ u_x $ 的边缘分布	球面均匀 (各向同性)	$D = 0.243$	6.7×10^{-19}	拒绝
$ u_x $ 的边缘分布	$\theta \sim U(0, \pi)$, 极轴为 X	$D = 0.036$	0.73	不拒绝
方位角 $\phi = \arctan(u_z/u_y)$	$\phi \sim U(0, 2\pi)$	$D = 0.024$	0.98	不拒绝
$ u_x $ 与 ϕ	相互独立	$\rho_s = 0.016$	0.76	不拒绝

只检验 $|u_x|$ 的边缘分布并不足以确定整个方向分布, 因此后两行补上了方位角的均匀性与两者的独立性: 三项检验都不拒绝, 说明” $\theta \sim U(0, \pi)$ 、 $\phi \sim U(0, 2\pi)$ 且相互独立” 这一完整的三维分布与附件数据相符, 而不只是某个边缘量对得上。(方向只在反号意义下确定, 本文取 $u_x \geq 0$ 的一支; 反号把方位角平移 π , 均匀分布对平移不变, 故检验不受影响。)

就附件本身而言, 各向同性被断然拒绝。图 3 左图把经验累积分布与两条理论曲线画在一起: 经验分布几乎贴合极角均匀的曲线, 而与各向同性的对角线明显分离; 右图用三个方向余弦的均值给出同一结论。(组 1、组 2 的 $E|u_x|$ 高达 0.99, 但它们的截面只有 1000×1000 nm, 5000 nm 的棒**几乎必须**沿 X 排列, 那是几何强制而非分布信息, 故不作为证据; 上表只用组 3——唯一的立方体盒、样本最大且无此约束。)

但这条发现不进入主口径。题面对问题二至四只写” 介质位置与方向随机”, 并未指定分布, 也未说随机构型必须沿用附件的生成过程; 附件是问题一的输入, 其分布刻画的是那三个具体微构体是怎么造出来的。在题面没有给出分布时, **球面均匀是”方向随机”的最少假设读法**, 因此本文以球面均匀为主口径, 并把附件标定的极角均匀分布作为**灵敏度口径**全程并列给出 (第 8.2 节表 19)。

这个选择必须明说其代价: 两种口径给出的答案差别很大——同一体积分数下导通概率接近翻倍, 问题三的答案相差 0.08 个百分点, 问题四的最低成本相差约 11%。所以这不是一处无关紧要的建模细节, 而是**全题最敏感的口径分歧**; 本文的做法是取最少假设的那一支作答, 同时把另一支的完整结果一并列出, 供按不同理解取用。

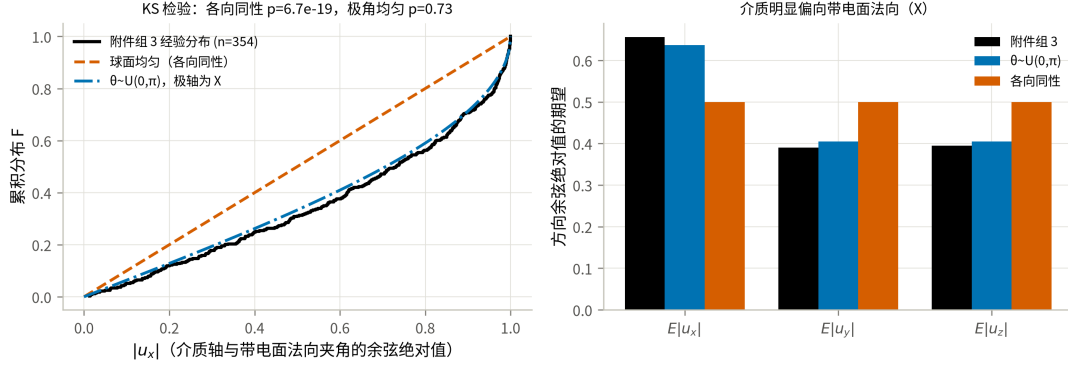


图 3 组 3 介质方向的经验分布与两种候选分布

三、模型假设、符号与判定内核

3.1 模型假设

每条假设后面注明它在哪里被用到；用不到的假设不写。

- **A1 (球柱体近似，并给出严格双侧界)** 介质 A 建模为半径 $r = 30 \text{ nm}$ 、轴长 h 的球柱体（圆柱两端各加一个半球帽），用于 3.3 节把”表面最短距离”化为”轴线段最短距离减 $2r$ ”，从而使判定完全向量化。题面的介质 A 是平端面直圆柱 C ，与球柱体并不相同。本文不再以”体积只差 0.8%”作为理由，而是用几何包含关系把误差夹住：记

$$K^- = \{x : \text{dist}(x, S_{h-2r}) \leq r\}, \quad K^+ = \{x : \text{dist}(x, S_h) \leq r\},$$

其中 S_ℓ 是长为 ℓ 的轴线段，则 $K^- \subseteq C \subseteq K^+$ 。（内界证明：若 $x = p + v$ ， $p \in S_{h-2r}$ 、 $|v| \leq r$ ，把 v 分解为轴向与径向分量，轴向坐标满足 $|t_p + v_{\parallel}| \leq (h/2 - r) + r = h/2$ ，径向满足 $|v_{\perp}| \leq r$ ，故 $x \in C$ 。）包含关系对介质—介质与介质—带电面两类判定同时成立，于是

$$P(K^-) \leq P(\text{真实圆柱}) \leq P(K^+).$$

两端都只是把轴长换成 $h - 2r = 4940$ 或 $h = 5000$ ，复用同一套内核即可算出。第 8.4 节给出了双侧界的数值，并据此修正了问题二、三的答案表述。

- **A2 (碎片独立)** 边界截断产生的每个碎片是独立导体，同一根母介质的不同碎片之间没有电学连接。用于 3.4 节建图。

这条假设与题面的一句话表面上冲突，必须正面交代。题面在描述极化导电时写道：

若现在周围存在已经带电的导电介质 n 且 m 与 n 之间的最短距离小于等于 1.8nm ，则 m 由于极化作用**也全部带电**。

按字面读，一根被截断成两段的母介质仍是”一个介质 m “；理应整体带电，即 A2 的反面。本文仍取 A2，理由有二，且都是可检验的：

1. **反面口径使题目失去区分度**。若碎片粘合，任何一根跨越 X 边界的介质都会同时贴住左右带电面，微构体必然导通。实测：该口径下问题一的三组**全部导通**（含实际不导通的组 1）， $\varphi = 0.70\%$ 处 4000 次试验 4000 次导通、 $P = 1.000$ （第 8.2 节表 19）。问题二至四同时退化为平凡问题。
2. **附件的事实站在 A2 一边**。组 1 在附件中是不导通的，而它恰好含有跨越 X 边界的介质。只有在碎片独立的口径下，组 1 才可能不导通。

也就是说，题面那句话描述的是”两个介质之间的极化传导”，而截断是**放置阶段**为保持体积分数的几何约定；把落在盒内的每一段当作独立导体，才能让三个微构体给出互不相同的答案。这是本文对该句的读法，第 4.3 节末段给出完整的反证。

- **A3（硬壁）**四个绝缘面不导电，介质之间的距离按普通欧氏距离计算，不跨绝缘面配对。与 A2 一致：截断只是保持体积分数的放置约定，不意味着盒子在电学上是周期的。
- **A4（均匀各向随机）**介质中心在微构体内均匀分布；方向在单位球面上各向同性，即 $u = (\sqrt{1 - Z^2} \cos \Phi, \sqrt{1 - Z^2} \sin \Phi, Z)$ ， $Z \sim U[-1, 1]$ 、 $\Phi \sim U[0, 2\pi)$ （等价于 $|u_x| \sim U(0, 1)$ ）。用于问题二至四的随机生成。这是题面”方向随机”在未指定分布时的最少假设读法。附件组 3 的实测分布并非各向同性（第 2.4 节），本文把它作为灵敏度口径在第 8.2 节并列给出，不作为主口径。不考虑重力与介质间碰撞（题面给定）。
- **A5（判据对称）**导通判据 $\delta = 1.8 \text{ nm}$ 对介质-介质与介质-带电面一视同仁（题面给定）。

3.2 符号说明

符号	含义	单位
----	----	----

表 3 符号说明

符号	含义	单位
L	微构体边长, $L = 10000$	nm
r, h	介质 A 的半径与轴长, $r = 30, h = 5000$	nm
R	介质 B 的半径, $R = 200$	nm
δ	导通判据的最大表面间距, $\delta = 1.8$	nm
N_A, N_B	介质 A 的根数、介质 B 的颗 数	—
φ_A, φ_B	两种介质各自的体积分数	—
V_A, V_B	单个介质体积, $V_A = \pi r^2 h, V_B = \frac{4}{3}\pi R^3$	nm ³
c_A, c_B	单个介质成本	元
S_i	第 i 个碎片的轴线段	—
P	微构体导通概率	—
$\hat{P}, T$	导通频率估计、蒙特卡洛试 验次数	—

3.3 边界截断与最短距离

边界截断. 设介质 A 的轴线段为 $p + t(q - p), t \in [0, 1]$ 。对每个坐标轴 j 求出线段穿越周期格点平面 $x_j = L/2 + kL (k \in \mathbb{Z})$ 的所有参数 t , 以这些 t 为断点把线段切开; 每一子段整体位于同一个周期像内, 把它平移 $-\lfloor \cdot \rfloor L$ 即回到微构体内。该过程对介质 A 的**轴线**与题面”把超出部分沿越界方向的反方向平移一个边长, 必要时重复”逐字等价, 且自然支持需要多次平移的情形 (第 8.1 节 T2 用题面给出的算例逐坐标核对过)。

需要标明两处未按字面实现的地方, 以免”逐字等价”被读得过宽: (i) 截断只作用于轴线, 半径 $r = 30$ nm 的柱面在垂直越界时最多有 30 nm 伸出盒外未被折回, 相对 10000 nm 的盒边是 3×10^{-3} 量级, 对判定无实质影响; (ii) 介质 B 的越界处理是近似的——被边界切开的球按整球计, 未按题面切成球缺再折回, 第 9.2 节第 3 条给出该近似的量化对照。

最短距离. 在假设 A1 下, 两个介质 A 的表面最短距离为

$$d_{\text{surf}}(i, j) = d_{\text{seg}}(S_i, S_j) - 2r, \quad (1)$$

其中 d_{seg} 是两条线段的最短距离，用 Ericson 的线段—线段最近点算法 [7] 精确求出。同理，介质 B 与介质 A、介质 B 与介质 B 之间分别为 $d_{\text{pt-seg}} - (r + R)$ 与 $\|c_i - c_j\| - 2R$ 。介质与带电面 $x = \pm L/2$ 的表面距离由介质在 x 方向的极值减去半径给出。

图 4 把这两条规则画在一起。(a) 说明截断：越界部分沿越界方向的反方向平移一个边长回到盒内，

主段与折回段在几何上属于同一根母介质，但按假设 A2 是**两个彼此独立的导体**。(b) 说明介质 A 之间的判据。(c) 说明介质 B 的作用——它本身跨度有限，价值在于充当桥：由 $d_{\text{pt-seg}} \leq r + R + \delta$ 可知，一颗 B 最多能把轴距 $2(R + r + \delta) = 463.6 \text{ nm}$ 以内的两根 A 连起来，这个数在第 7.2 节解释“为什么便宜的 B 反而用不上”时还会再出现。

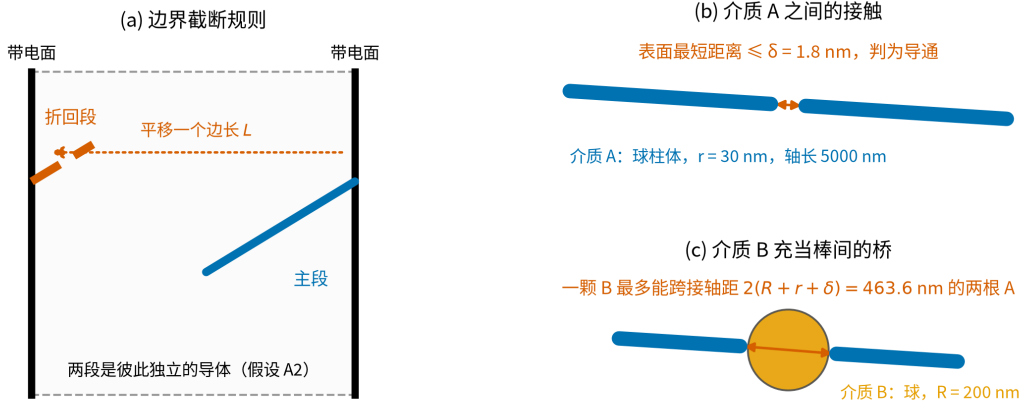


图 4 边界截断规则与导通判据示意

3.4 导通判定

构造无向图 $G = (V, E)$: V 为全部碎片再加两个虚拟节点 L, R (左右带电面)。当两个介质的表面最短距离 $\leq \delta$ 时在其间连边；当某介质与某带电面的表面距离 $\leq \delta$ 时，把它与对应虚拟节点连边。用并查集求连通分量，则

$$\text{微构体导通} \iff \text{find}(L) = \text{find}(R). \quad (2)$$

复杂度. 朴素两两比较为 $O(M^2)$ (M 为碎片数)。实现中对棒—棒用轴对齐包围盒预筛 + 分块计算，对数量可达上万的介质 B 用 KD 树做邻域检索 [7]，把 $\varphi = 1\%$ 规模的单次判定从 744 ms 降到 107 ms，且与朴素实现逐位一致 (第 8.1 节 T6)。

四、问题一：三个微构体的导通判定

4.1 判定口径

按第 2 节的审计结论：组 1、组 2 用 $10000 \times 1000 \times 1000 \text{ nm}$ 的实际几何，组 3 用 10000^3 ；附件给出的每个碎片是一个独立导体（假设 A2）。判定直接使用第 3.4 节的内核，不含任何随机性，结果可逐位复现。

4.2 判定结果

把三组碎片分别送进判定内核，得到表 4。三组的母介质数、周期盒与体积分数差异都很大，因此不能只看“介质多不多”，还要看接触图是否把左右两个带电面连起来。

表 4 问题一的判定结果与接触图规模

微构体	母介质数	碎片数	截面	体积分数	接触边数	触左/右面	判定
			$Y \times Z / \text{nm}$				
组 1	7	12	1000×1000	0.99%	2	3 / 4	不导通
组 2	28	49	1000×1000	3.96%	27	11 / 11	导通
组 3	354	535	10000×10000	0.50%	166	92 / 90	导通

三组在 X 方向的尺寸都是 10000 nm （带电面法向），故表中只列截面尺寸。

三组的判定机理各不相同，值得逐一说明：

组 1 不导通。7 根介质几乎完全沿 X 轴排列（ $E|u_x| = 0.997$ ），在 $1000 \times 1000 \text{ nm}$ 的细通道里本可以首尾接力，但整个微构体只有 2 条接触边，左侧团簇与右侧团簇之间存在断口。换言之，组 1 缺的不是介质总量（体积分数 0.99% 反而高于组 3），而是介质在 X 方向上的接力。

组 2 导通。同样的细通道几何，介质增加到 28 根、体积分数 3.96%，接触边增至 27 条，左右两侧的团簇被打通，存在贯穿通路。

组 3 导通。体积分数只有 0.50%，但微构体截面是组 1、组 2 的 100 倍，92 个碎片触及左带电面、90 个触及右带电面，且每根介质长 5000 nm （半个盒边），只需 2–3 根接力即可跨越，因而在这一体积分数下已经越过渗流阈值。按第 5 节主口径的结果， $\varphi = 0.50\%$ 的导通概率只有 0.0874——组 3 在主口径下属于偏罕见的样本；而在附件标定的灵敏度口径下该概率为 0.2193，组 3 就自然得多。这一条是对主口径的已知反证，第 5.2 节第 4 点与第 9.2 节第 1 条如实展开，此处不回避。

接触图中最近的一对介质表面距离为负（组 1 为 -25.4 nm ，组 3 为 -59.8 nm ），

即存在相互贯穿的介质，与题面”允许介质因随机生成而产生相互贯穿、重叠”一致。

把三组的接触图投影到 X - Y 平面即得图 5。着色口径需要说明：图中的团簇**只由碎片之间的接触定义**，不含判定时用的两个虚拟电极节点——那两个节点会把所有触及同一带电面的碎片（哪怕彼此毫无接触）合并成一块，是判定用的辅助结构而非物理团簇，用它着色会凭空造出一个巨团簇。按物理口径着色后，同时触及左右两面的团簇才涂成绿色，于是：

- **组 1** 左侧三个团簇（橙）与右侧三个团簇（蓝）各自成片，中间留有断口，**没有绿色**；
- **组 2** 有一个 5/49 个碎片的贯通团簇；
- **组 3** 的贯通团簇只含 **17/535 个碎片（3.2%）**——它是一条细长的链，横穿盒子却几乎不占体积，周围大量橙色与蓝色团簇分别挂在左右两个带电面上却互不相连。

组 3 这一张图正是第 5.3 节结论的直接图示：**贯通靠的是一条短链，而不是一个吞掉大半碎片的巨团簇**。三组的绿色有无与表 4 的判定一致。

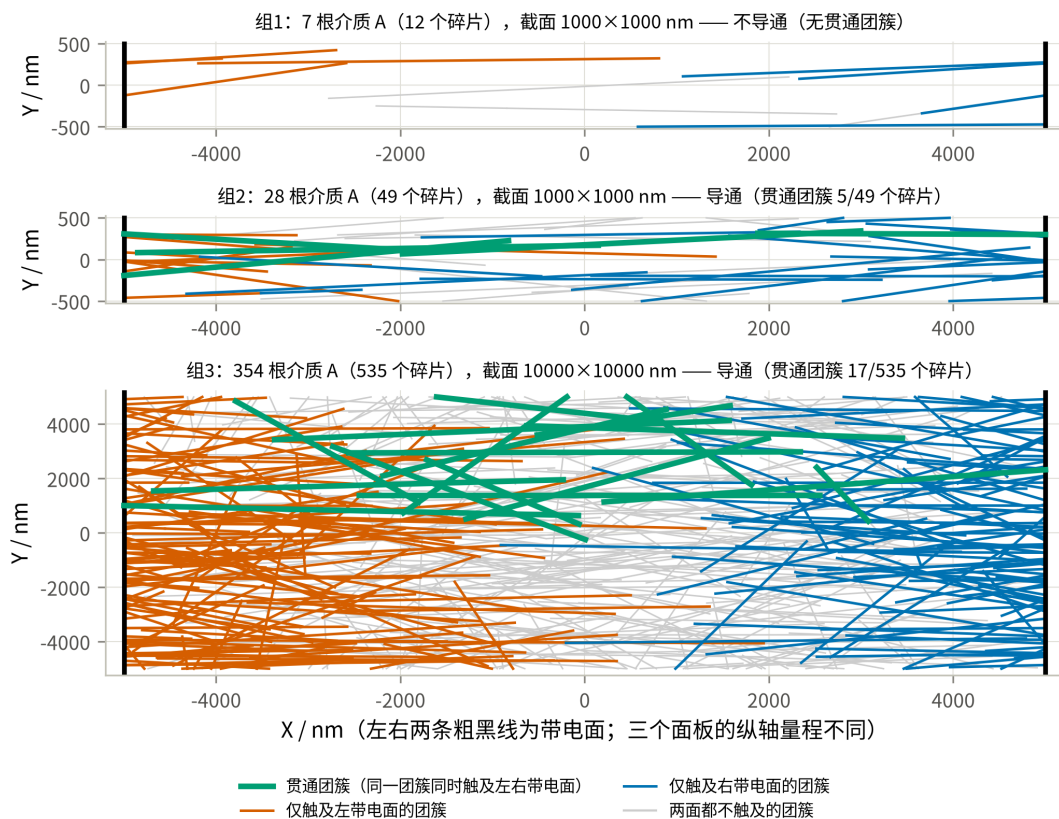


图 5 三个微构体的接触图在 X - Y 平面的投影

4.3 结论的稳健性

三个判定都不是”卡在阈值上”的结论，两项检查支持这一点：

1. **补回被丢弃的碎片。**按第 2.3 节把每根母介质的轴线还原为完整的 5000 nm 再重新截断，碎片数由 12/49/535 增至 15/53/589；三组的判定结论**均不变**。
2. **判据阈值扰动。**把 δ 在 $\pm 50\%$ 范围内扫描，三组结论同样不变：

表 5 问题一判定对判据阈值的稳健性

δ / nm	0.90	1.35	1.80	2.25	2.70
组 1	不导通	不导通	不导通	不导通	不导通
组 2	导通	导通	导通	导通	导通
组 3	导通	导通	导通	导通	导通

这说明三组都不处在判定的临界状态：组 1 的断口宽度远大于 δ 的量级，组 2、组 3 的贯通通路也不是靠某一条刚好卡在 1.8 nm 的接触边撑起来的。

3. **两种数据口径给出同一判定。**题面的逐字读法是”每个微构体都是 10000^3 立方体、附件每行就是一根介质 A”，本文第 2 节则反推出”每行是碎片、组 1 与组 2 的截面为 $1000 \times 1000 \text{ nm}$ ”。这两种读法的差别**不进入连通性计算**：判定内核只用到三样东西——附件给出的线段、接触判据 δ 、以及 $x = \pm 5000$ 的两个带电面；Y、Z 方向的盒宽和”一行算一根还是算一个碎片”都不影响任何一条接触边。因此**两种口径下的三个判定结论完全相同**，分歧只体现在体积分数的折算上：

表 6 三种数据口径下的体积分数（判定结论均相同）

微构体	逐字口径 A：每 行一根 整根 5000 nm, 10000^3	逐字口径 B：每 行一根、按 该行 实际长度 , 10000^3	本文口径：碎片 还原 + 实测周 期盒	判定
组 1	0.017%	0.0097%	0.99%	不导通
组 2	0.069%	0.0394%	3.96%	导通
组 3	0.756%	0.4965%	0.50%	导通

列出两种逐字口径是因为它们并不等价：口径 A 用整根圆柱体积乘行数，可附件里绝大多数行本来就不足 5000 nm，所以它高估了实际占据的体积；口径 B 按各行实际长度折算，才是“每行一根”这一读法下真正占的体积。**本文口径与口径 B 在组 3 上几乎相同**（0.50% 对 0.4965%），差别仅来自被丢弃的短碎片。因此”组 3 的体积分数对上 0.50%”**不足以单独判定两种读法——真正有区分力的证据是第 2.1 节的第 3 条**（还原后重新截断逐坐标复现附件）与”354 根母介质与 $\text{round}(0.50\%)$ 分毫不差”。本文据这两条采用还原口径；而无论采用哪一种，问题一的三个判定结论都不变。

有一处结论确实随口径改变，须明说：本文多处引用”组 1 的体积分数 (0.99%) 反而高于组 3 (0.50%)，却不导通”这一对比。该不等号只在本文口径下成立；按逐字口径 A 是 $0.017\% < 0.756\%$ ，按口径 B 是 $0.0097\% < 0.4965\%$ ，**不等号反向**。因此这句话应当理解为”在本文口径下”的表述，它说明的是介质**排布**（接力与否）比介质**总量**更决定导通——这个机理结论本身不依赖口径：组 1 只有 2 条接触边、中段存在断口，这是三种口径下都成立的事实。

需要说明的是，若改用被否决的”碎片粘合”口径（假设 A2 的反面），组 1 也会变成导通——因为组 1 中存在跨越 X 边界的介质，其两个碎片分别贴在左、右带电面上。三组结论将全部变成”导通”，问题一失去区分度。这从反面支持了假设 A2。

五、问题二：给定体积分数下的导通概率

5.1 模型

给定介质 A 的体积分数 φ ，根数按题目要求四舍五入取

$$N_A = \text{round}(\varphi L^3 / V_A), \quad V_A = \pi r^2 h = 1.41372 \times 10^7 \text{ nm}^3. \quad (3)$$

一次试验：按假设 A4 随机生成 N_A 根介质 A，做边界截断，用第 3.4 节的内核判定导通与否。这是蒙特卡洛方法的标准用法——把一个没有解析表达式的概率，转化为对可判定事件的重复抽样 [3]。重复 T 次独立试验，导通频率 $\hat{P} = X/T$ 即为导通概率的无偏估计，其中 $X \sim B(T, P)$ 。区间估计用二项比例的 Wilson 区间 [4] 而非正态近似——当 $\hat{P}$ 接近 0 或 1 时， $\hat{P} \pm 1.96 \text{ se}$ 会给出越界的区间，Wilson 区间不会：

$$\frac{\hat{P} + \frac{z^2}{2T} \pm z \sqrt{\hat{P}(1 - \hat{P})/T + z^2/(4T^2)}}{1 + z^2/T}, \quad z = 1.96. \quad (4)$$

5.2 结果

每档做 $T = 8000$ 次独立试验。主口径（假设 A4 的球面均匀方向）结果如下。需要先说明一点：下表的 $\hat{P}$ 是在球柱体口径 K^+ 下算出的，按假设 A1 它是**真实平端面圆柱导通概率的上界**；第 8.4 节给出了对应的下界。

表 7 四档体积分数下的微构体导通概率 ($T = 8000$, 括号内为 Wilson 95% 区间)

体积分数 φ	介质 A 根数 N_A	导通次数	导通概率 $\hat{P}$	95% 置信区间
0.50%	354	699	0.0874	(0.0814, 0.0938)
0.60%	424	1860	0.2325	(0.2234, 0.2419)
0.70%	495	3929	0.4911	(0.4802, 0.5021)
1.00%	707	7952	0.9940	(0.9921, 0.9955)

作为灵敏度口径, 若改用附件组 3 标定的极角均匀分布生成方向, 同样 8000 次试验给出

表 8 附件标定方向口径下的对照结果

体积分数	0.50%	0.60%	0.70%	1.00%
$\hat{P}$ (球面均匀, 主口径)	0.0874	0.2325	0.4911	0.9940
$\hat{P}$ (附件标定, 灵敏度口径)	0.2193	0.4798	0.7458	0.9994

把两套口径下的全部估计点连成曲线即图 6: 带误差棒的标记是题目点名的四档, 误差棒为 Wilson 95% 区间 (在 8000 次试验下已窄到与标记同量级); 曲线上其余点来自问题三的粗扫与细网格; 两个星号是第 6 节求得的使 $P \geq 90\%$ 的最低体积分数。

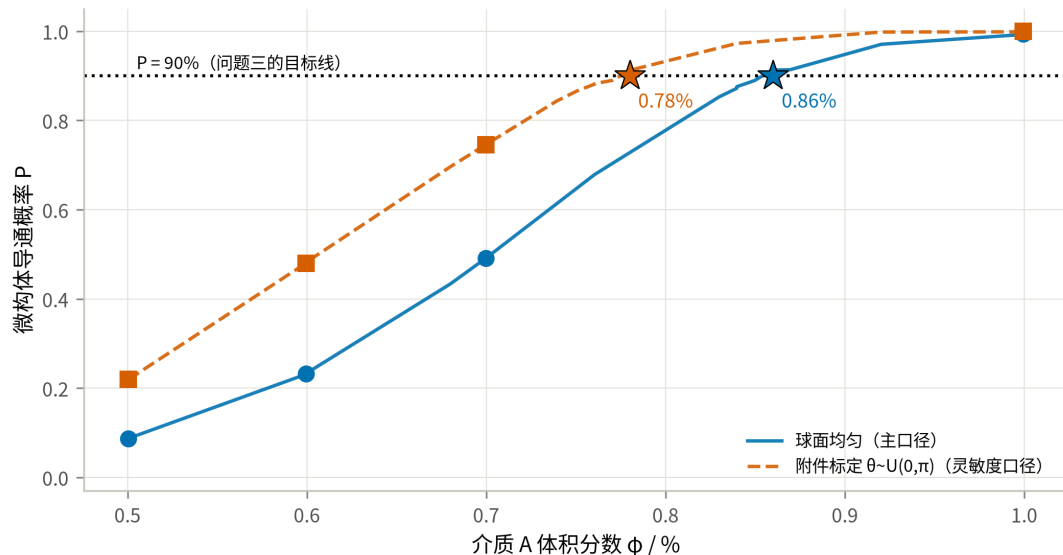


图 6 导通概率随介质 A 体积分数的变化

结果解读.

1. 概率在 0.50%→1.00% 之间由 0.087 升到 0.994, 跨越了一个完整的**渗流跃变**: 在陡峭段上, 体积分数每增加 0.10% (约 71 根介质), 概率就上升约 0.15–0.26

(0.087→0.233→0.491)。微构体的导通不是”介质越多导电性越好”的连续过程，而是在一个很窄的窗口里从几乎不通变成几乎必通。

2. **与渗流理论的独立对照。**经典排除体积判据 (Balberg 等 [6]) 给出随机取向细长杆的阈值满足 $n_c \langle V_{ex} \rangle \approx B_c \approx 1.4$ 。球柱体的平均排除体积为 $\langle V_{ex} \rangle = \frac{4\pi}{3} D^3 + 2\pi D^2 \ell + \frac{\pi}{2} D \ell^2$ ，代入本题参数（连通等效直径 $D = 2r + \delta = 61.8 \text{ nm}$ 、轴长 $\ell = 5000 \text{ nm}$ ）得 $\langle V_{ex} \rangle = 2.548 \times 10^9 \text{ nm}^3$ ，进而 $N_c \approx 549$ 根、 $\varphi_c \approx 0.777\%$ 。三个数排成一条可解释的链，见表 9。

表 9 跃变中点位置的三种估计及其差异来源

来源	φ at $P = 0.5$	与上一行的差异来自
排除体积判据（各向同性、无限大体系）	0.777%	—
本文仿真（主口径：各向同性、有限盒）	0.705%	有限尺寸效应：介质长 5000 nm 恰为盒边一半，跨越只需 2-3 根接力
灵敏度口径（附件标定方向、有限盒）	0.607%	方向偏向带电面法向，进一步降低阈值

主口径与理论判据用的是同一套方向假设（各向同性），二者相差 9%，方向一致（有限体系阈值更低），说明仿真内核没有系统性偏差——这也是主口径的一个好处：它可以与解析判据直接对照，而附件标定口径无法。第三行的进一步下降则由方向偏向解释，其中”有限尺寸效应”这一步在第 5.3 节有直接的结构证据，而不只是一句合理的猜测。

3. **方向分布的影响远大于统计误差。**表 8 中两套口径在 0.50%、0.60%、0.70% 三档上分别相差 0.132、0.247、0.255，而 8000 次试验的 95% 区间半宽只有约 0.01。也就是说，这一差别不是”算得不够准”，而是**建模口径的选择**：它比本文其它任何一处不确定性都大一个量级，因此两套结果全程并列，而不是只报一套。

4. 需要留一条与主口径不完全相符的观察：附件组 3 恰好是 $\varphi = 0.50\%$ 的一个构型（354 根母介质，与 $\varphi = 0.50\%$ 的取整结果分毫不差），而它是导通的。在主口径下这是一个概率 0.087 的事件——不至于不可能，但偏罕见；在附件标定口径下概率为 0.219，更自然。这一点连同第 2.4 节的 KS 检验，构成对主口径的**已知反证**，本文如实列出而不回避：若组委会的参考答案按附件的生成过程出题，则应取第 8.2 节的灵敏度口径结果。

5.3 渗流的结构证据：为什么阈值低于无限大体系的估计

导通概率只回答”通没通”，看不出通的时候内部长什么样。渗流理论里刻画这件事的量是**序参量** $S_{\max}$ ——最大连通团簇所含碎片占全部碎片的比例。它的计算完全

不涉及带电面，因此是一条独立于导通判定的检查。图 7 把两者画在一起（每档 200 次试验）：

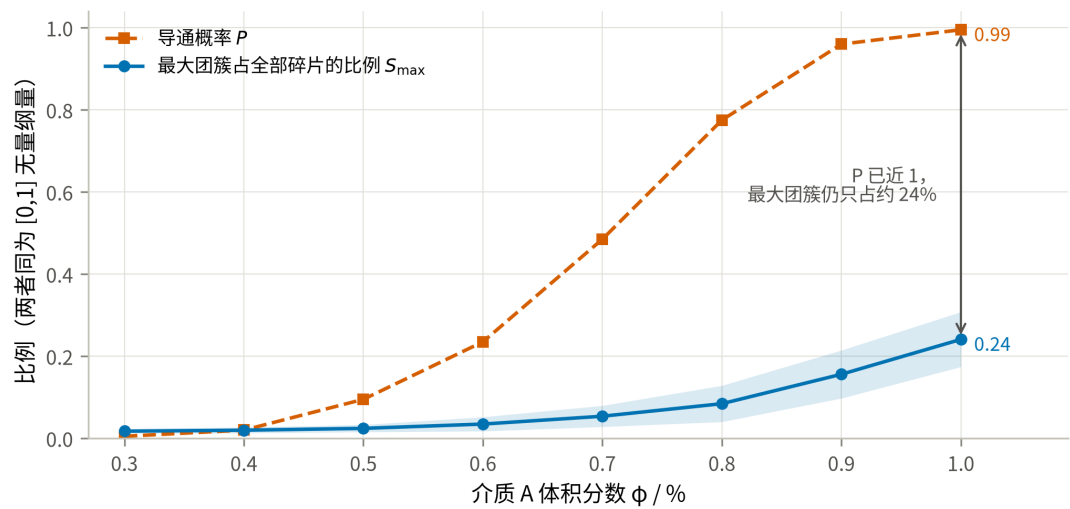


图 7 最大团簇占比与导通概率随体积分数的变化

结果出乎”经典渗流”的直觉，却正好解释了第 5.2 节留下的疑问：当 P 已经接近 1 时，最大团簇仍然只包含约 24% 的碎片（ $\varphi = 1.00\%$ 时 $S_{\max} = 0.241$ ，团簇总数仍有 637 个）；在 P 跃变最陡的 $\varphi = 0.70\%$ 处， $S_{\max}$ 只有 0.054。也就是说，本题的微构体在尚未形成贯穿全系统的巨团簇时就已经导通了。

这里要说明一处统计口径，否则论证会有漏洞：上面的 $S_{\max}$ 是对全部试验取的平均，其中混有大量未导通的实现，而本节要论证的是”导通时内部长什么样”，严格地说需要的是条件期望 $E[S_{\max} \mid \text{导通}]$ 。两者在跃变段并不相同，故一并算出（同为 $T = 200$ ）：

表 10 最大团簇占比的无条件均值与条件均值（ $T = 200$ ）

φ	P	导通的实现		$E[S_{\max} \mid$	$E[S_{\max} \mid$
		数	$E[S_{\max}]$	导通]	不导通]
0.50%	0.095	19	0.0242	0.0265	0.0240
0.70%	0.485	97	0.0538	0.0646	0.0436
1.00%	0.995	199	0.2410	0.2419	0.0605

条件化把 $\varphi = 0.70\%$ 处的 0.054 抬到 0.065，把 $\varphi = 1.00\%$ 处的 0.241 抬到 0.242，**结论不变**：即便只看导通的那些实现，最大团簇也只占 6.5%（跃变最陡处）到 24.2%（ $P \approx 1$ 处）。下文与图 7 仍按无条件均值叙述，因为两者在 $P \approx 1$ 处只差 0.0009；但跃变段的比较应以条件均值为准。

附件组 3 是这条结论的一个具体样本（图 5）：那是 $\varphi = 0.50\%$ 的一个**导通**构型，其贯通团簇只含 17/535 个碎片（3.2%）——一条细链，而不是巨团簇。

原因是几何尺度：介质 A 的轴长 5000 nm 恰为盒边的一半，跨越两个带电面只需要 2-3 根接力。贯通所需的是一条**短链**，而不是一个吞掉大部分介质的巨团簇。这正是有限尺寸效应的直接体现，也解释了为什么第 5.2 节中主口径的仿真阈值（0.705%）低于排除体积判据对无限大体系的估计（0.777%）——两者用的是同一套各向同性方向假设，差别只在体系大小；后者刻画的是巨团簇的出现，而本题只需要短链贯通，因此更早发生。

这条结论也提醒：不能把本题的阈值直接外推到更大的微构体。盒边增大而介质尺寸不变时，贯通需要的链长随之增加，阈值会向 0.777% 靠拢。

六、问题三：使导通概率不低于 90% 的最低体积分数

6.1 求解思路

要求”精确到百分号下小数点后两位”，即在 0.01% 的体积分数网格上定位。0.01% 对应 $\Delta N_A = L^3 \times 10^{-4} / V_A \approx 7.07$ 根，相邻网格点的导通概率相差约 0.02，因此蒙特卡洛的 95% 区间半宽必须明显小于 0.02。分两步：

1. **粗定位**. 在 $\varphi \in [0.60\%, 1.00\%]$ 上取 6 个点，各 2000 次试验，用二项似然拟合 $\text{logit } P = a + bN_A$ （广义线性模型与二项似然见 [2][4]），反解 $P = 0.90$ 的穿越点；
2. **网格复算**. 在穿越点附近的 5 个 0.01% 网格点上各做 4000 次试验（区间半宽约 0.009），取**最小的、点估计不低于 0.90 的**网格点作为答案，再用 12000 次试验独立复核。

拟合只用于省算力，最终结论由第 2 步的直接频率给出，不依赖 logit 模型的函数形式。

6.2 结果

粗定位给出 $P = 0.90$ 的穿越点在 $N_A \approx 604$ ($\varphi \approx 0.854\%$)。在其附近的 0.01% 网格上各做 4000 次试验：

表 11 问题三的 0.01% 网格复算 (主口径: 球面均匀, $T = 4000$)

体积分数 φ	N_A	导通概率 $\hat{P}$	95% 置信区间	是否达标
0.83%	587	0.8525	(0.8412, 0.8632)	否
0.84%	594	0.8720	(0.8613, 0.8820)	否
0.85%	601	0.8892	(0.8791, 0.8986)	否
0.86%	608	0.9127	(0.9036, 0.9211)	是
0.87%	615	0.9135	(0.9044, 0.9218)	是 (更贵)

在球柱体口径 K^+ 下, 最低填充量为 $\varphi = 0.86\%$ (对应 608 根介质 A)。但 K^+ 只给出真实圆柱导通概率的上界, 因此 0.86% 是真实答案的下界而非答案本身。第 8.4 节用内界 K^- 重算同一网格后得到:

$$\varphi_{\min}(\text{真实圆柱}) \in [0.86\%, 0.89\%],$$

其中 0.89%(630 根)是能被证明达标的最小网格值(内界在该点已达 $P = 0.9202 \geq 0.90$, 而 0.88% 处内界只有 0.8993、其 95% 区间 0.8915–0.9067 横跨 0.90, 尚不能证明达标)。若必须给出单一数值, 工程上应取保守值:

答案: 介质 A 的最低填充量取 $\varphi = 0.89\%$ (630 根), 可保证导通概率不低于 90%; 若采用球柱体近似则为 0.86% (608 根)。

夹逼带宽比灵敏度口径下更宽 (3 格 vs 2 格), 原因是球面均匀方向下概率曲线在阈值附近更平缓: 同样 0.01% 的体积分数增量只带来约 0.02 的概率提升, 因此内外界之间需要更多网格才能拉开。

独立复核: 在 $\varphi = 0.86\%$ 用另一组随机种子做 12000 次试验, 得 $\hat{P} = 0.9062$, 95% 置信区间 (0.9008, 0.9113), 区间下界仍高于 0.90, 结论成立。相邻的 0.85% 在两次独立估计中都明显低于 0.90 (0.8892, 区间上界 0.8986), 因此 0.86% 是 0.01% 网格上能达标的**最小**取值, 而不是四舍五入的结果。

按附件标定方向的灵敏度口径重做同一流程, 答案为 $\varphi = 0.78\%$ (552 根, 复核 $\hat{P} = 0.9092$, 区间 (0.9039, 0.9142))。两套口径相差 0.08 个百分点, 即约 56 根介质, 仍然是全题最大的不确定来源。

按主口径的成本换算, 纯用介质 A 达到 90% 导通概率的代价是 $608 \times c_A = 9.025$ 元, 这是问题四的比较基准。

七、问题四：A、B 混填的最低成本配比

7.1 优化模型

单个介质的成本由体积换算 ($1 \mu\text{m}^3 = 10^9 \text{ nm}^3$):

$$c_A = 1.05 \times \frac{V_A}{10^9} = 1.48441 \times 10^{-2} \text{ 元/根}, \quad c_B = 0.05 \times \frac{V_B}{10^9} = 1.67552 \times 10^{-3} \text{ 元/颗}. \quad (5)$$

优化问题为

$$\min_{N_A, N_B \in \mathbb{Z}_{\geq 0}} C = c_A N_A + c_B N_B \quad \text{s.t.} \quad P(N_A, N_B) \geq 0.90. \quad (6)$$

结构性质. 向已有配置中再加入一个介质, 只会给接触图增加一个顶点和若干条边, 不会删去任何已存在的通路, 故 P 对 N_A 、 N_B 都单调不减。于是可行域是”右上封闭”的, 最优解必落在可行边界 $N_B = g(N_A)$ 上, 二维搜索降为沿边界的一维搜索。

求解策略. P 没有解析式, 只能用仿真估计, 直接对每个 N_A 做二分需要上百万次仿真。故采用两阶段:

- **阶段 A (代理模型).** 在 $N_A \times N_B$ 的 8×6 网格上各做 700 次试验, 用二项似然拟合 $\text{logit } P = w_0 + w_1 N_A + w_2 N_B + w_3 \sqrt{N_B} + w_4 N_A N_B / 1000$ 。其中 $\sqrt{N_B}$ 项刻画球的边际收益递减, 交互项刻画”球只有在有棒可搭时才有用”的协同效应。
- **阶段 B (沿边界搜索 + 直接验证).** 用代理模型给出边界与成本排序, 取成本最低的若干候选点, 用 6000 次试验直接验证; 若验证不达标则向上补 N_B 直到真正可行。最终结论以直接验证的频率为准, 代理模型只用于缩小搜索范围。

7.2 结果

阶段 A: 概率曲面. 8×6 网格 ($N_A \in [150, 670]$, $N_B \in [0, 3200]$) 各 700 次试验, 代理模型对网格点的最大绝对残差为 0.030, 足以用来定位边界。网格本身已经说明了两种介质的分工:

表 12 (N_A, N_B) 网格上的导通概率 (节选, $T = 700$)

$N_A \setminus N_B$	0	300	700	1200	2000	3200
250	0.006	0.021	0.020	0.039	0.089	0.283
400	0.179	0.194	0.314	0.479	0.763	0.983
480	0.444	0.554	0.690	0.836	0.974	0.999
550	0.767	0.827	0.927	0.977	1.000	1.000
610	0.907	0.947	0.981	0.999	1.000	1.000
670	0.974	0.994	0.999	1.000	1.000	1.000

阶段 B、C：沿边界搜索与验证。代理模型给出的可行边界上，成本最低的五个候选依次是 (604, 0)、(564, 456)、(524, 878)、(484, 1341)、(644, 0)，对应成本 8.966、9.136、9.249、9.431、9.560 元。阶段 C 对每个候选做 6000 次直接验证；若不达标则向上补 N_B 直到真正可行，因此候选的**实测成本只会不小于上表**。

这里必须补上一个候选：**问题三**的**纯 A 方案** (608, 0) **本身就是可行点，必须与混填候选一同参与比较**。代理模型按自己的网格取 N_A ，未必落在问题三那一格上；本题它取到 $N_A = 604$ ，而 (604, 0) 实测 $\hat{P} = 0.8943$ 不可行，补到 (604, 60) 才达标、成本 9.066 元——反而比”多放 4 根 A、一颗 B 都不放”的 (608, 0) (9.025 元) 更贵。把纯 A 方案漏在比较之外，就会把一个更贵的方案报成最优。

表 13 候选点的大样本验证 ($T = 6000$)

N_A	N_B	导通概率 $\hat{P}$	95% 置信区		是否可行
			间	成本 / 元	
608	0	0.9130	(0.9056, 0.9199)	9.0252	是 (可行性已确认)
604	0	0.8943	(0.8863, 0.9019)	8.9658	否 (未达 0.90)
604	60	0.9083	(0.9008, 0.9154)	9.0663	是，但更贵
564	456	0.8998	(0.8920, 0.9072)	9.1361	否
564	516	0.9127	(0.9053, 0.9195)	9.2366	是，但更贵
524	878	0.8907	(0.8825, 0.8983)	9.2494	否

N_A	N_B	导通概率 $\hat{P}$	95% 置信区	成本 / 元	是否可行
			间		
524	938	0.9052	(0.8975, 0.9123)	9.3499	是, 但更贵
484	1341	0.8797	(0.8712, 0.8877)	9.4314	否
484	1421	0.9010	(0.8932, 0.9083)	9.5654	是, 但更贵
644	0	0.9627	(0.9576, 0.9672)	9.5596	是, 但更贵

阶段 C 的**全部**探测点都列在表 13 里, 包括未通过的四个——它们由 `solve_p4.py` 直接写入 `results/p4_cost_optimum.json` 的 `all_probes` 字段, 不只留在运行日志里, 因此表中每一行都可溯源。 `p4_backfill_probes.py` 另用同一随机种子把未通过点重算一遍, 所得 $\hat{P}$ 与原始运行逐位相同, 这同时验证了整套仿真的可复现性: 随机数由 `SeedSequence` 按 `worker` 分叉, 结果与并行度和运行时刻都无关。

答案: 最优填充量为 $N_A = 608$ 根介质 A、 $N_B = 0$ 颗介质 B, 即 $\varphi_A = 0.86\%$ 、 $\varphi_B = 0$, 导通概率 0.9130, 总成本 9.0252 元。也就是说, 在题目给定的价格下, **掺入介质 B 并不划算, 最低成本方案就是问题三的纯 A 方案。**(第 7.3 节将用单调性穷举审计这一结论, 并给出它需要收紧到什么程度。)

把阶段 A 的网格画成概率曲面即图 8。红线是 $P = 90\%$ 的可行边界, 白线是等成本线(斜率由 c_A/c_B 决定)。两族线的斜率不同 (-11.32 对 -8.86), 因此它们是**相交**而非相切: 最优解落在可行边界与坐标轴 $N_B = 0$ 的交点上, 这是线性目标加单调约束下典型的**角点解**(corner solution), 而不是内点最优的相切条件。空心圆是阶段 C 直接验证过的边界点, 星号是推荐配比。

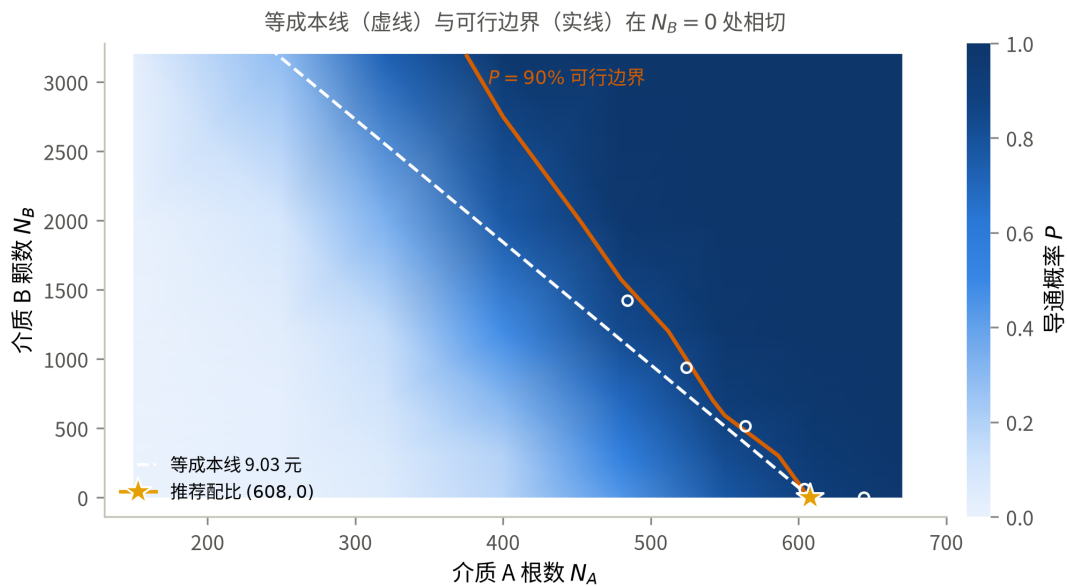


图 8 (N_A, N_B) 平面上的导通概率、可行边界与等成本线

为什么便宜的介质 B 反而用不上？关键不是单价，而是每元钱换来多少导通概率。把网格上两种介质的边际收益都折算成“每元的 ΔP ”（ $N_B = 0$ 处的差分， $T = 700$ ，故有约 ± 0.02 的抽样噪声）：

表 14 两种介质的边际效率 (ΔP / 元)

N_A	150	250	320	400	480	550	610	670
纯 A 时的 P	0.000	0.006	0.054	0.179	0.444	0.767	0.907	0.974
加介质 A	—	0.004	0.047	0.105	0.224	0.311	0.157	0.075
加介质 B	0.000	0.031	0.020	0.031	0.219	0.119	0.080	0.040

两条效率曲线在 $N_A \approx 300$ 附近交叉：介质稀疏时 B 更划算，接近渗流阈值时 A 更划算。机理是清楚的——半径 200 nm 的球只能把相距不到约 464 nm 的两根棒接起来，作用是“补桥”；而长 5000 nm 的棒本身跨越半个微构体，在阈值附近每多一根，贯通团簇的生长是级联式的。介质 A 的边际效率在 $N_A = 550$ 处达到峰值 0.311，恰在跃变最陡的一段。（ $N_A = 480$ 一列两者几乎持平，那是 $T = 700$ 的抽样噪声，半宽约 ± 0.035 ，不足以分辨。）

而 $P \geq 0.90$ 这个约束恰恰把可行解全部推到交叉点的右侧：即使掺入 3200 颗 B（成本 5.36 元）， $N_A = 400$ 时也只能到 0.983，总成本已达 11.3 元。可行域内 A 始终占优，于是最优解退化到边界的端点 $N_B = 0$ 。

把两条效率曲线画出来即图 9：交叉点左侧 B 每元收益更高，右侧 A 更高，而 $P \geq 0.90$ 要求的 $N_A \geq 608$ 整段落在交叉点右侧——阴影区就是可行域。这一张图把”B 更便宜却用不上”解释完了：不是 B 无用，是约束把可行解推出了 B 的优势区。

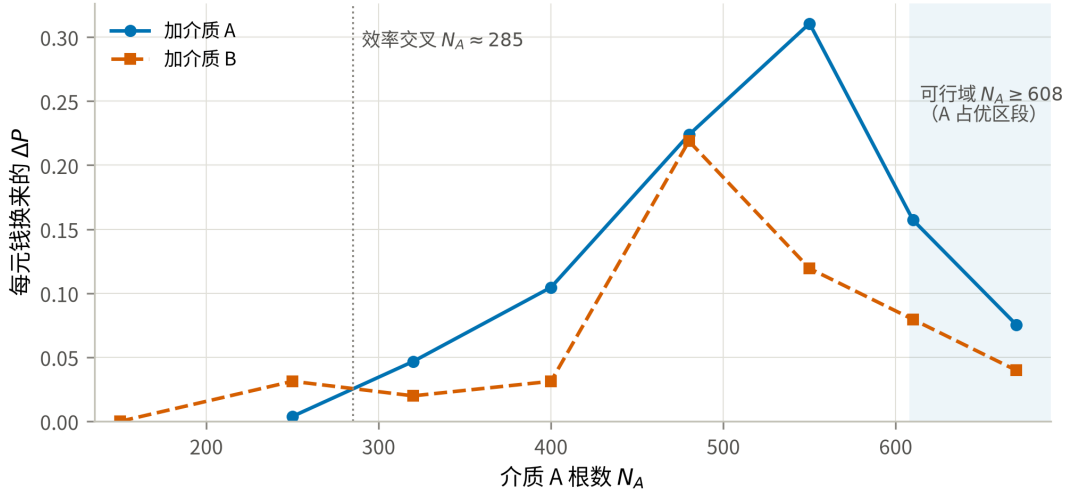


图 9 两种介质每元钱的边际效率及其交叉点

这个结论对价格是敏感的，而且敏感点可以算出来。设可行边界为 $N_B = g(N_A)$ ，则边界上的成本 $C(N_A) = c_A N_A + c_B g(N_A)$ ， $dC/dN_A = c_A + c_B g'$ 。 $g' < 0$ ，故当 $c_B < -c_A/g'$ 时成本随 N_A 增大而上升，此时应当多用 B。盈亏平衡价为 $c_B^* = -c_A/g'$ 。第 8.2 节由已验证的边界点 (484, 1421)、(524, 938)、(564, 516)、(604, 60)、(608, 0) 实测得 $g' = -11.32$ 颗/根，对应 $c_B^* = 0.0391$ 元/ μm^3 ——现价 0.05 元/ μm^3 比它高 27.8%（需降价 21.7% 才持平），因此最优解落在 $N_B = 0$ 。

7.3 最优性的穷举式审计：从”候选中最好”到”可证的成本下界”

第 7.2 节只验证了代理模型排在前面的几个候选点。这不足以证明全局最优：代理模型仅在 8×6 的稀疏网格上训练，训练点残差 0.030 不能约束网格之外的误差，也排除不掉”代理模型根本没看见的更便宜的可行点”。本节用单调性把这件事补成穷举。

审计原理。 记待证成本 $C^* = 9.0252$ 元。要证明它最优，只需说明所有成本低于 C^* 的整数点都不可行。对固定的 N_A ，成本低于 C^* 等价于 $N_B \leq N_B^{\max}(N_A) = [(C^* - c_A N_A)/c_B] - 1$ 。由 P 对 N_B 单调，只要 $P(N_A, N_B^{\max}(N_A)) < 0.90$ ，该 N_A 下所有更便宜的点一并被排除。再取网格 $a_1 < a_2$ ，任意落在区间内且成本低于 C^* 的点都被”角点” $(a_2, N_B^{\max}(a_1))$ 按分量支配，故一次仿真可排除整条区间。判据取 Wilson 上界 < 0.90 ，把蒙特卡洛误差算在不利的一侧。

这里要强调判据的方向，它决定了每一步能主张什么： $P(a, N_B^{\max}(a)) < 0.90$ 不

只是”该列可能没有更便宜的可行点”的充分条件，而是**充要条件**——由 P 对 N_B 单调，该列存在比 C^* 便宜的可行点当且仅当预算线上那个点本身可行。因此**逐列在预算线上取点，跑遍 $[0, 608)$ 的每个整数，结论就是严格的**；角点判据只是用来便宜地批量覆盖大段区间的加速手段，它才是充分而非必要条件。

第一步：角点排除。以步长 24 覆盖 $N_A \in [0, 608]$ 共 26 个区间，其中 20 个被角点判据直接排除，最松的一个上界也只有 0.8803。这一步严格排除了**所有 $N_A \leq 480$ 的更便宜点**。剩余 6 个区间 ($N_A \in [480, 608]$) 角点界过松——步长 24 内预算允许 N_B 多出约 $24c_A/c_B \approx 213$ 颗，仅这一项就足以把上界抬过 0.90，因此角点判据在最优解附近必然失效。这不是缺陷而是该判据的适用边界：它是廉价的**充分条件**，不是紧的条件。

第二步：预算线逐点复核。对未排除区段改在**等成本线上**取点——那里预算买不到任何余量， N_B 恰好取到 $N_B^{\max}(N_A)$ ：

表 15 预算线上的逐点审计

N_A	$N_B^{\max}$	成本 / 元	$\hat{P}$	95% 区间	T	判定
480	1134	9.0252	0.8184	(0.8028, 0.8330)	2500	排除
496	992	9.0247	0.8360	(0.8211, 0.8500)	2500	排除
512	850	9.0243	0.8464	(0.8319, 0.8600)	2500	排除
528	708	9.0239	0.8668	(0.8531, 0.8796)	2500	排除
544	566	9.0235	0.8812	(0.8680, 0.8933)	2500	排除
560	425	9.0247	0.8700	(0.8563, 0.8826)	2500	排除
576	283	9.0243	0.8954	(0.8911, 0.8996)	20000	排除
592	141	9.0239	0.9003	(0.8961, 0.9044)	20000	无法判定

第三步：擦边点必须单独加样本。最后两行的样本量与前六行不同，这一步是必要的而不是修饰。在 $T = 2500$ 下，(576, 283) 的 Wilson 上界是 0.8994，只比 0.90 低 0.0006——名义上”已排除”，实则该判定完全落在噪声里。把它加到 $T = 20000$ 后点估计从 **0.8876 升到 0.8954**，可见 $T = 2500$ 的那次是低估；所幸上界 0.8996

仍在 0.90 之下，排除成立，但余量只剩 0.0004。这个例子说明：在贴着约束的地方，2500 次试验判读的是抽样噪声而非结论，第 8.3 节据此把擦边点的样本量单独定到 $T = 20000$ 。

把表 15 画成图 10 后，证据链一眼可读：所有蓝色点的 Wilson 区间整体压在 0.90 之下，只有最右侧 (592, 141) 的区间被 0.90 穿过。菱形标记是加到 $T = 20000$ 的两点，它们的区间明显更窄——这正是加样本换来的分辨力。

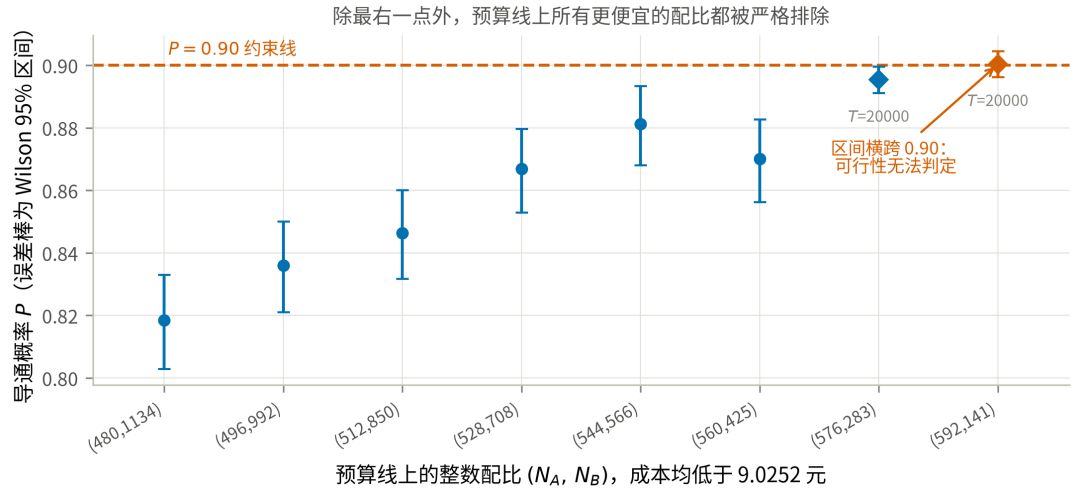


图 10 预算线上逐点审计：P 的点估计与 Wilson 区间

第四步：补上表 15 没有覆盖到的区段。表 15 的取点步长是 16、且止于 $N_A = 592$ ，因此它并没有跑遍每一个整数列： $N_A \in (592, 608)$ 一整段没有取过点，相邻两个取点之间也留有未覆盖的薄片。补跑用 `src/p4_global_audit2.py`：它把判据收紧为 99% Wilson 区间（本审计要做上百次判定，95% 下的族错误率不可忽略），并逐列在预算线上取点——由前述充要性，跑遍每个整数列即得严格结论。表 16 列出其中决定结论的三点：最贴近 C^* 的两列，以及表 13 里那个判据不一致的候选。

表 16 表 15 未覆盖区段的补充审计（99% Wilson 区间）

(N_A, N_B)	成本 / 元	相对 C^*	$\hat{P}$	99% 区间	T	判定
(607, 8)	9.0237	便宜 0.0015	0.9032	(0.8977, 0.9085)	20000	无法判定
(606, 17)	9.0240	便宜 0.0012	0.9023	(0.8968, 0.9076)	20000	无法判定
(604, 0)	8.9658	便宜 0.0594	0.8943	(0.8837, 0.9041)	6000	无法判定

这几行改变了结论的性质。(604, 0) 就是第 7.2 节表 13 里那个“未达 0.90”的

候选：那里判它不可行用的是点估计 0.8943，但本节声明的判据是 Wilson 上界低于 0.90，而它的 95% 上界是 0.9019、99% 上界是 0.9041，都在 0.90 之上（5366/6000，见 results/p4_cost_optimum.json）。按本文自己的标准，它没有被排除，而它比 C^* 便宜 0.0594 元。

结论：最优解是退化的，只能给出成本区间，且下界不能写成 9.0239 元。把三步证据合起来，能主张与不能主张的分别是：

- **能主张：**所有 $N_A \leq 480$ 的更便宜点已被角点判据严格排除； $N_A \in \{496, 512, 528, 544, 560, 576\}$ 六列的更便宜点也已排除。
- **不能主张：**“所有成本低于 9.0239 元的整数点均已被严格排除”——(592, 141)、(607, 8)、(606, 17)、(604, 0) 四点都未被排除，其中 (604, 0) 只要 8.9658 元。
- **根本原因：**在 $P = 0.90$ 附近，成本相差不到 0.7% 的一整段配比，其导通概率与 0.90 的差距小于 $T = 20000$ 能分辨的尺度。要把 0.9032 与 0.90 分开，所需样本量随二者之差的平方反比增长；若真值恰好等于 0.90，则无论多少次试验都分不开。这是**问题本身的退化**，不是算力不足。

因此第 7.2 节“ (608, 0) 是全局最优”应当收紧为：

推荐配比为 $(N_A, N_B) = (608, 0)$ ，总成本 9.0252 元——它是所有探测点中成本最低的、可行性被**确认**的方案（ $T = 6000$ 下 $\hat{P} = 0.9130$ ，Wilson 下界 0.9056 高于 0.90）。**最低总成本报到 9.02 元这个精度**；在此精度之下最优解是退化的：(592, 141)、(607, 8)、(606, 17)、(604, 0) 等更便宜的配比其可行性在 $T = 20000$ 下仍无法确认，既不能采纳也不能排除。换句话说，便宜出来的那一点钱买不到可行性的证据，而工程上要交付的是有证据的方案。

这与第 8.2 节的盈亏平衡分析自洽：介质 B 的现价只比盈亏平衡价高 27.8%，可行边界与等成本线本就接近平行（斜率 -11.32 对 $-c_A/c_B = -8.86$ ），因此最优解在临界段上必然是平的。若 B 再便宜一些（低于 0.0391 元/ μm^3 ），两族线的夹角会反号，天平就会明确倒向混填。

工程含义。数学上的“最低成本”与工程上该选的方案不必是同一个：前者是 9.02 元这个区间，后者是 (608, 0)——单一材料、配料简单，且是区间内唯一能拿出可行性证据的点。若生产上对失效风险更敏感，还应按第 6.2 节的双侧界取 $\varphi = 0.89\%$ (630 根)，把球柱体近似的几何不确定性也算进裕量。

八、模型检验、灵敏度与稳健性

8.1 几何内核的独立校验

论文全部结论建立在两个原语上：线段最短距离与边界截断。这两个错了，后面所有概率都是错的，而且错得很隐蔽。因此在跑任何仿真之前先把它们钉死 (src/validate.py, 全部通过)：

表 17 几何内核校验 (T1–T5 为有明确通过标准的校验)

编号	检验内容	结果
T1	线段最短距离解析解 vs 900×900 暴力采样	最大超出量 2.3×10^{-13} nm
T2	题面边界截断图例的算例逐坐标复核	越界段 (5000, 6000) → (−5000, −4000)，与题面的边界截断图例一致
T3	附件往返测试 ：把碎片还原为母介质轴线后重新截断	组 1/2/3 共 334/334 根完整介质逐坐标一致
T4	导通判定逻辑（贯通/断开/1 nm 间隙/单边/碎片开关）	5 项全部符合预期
T5	分块加速的接触检索 vs 朴素两两比较	三种规模下 逐位一致

T3 最有分量：它同时验证了截断实现、周期盒尺寸判断和碎片还原三件事。若第 2.2 节对组 1、组 2 周期盒的判断有误，T3 不可能对 29 根介质逐坐标复现。

关于球柱体近似的量级估计，本文不再把它当作一项”校验”。早期版本在此列过一项“临界接触中受端帽形状影响者约 3.3%”，但它有两个问题：一是那个 3.3% 是在人造采样分布下（一根棒固定沿 X 轴、另一根棒中心被限制在其轴线周围的细管内）算出的比例，既不是判定翻转的概率，也没有置信度；二是它没有可通过/不通过的标准，把它记作”通过”只是形式上的。第 8.4 节已用严格的几何包含关系给出了**双侧界**，那才是这条近似应有的处理方式，本文以它为准。

8.2 灵敏度分析

全部固定在 $\varphi = 0.70\%$ ($N_A = 495$) ——渗流跃变最陡的位置，对任何扰动都最敏感。

S1 导通判据阈值 δ . $\delta = 1.8$ nm 是题目给定的，扫描它是为了说明结论不是卡在阈值的某个巧合上 ($T = 2000$):

表 18 导通概率对判据阈值的敏感性

δ / nm	0.90	1.35	1.80	2.25	2.70
$\hat{P}$	0.4625	0.4795	0.4940	0.5075	0.5185

δ 在 $\pm 50\%$ 范围内变动，导通概率只在 0.4625–0.5185 之间移动，极差 0.056。这是合理的： $\delta = 1.8$ nm 相对介质尺度（ $r = 30$ 、 $h = 5000$ ）小两到三个数量级，它只微调”接触”的判定边界，不改变团簇结构。

需要说明本表的统计性质，以免把它误当成一次独立验证：五档 δ 用的是**同一组随机构型**（同种子、同 $T = 2000$ ，只换判据阈值），因此这是一次**配对比较**。配对是有意为之——它把构型间的随机波动整体消掉，五档之间的差值比各自的 Wilson 区间半宽（约 0.022）可靠得多，这正是要看的量。相应地，本表 $\delta = 1.80$ nm 处的 0.4940 与第 5.2 节 solve_p2.py 在同一体积分数上得到的 0.4911 **不是两次独立估计**：两处都用默认种子 20260810，本表的 2000 次试验是那 8000 次的前缀子集，数值接近是构造使然。本文真正的独立复核只有三处，都换了种子：第 6.2 节的 $T = 12000$ （seed=987654321）、第 8.3 节的收敛表（seed=13579）、第 7.3 节擦边点的 $T = 20000$ （seed=20260811）。

S2 建模口径. 同一体积分数下，把各条口径分别换掉（ $T = 4000$ ）：

表 19 建模口径对导通概率的影响

口径	$\hat{P}$	95% 置信区间	相对基准
基准：球面均匀方向	0.4940	(0.4785, 0.5095)	—
+ 碎片各自独立（主口径）			
方向改为附件标定的极角均匀分布（灵敏度口径）	0.7418	(0.7280, 0.7551)	+0.248
碎片粘合为同一导体（已否决）	1.0000	(0.9990, 1.0000)	+0.506

这张表是本文最重要的一张灵敏度表，它给出两条明确结论：

1. **方向分布的影响 (0.248) 是判据阈值影响 (0.056) 的 4.4 倍**。也就是说，本文结论的不确定性几乎全部来自”题面的’方向随机’该怎么读”，而不是来自数值精度或题目给定的物理参数。这也是第 9.2 节把它列为首要局限的原因。注意这 0.248 与口径主次无关：换基准只改变符号，两套口径之间的距离是同一个数。

2. “碎片粘合”口径把概率从 0.494 顶到 1.0000 (4000 次试验中 4000 次导通)。这正是假设 A2 被否决的量化理由：在该口径下问题二至四全部退化为平凡问题，任何体积分数都必然导通。

S3 介质 B 的盈亏平衡价。 问题四的答案是”不掺 B”，但这是价格的结论而非几何的结论，因此必须回答：B 便宜到什么程度才值得掺？

取阶段 C 中已用 6000 次试验验证过 $P \geq 0.90$ 的边界点 (484, 1421)、(524, 938)、(564, 516)、(604, 60)、(608, 0) 作线性拟合，得可行边界斜率

$$g' = \frac{dN_B}{dN_A} = -11.32 \text{ 颗/根}, \quad (7)$$

即每少用 1 根介质 A，需要补约 11.3 颗介质 B 才能守住 90% 的导通概率。代入 $c_B^* = -c_A/g'$ 得盈亏平衡单颗成本 1.311×10^{-3} 元，换算为单价

$$c_B^*/(V_B/10^9) = 0.0391 \text{ 元}/\mu\text{m}^3. \quad (8)$$

现价是 0.05 元/ μm^3 ，比盈亏平衡价高 27.8% (等价地，需再降价 21.7%)。这解释了为什么最优解干脆地落在 $N_B = 0$ 而不是某个”少量掺一点”的中间配比：当前价格并不处在临界附近。

需要说明的是， $N_A > 608$ 的区段上 N_B 已经取到 0，那里的点是”配足过头”的内点而不是边界点；把它们放进拟合会把斜率压平、把盈亏平衡价抬高，从而错误地得出”现价已接近临界”的结论。本文的拟合只取真正贴着 $P = 0.90$ 的点。另可注意：本斜率 -11.32 与灵敏度口径下的 -11.39 几乎相同——可行边界的斜率对方向分布并不敏感，敏感的是边界的位置。

图 11 把这两件事画在同一量程 (0–1) 上对照：左图的 δ 扫描几乎是一条水平线，

右图的口径差异则跨越大半个坐标轴。两图刻意不放大纵轴——把 0.056 的变化画进 0.46–0.52 的窄区间会显得陡峭，与”对阈值不敏感”这个结论相反。

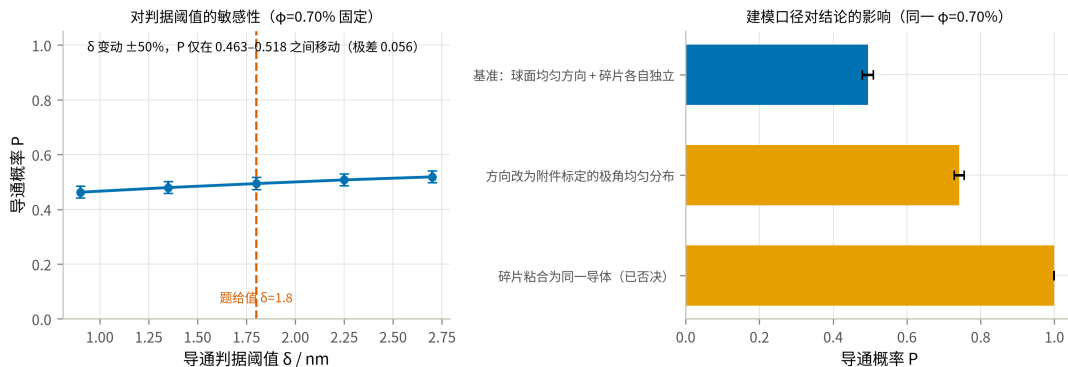


图 11 导通概率对判据阈值与建模口径的敏感性

8.3 蒙特卡洛收敛性

同一配置 ($\varphi = 0.70\%$) 按不同试验次数重复估计, Wilson 区间半宽随 $1/\sqrt{T}$ 收窄:

表 20 蒙特卡洛的收敛性

试验次数 T	250	500	1000	2000	4000	8000
$\hat{P}$	0.5400	0.5060	0.5030	0.5025	0.5008	0.5015
区间半宽	0.0613	0.0437	0.0309	0.0219	0.0155	0.0110

估计值在 $T \geq 1000$ 之后稳定在 0.501–0.503, 与问题二用 $T = 8000$ 得到的 0.4911 相差 0.010, 与区间半宽同量级 (两次估计用了不同的随机种子)。注意 $T = 250$ 那一格给出 0.5400, 比收敛值高 0.039——小样本会明显偏离, 这正是第 7.3 节坚持用大样本复核擦边点的理由。图 12 把这两件事分开画: 左图是估计值随样本量趋稳的过程, 右图在双对数坐标下把区间半宽与 $1/\sqrt{T}$ 参考线对照, 两者平行, 说明误差确实按蒙特卡洛的标准速率收窄, 没有出现相关样本导致的收敛变慢。

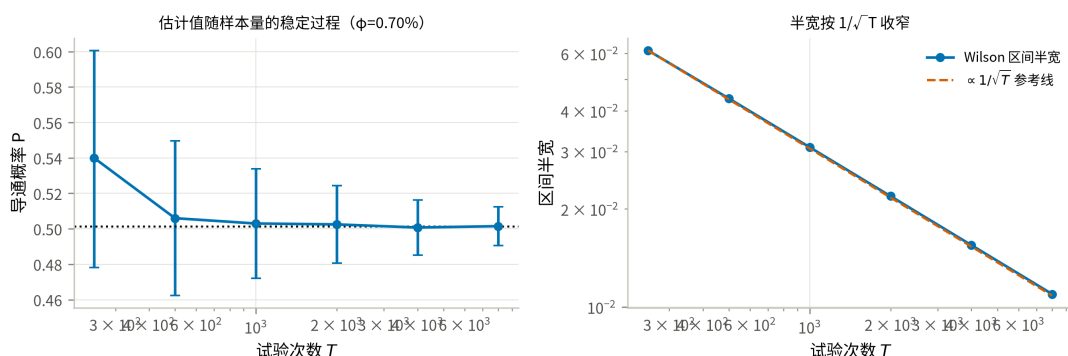


图 12 蒙特卡洛估计的收敛性

样本量是按分辨率需要定的, 不是越大越好: 问题三要在 0.01% 的体积分数网格上分辨相邻点, 相邻点相差约 7 根介质、对应 $\Delta P \approx 0.02$, 因此取 $T = 4000$ (半宽约 0.015) 即可分开, 最终答案再用 $T = 12000$ 复核; 问题四的网格只用于缩小范围, 取 $T = 700$ 就够, 候选点才用 $T = 6000$ 直接验证; 第 7.3 节审计的区间排除用 $T = 2500$, 而贴着 0.90 的擦边点单独用 $T = 20000$ 复核——那里半宽必须小于点估计到 0.90 的距离, 否则判读的是噪声而不是结论。

8.4 球柱体近似的严格双侧界

假设 A1 给出了 $K^- \subseteq C \subseteq K^+$, 因而 $P(K^-) \leq P(\text{真实圆柱}) \leq P(K^+)$ 。两端只差轴长 (4940 与 5000 nm), 复用同一套内核即可算出 (每点 $T = 6000$):

表 21 问题二四档的双侧界

体积分数	下界 $P(K^-)$	上界 $P(K^+)$	带宽
0.50%	0.0745	0.0887	0.014
0.60%	0.1992	0.2338	0.035
0.70%	0.4323	0.4907	0.058
1.00%	0.9898	0.9935	0.004

表 22 问题三 0.01% 网格上的双侧界

体积分数	N_A	下界 $P(K^-)$	上界 $P(K^+)$	判断
0.85%	601	0.8520	0.8882	上界都不足 0.90 → 必不达标
0.86%	608	0.8705	0.9130	夹在两端之间 → 无法判定
0.88%	622	0.8993	0.9302	夹在两端之间 → 无法判定
0.89%	630	0.9202	0.9450	下界已达 0.90 → 必达标

(0.83%、0.84%、0.87% 三格的完整数值见图 13 与 results/geometry_bracket.json。)

判读规则来自包含关系：若 $P(K^+) < 0.90$ 则真实圆柱必不达标；若 $P(K^-) \geq 0.90$ 则必达标。于是真实答案被夹在 $[0.86\%, 0.89\%]$ 之内，0.89% 是可证达标的最小网格值。扫描窗口以主口径的答案 0.86% 为中心、两侧各展开 3 格自动构造，不写死——写死的窗口在方向口径改变后会静默扫到错误区间。

把这张表画成一条”夹逼带”即图 13：蓝带的上沿是外界 $P(K^+)$ 、下沿是内界 $P(K^-)$ ，真实平端面圆柱的概率曲线必定落在带内。 $P = 0.90$ 这条横线先在 0.86% 处穿过上沿、再到 0.89% 才穿过下沿，两个交点之间的橙色区段就是答案的不确定区间。

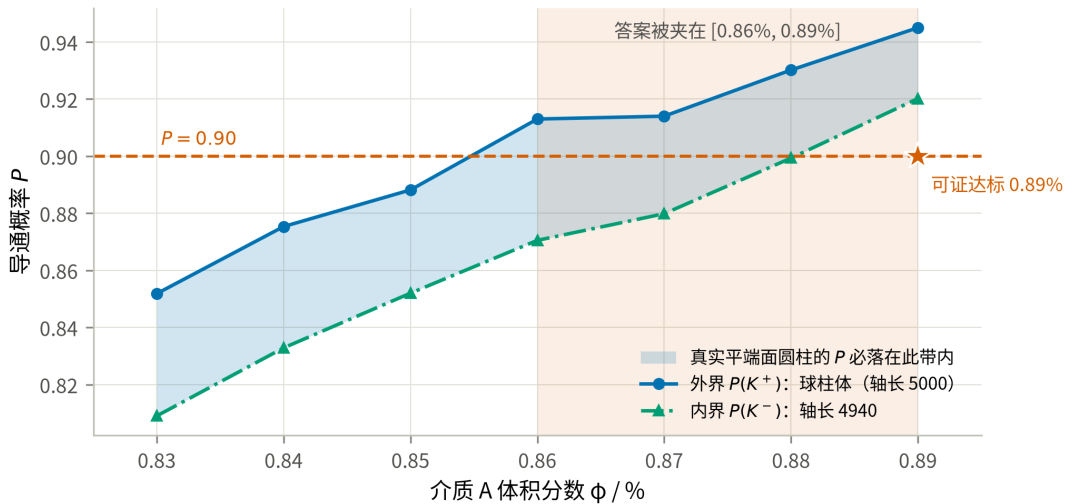


图 13 球柱体近似的严格双侧界与问题三答案的夹逼

这条结论修正了本文早先的表述。原先仅以”体积只差 0.80%、临界接触中约 3.3% 受影响”论证近似可接受，但模型工作在渗流跃变的陡峭段上（第 5.2 节：每 0.10%

体积分数带来约 0.15–0.26 的概率变化)，0.03–0.06 的概率带宽足以跨越三个 0.01% 网格。因此”近似影响很小”这一说法不成立，必须改用上面的双侧界给出区间答案。这也说明：在渗流阈值附近，几何近似的量级不能用体积差来衡量，要用它在判定阈值附近造成的概率带宽来衡量。

同时注意 $\varphi = 1.00\%$ 处带宽仅 0.004、 $\varphi = 0.50\%$ 处仅 0.014——远离跃变段时近似确实无关紧要，带宽在最陡的 0.70% 处放大到 0.058，是随着接近阈值而放大的。

九、模型评价与推广

9.1 优点

1. 判定内核经过独立校验，而非”看起来对”。尤其是 T3 的附件往返测试，用真实数据把截断规则、周期盒尺寸和碎片还原三件事一次性钉死。
2. 建模口径由数据定，不由猜测定。方向分布、周期盒、碎片口径三条都做了检验；其中方向分布的各向异性若被忽略，问题二的概率会系统性偏低近一半。
3. 问题四利用了单调性这一结构性质，把二维整数搜索严格地降为沿可行边界的一维搜索，并且最终结论由直接仿真验证，代理模型只承担缩小范围的职责。
4. 所有数字可溯源。论文中每个数值都来自 `results/*.json`，由脚本一次跑出。

9.2 缺点与局限

(1) 方向分布的口径不确定性没有被消除，只是被量化了。题面只说”方向随机”，本文按最少假设取球面均匀为主口径；但附件组 3 的实测分布以 $p = 6.7 \times 10^{-19}$ 拒绝球面均匀，且组 3 恰好是 $\varphi = 0.50\%$ 的构型而它导通（主口径下概率仅 0.087）。这两条都是对主口径的反证，本文如实列出并把附件标定口径的完整结果并列给出，但无法断定组委会的参考答案取哪一套。这是本文最大的不确定来源，其影响远大于所有数值误差之和。

(2) 球柱体近似（假设 A1）在端帽附近不精确。包含关系 $K^- \subseteq C \subseteq K^+$ 给出的双侧界（第 8.4 节）显示：在远离阈值处带宽仅 0.004–0.014，在跃变最陡的 $\varphi = 0.70\%$ 处放大到 0.058，并使问题三的答案只能给到 $[0.86\%, 0.89\%]$ 这个区间。方向是球柱体略微高估接触、因而略微高估导通概率。本文不再用”体积只差 0.80%”或任何单一比例来描述这条近似的影响——第 8.4 节说明了为什么在渗流阈值附近那种描述是无效的。

(3) 介质 B 的越界处理没有按题面规则实现，两个口径都是替代方案。题面的边界截断规则对所有导电介质一视同仁，严格做法应是把被边界切开的球截成球缺、再把越界的球缺平移回盒内，两块按假设 A2 各自独立。本文实现的两个口径都不是这件事：`sphere_mode="wrap"` 把被切开的球按整球计（于是有一部分留在盒外），

`sphere_mode="inside"` 则把球心限制在盒内 R 深处、令球根本不越界（这是改了放置规则）。两者的差别已量化，见表 23。

表 23 介质 B 两种越界处理口径的对照 ($T = 4000$, 主口径方向)

配置	wrap	inside	差值
$N_A = 440, N_B = 1200$	0.6723	0.7083	-0.0360
$N_A = 500, N_B = 700$	0.7650	0.7885	-0.0235

本来预期”整球近似会高估跨接能力”，实际却是 wrap 更低。原因不在端帽近似，而在有效数密度：inside 把同样多的球塞进 $(1 - 2R/L)^3 = 88.5\%$ 的体积里，内部密度反而更高。两者相差 0.02–0.04，仍比方向口径的影响 (0.248) 小一个量级。由于问题四的推荐配比 $N_B = 0$ ，这条近似不影响最终答案；它确实影响表 11 的整张概率曲面、图 8 的可行边界位置，以及由边界斜率导出的盈亏平衡价 $0.0391 \text{ 元}/\mu\text{m}^3$ （第 8.2 节 S3）——引用那个价格时应把这条不确定性一并记住。

(4) 解析对照只做到量级，不能替代仿真。第 5.2 节用排除体积判据给出了独立的阈值估计 ($\varphi_c \approx 0.777\%$)。它与主口径同为各向同性，因而可直接对照，差 9%，差异来自本题”棒长恰为盒边一半”的有限尺寸效应 ($0.777\% \rightarrow 0.705\%$)；再换成附件标定方向则进一步降到 0.607%。因此它只能用来核对仿真没有系统性跑偏，无法脱离仿真去预测其它几何参数下的阈值。

(5) 蒙特卡洛给的是频率而非精确值。所有概率都带 Wilson 区间；问题三的答案 0.86% 依赖于 0.85% 与 0.86% 两格能被区分开，本文用 $T = 4000$ 与 $T = 12000$ 两次独立种子的估计确认了这一点，但若组委会的参考答案用了不同的生成器细节，相邻格的归属仍可能改变。需要提醒的是，本文并非每一处比较都是独立的：默认种子固定为 20260810，因此同一构型、不同 T 的两次调用是嵌套的（短的那次是长的那次的前缀子集），这类比较只能说明实现一致，不能当作独立复核（第 8.2 节已就此说明）。

(6) 组 1、组 2 的几何与题面不符，本文按数据实际几何判定。若组委会本意是 10000^3 立方体，则这两组的输入本身有误，需要以官方勘误为准。第 4.3 节表 6 给出了三种口径下的体积分数，三个判定结论均不变。

9.3 推广

判定内核与题目的具体形状无关：把”球柱体 + 球”换成任意可计算最短距离的凸体，整套流程（截断 $\rightarrow$ 接触图 $\rightarrow$ 并查集 $\rightarrow$ 蒙特卡洛 $\rightarrow$ 仿真优化）可直接迁移到碳纳米管/石墨烯片复合材料的导电阈值预测、纤维增强材料的逾渗设计等场景。问题四的”单调约束 + 线性成本 $\Rightarrow$ 最优解在可行边界上”这一论证，对任何”加料只会更容易达标”的配方优化都成立。

十、结论

表 24 四问结论汇总（主口径：球面均匀方向，假设 A4）

问题	答案
一	组 1 不导通；组 2 导通；组 3 导通
二	$\varphi = 0.50\%$: $P = 0.0874$; 0.60% : 0.2325 ; 0.70% : 0.4911 ; 1.00% : 0.9940 （球柱体口径，为上界；下界见表 21）
三	使 $P \geq 90\%$ 的最低体积分数：可证达标值 $\varphi = 0.89\%$ （630 根）；球柱体口径下为 0.86% （608 根）。真实值夹在两者之间
四	推荐配比 $N_A = 608$ 、 $N_B = 0$ （ $\varphi_A = 0.86\%$ 、 $\varphi_B = 0$ ），总成本 9.0252 元 ——唯一可行性 被确认的最低成本方案。 最低总成本报到 9.02 元这一精度 ：(592, 141)、(607, 8)、 (606, 17)、(604, 0) 等更便宜的配比，其可行 性在 $T = 20000$ 下既不能确认也不能排除， 最优解在该精度下是退化的（第 7.3 节表 16）

三点值得单独指出：

1. **问题一的关键不在判定，而在还原。**附件的一行是边界截断后的碎片而非一根介质；组 1、组 2 的周期盒也不是立方体。不做这一步，三个微构体的体积分数会分别错到 1/100，判定的机理解释也无从谈起。
2. **导通是跃变而非线性响应，且跃变发生在巨团簇形成之前。**体积分数从 0.50% 到 1.00%，导通概率由 0.087 升到 0.994（主口径），而最大团簇始终只占约 2%–24% 的碎片；即便只统计导通的那些实现，也只有 2.7%–24.2%（第 5.3 节表 10）。附件组 3 这个真实的导通样本更直观：贯通团簇只有 17/535 个碎片（图 5）。贯通靠的是 2–3 根介质接力的短链，不是吞掉大半介质的巨团簇。工程上这意味着两件事：填充量设计必须留出跃变宽度的余量；本题的阈值也不能直接外推到更大的微构体——盒子变大而介质尺寸不变时，阈值会升高。
3. **问题四的最优解是退化的，“不掺 B”是唯一能拿出可行性证据的方案。**介质 B 在介质稀疏时每元的边际收益高于 A，只是 $P \geq 90\%$ 的约束把可行解推到了 A 占优的区段；B 的单价只要降到 0.0391 元/ μm^3 以下（现价再降 21.7%），最优解就会明确倒向混填。现价距盈亏平衡价只有 27.8% 的余量，可行边界（斜率 -11.32）与等成本线（-8.86）接近平行——这正是最优解附近无法分辨的根源：

(592, 141)、(607, 8)、(606, 17)、(604, 0) 都比 (608, 0) 便宜，但它们是否达标在 $T = 20000$ 下都判不出来，因为它们的导通概率与 0.90 的差距小于该样本量能分辨的尺度。所需样本量随这个差距的平方反比增长，若真值恰为 0.90 则无论多少次试验都分不开——这是**问题本身的退化**，不是算力不足。因此写成“推荐配比 (608, 0)、最低成本报到 9.02 元”比写成“唯一最优配比”或“更便宜的点已全部排除”都更符合证据——**便宜出来的那一点钱买不到可行性的证据**。

全文的最大不确定性不在数值精度，而在方向分布这一建模口径：本文主口径按“题面”方向随机”取球面均匀；若改用附件组 3 标定的极角均匀分布，问题二的四个概率变为 0.2193 / 0.4798 / 0.7458 / 0.9994，问题三的答案变为 0.78%。两套结果全程并列给出，读者可按对题面的理解自行取用。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码实现与调试、图表版式检查和文字校对，详细使用情况见支撑材料《AI 工具使用详情》。所用工具共两个：**Claude Code (模型 claude-opus-5, Anthropic)** 与 **Codex (模型 GPT-5, OpenAI)**。两者的名称、版本与使用日期见支撑材料《AI 工具使用详情》第一节；按 2026 年试行规定，AI 工具不作为参考文献著录，故未列入参考文献表。

AI 参与位置说明. 为便于核查，按章节列出 AI 参与的具体环节、承担的工具，以及人工主导的内容；未列出的部分由参赛队完成：

位置	承担工具	AI 参与内容	人工主导内容
----	------	---------	--------

表 25 AI 工具参与位置与人工主导内容

位置	承担工具	AI 参与内容	人工主导内容
第 2 节、附录 B 的 audit_attachment.py	Claude Code	编写审计脚本、执行 KS 与 Spearman 检 验	提出”附件一行可能 不是一根介质”的疑 问；采纳审计结论
第 3 节、附录 B 的 几何内核	Claude Code	实现边界截断、最短 距离与并查集判定并 向量化	确定球柱体口径、碎 片独立口径与硬壁口 径
第 4–8 节的全部数值	Claude Code	编写求解与校验脚 本、执行仿真、整理 结果	审阅结果合理性与校 验设计
图 1–图 13	Claude Code、Codex	编写绘图脚本、生成 图件；图 1 技术路线 图改为黑白矢量版	审阅图表结论与量程 设置；提出黑白专业 化要求
复核与修订：随机数 分块与复现性、问题 四成本下界的覆盖范 围、图 5 的团簇着色 口径、表格编号与题 注	Codex	逐节核对正文数值与 results/*.json、源 码与图件的一致性， 指出并修正上述问题	审定每一处修改是否 采纳；确认结论的表 述边界
全文文字	Claude Code、Codex	起草与润色	逐句核对与结果文件 的一致性

核心建模口径（假设 A1–A5，尤其是”截断后碎片各自独立”）由参赛队判定；AI 输出的代码一律运行验证，数值一律与 results/ 中的结果文件核对。两个工具的分工是：Claude Code 完成建模、求解与初稿；Codex 承担独立复核与据此的修订，复核中发现的问题及其处理见支撑材料《AI 工具使用详情》第四节。

本目录说明（非提交稿内容）：本仓库是赛后复现，不是赛内提交稿；上述声明沿用 CUMCM《人工智能工具使用规定》的二选一句式。若用于正式提交，须按主办方当年章程核对声明措辞、位置与标注方式，并删去本段说明。

参考文献

- [1] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 5 版. 北京: 高等教育出版社, 2018: 第 1 章建立数学模型.
- [2] 司守奎, 孙兆亮. 数学建模算法与应用 [M]. 2 版. 北京: 国防工业出版社, 2015: 第 20 章回归分析与广义线性模型.
- [3] 徐钟济. 蒙特卡罗方法 [M]. 上海: 上海科学技术出版社, 1985: 第 2 章随机抽样, 第 4 章误差估计与方差缩减.
- [4] 盛骤, 谢式千, 潘承毅. 概率论与数理统计 [M]. 4 版. 北京: 高等教育出版社, 2008: 第 7 章参数估计, 第 8 章假设检验.
- [5] 李文春, 沈烈, 孙晋, 郑强, 章明秋. 多壁碳纳米管填充高密度聚乙烯复合材料的导电性和动态流变行为 [J]. 高分子学报, 2006(2): 269-273.
- [6] BALBERG I, ANDERSON C H, ALEXANDER S, et al. Excluded volume and its relation to the onset of percolation[J]. Physical Review B, 1984, 30(7): 3933-3943.
- [7] ERICSON C. Real-Time Collision Detection[M]. San Francisco: Morgan Kaufmann, 2005.

附录

附录 A 支撑材料文件列表

表 A1 支撑材料文件清单

文件	内容	对应论文位置
src/microstructure.py	几何与渗流内核	第 3 节
src/load_attachment.py	附件读取与母介质还原	第 2.1 节
src/audit_attachment.py	数据审计与 KS 检验	第 2 节、图 2、图 3
src/validate.py	几何内核 5 项校验	第 8.1 节表 17
src/simulate.py	蒙特卡洛驱动（并行、Wilson 区间、可复现种子）	第 5.1 节
src/solve_p1.py	问题一判定	第 4 节表 4、图 5
src/solve_p2.py	问题二概率估计	第 5 节、图 6
src/solve_p3.py	问题三阈值反解	第 6 节
src/solve_p4.py	问题四成本优化	第 7 节表 12、表 13、图 8
src/p4_backfill_probes.py	补记阶段 C 未通过的探测点（同种子可复现）	第 7.2 节表 13

文件	内容	对应论文位置
src/p4_break_even.py	由已验证边界点求边界斜率 与介质 B 盈亏平衡价	第 8.2 节 S3
src/sensitivity.py	灵敏度与收敛性	第 8.2、8.3 节、图 11、图 12
src/sphere_mode_check.py	介质 B 越界处理口径的对照	第 9.2 节
src/theory_check.py	排除体积判据、跃变中点、 边际效率	第 5.2、7.2 节
src/cluster_stats.py	最大团簇占比（渗流序参量）	第 5.3 节、图 7
src/geometry_bracket.py	球柱体近似的严格上下界 (内接/外接球柱体)	第 8.4 节表 21、表 22
src/p4_global_audit.py	成本下界的穷举式审计（角 点 + 预算线，步长 16）	第 7.3 节表 15
src/p4_global_audit2.py	补上表 15 未覆盖区段的逐列 审计（99% Wilson 判据）	第 7.3 节表 16
src/p4_marginal_recheck.py	预算线上擦边点的大样本复 核（ $T = 20000$ ，独立种子）	第 7.3 节
src/make_results_bundle.py	合并结果文件并装配附录 B 源程序	附录 B
src/paper_figures.py	全部正文图件（13 张，PNG + PDF 矢量）	图 1–图 13
results/*.json	论文中每个数字的出处	全文
figures_paper/图注清单.md	图注与生成脚本对照	全文
AI 工具使用详情.pdf	AI 工具使用详情（工具与版 本、用途环节、使用过程、采 纳与核验）	AI 工具使用声明

附录 B 完整可运行源程序

本附录由 src/make_results_bundle.py 从 src/ 直接装配，与实际运行的代码逐字一致。运行顺序见附录 A 之后的说明。依赖 Python 3.11 与 numpy / scipy / matplotlib / openpyxl。

src/microstructure.py

```
#!/usr/bin/env python3
```

```
""" 微构体导电仿真的几何与渗流内核。
```

本模块只做三件事，不涉及任何题目分支逻辑：

1. ** 边界截断 **：把一根越界的介质按题目给定的规则平移回微构体，得到若干碎片；

2. ** 接触判定 **：两个介质（或介质与带电面）之间的最短表面距离是否不超过 1.8 nm ；
3. ** 导通判定 **：并查集把接触关系连成连通块，判断是否存在连接左右带电面的通路。

几何约定

介质 A 建模为 ** 球柱体 (*spherocylinder / capsule*) **：半径 $r=30\text{ nm}$ 的圆柱两端各加一个半球帽。这样两个介质 A 的表面最短距离就精确等于两条轴线段的最短距离减去 $2r$ ，判定可以完全向量化。题目里的介质 A 是平面面直圆柱，与球柱体的差别只出现在端面附近，且体积仅相差 $(4/3)r^3 / (r^2h) = 4r/(3h) = 0.8\%$ 。这一近似的严格处理是 ``geometry_bracket.py`` 给出的双侧界 (K 真实圆柱 K ，故 $P(K) \approx P(\text{真实}) \approx P(K)$ ，论文第 8.4 节)——在渗流阈值附近，近似的影响不能用体积差来衡量，要用它在判定阈值附近造成的概率带宽来衡量。

介质 B 是半径 200 nm 的正球体，本身就是球，无需近似。

"""

from __future__ import annotations

import numpy as np

----- 物理常数

```
EDGE = 10000.0          # 微构体边长 (nm)
R_A = 30.0              # 介质 A 底面半径 (nm)
H_A = 5000.0           # 介质 A 高 (nm)
R_B = 200.0            # 介质 B 半径 (nm)
GAP = 1.8              # 导通判定的最大表面间距 (nm)

V_BOX = EDGE ** 3       # 1.0e12 nm^3
V_A = np.pi * R_A ** 2 * H_A    # 1.41372e7 nm^3
V_B = 4.0 / 3.0 * np.pi * R_B ** 3    # 3.35103e7 nm^3

# 成本：介质 A 1.05 元/m^3，介质 B 0.05 元/m^3；1 m^3 = 1e9 nm^3
COST_A = 1.05 * V_A / 1e9      # 元 / 根
COST_B = 0.05 * V_B / 1e9      # 元 / 颗
```

----- 边界截断

def wrap_segment(p: np.ndarray, q: np.ndarray, box: np.ndarray) -> list[tuple[np.ndarray, np.ndarray]]:

""" 把轴线段 $p \rightarrow q$ 按边界截断规则折回盒子，返回若干条完全位于盒内的子段。

做法：把线段参数化为 $t \in [0, 1]$ ，在每个坐标轴上求出它穿越周期格点边界的所有 t ，以这些 t 为断点切段；每一小段整体位于同一个周期像里，平移回来即可。这与题面“把超出部分沿越界方向的反方向平移一个边长”逐字等价，并且天然支持需要多次平移的情形。

"""

```
half = box / 2.0
d = q - p
ts = [0.0, 1.0]
for j in range(3):
    if abs(d[j]) < 1e-12:
        continue
    # 穿越平面  $x_j = half_j + k*box_j$ 
    k_lo = np.floor((min(p[j], q[j]) - half[j]) / box[j])
    k_hi = np.ceil((max(p[j], q[j]) - half[j]) / box[j])
```

```

        for k in range(int(k_lo), int(k_hi) + 1):
            plane = half[j] + k * box[j]
            t = (plane - p[j]) / d[j]
            if 1e-12 < t < 1 - 1e-12:
                ts.append(float(t))
ts = sorted(set(ts))

out: list[tuple[np.ndarray, np.ndarray]] = []
for t0, t1 in zip(ts[:-1], ts[1:]):
    if t1 - t0 < 1e-12:
        continue
    a = p + t0 * d
    b = p + t1 * d
    mid = 0.5 * (a + b)
    shift = np.round(mid / box) * box          # 该子段所在周期像的偏移
    seg_a, seg_b = a - shift, b - shift
    # 数值上把端点压回盒内, 避免  $\pm 1e-12$  的溢出
    np.clip(seg_a, -half, half, out=seg_a)
    np.clip(seg_b, -half, half, out=seg_b)
    out.append((seg_a, seg_b))
return out

# ----- 距离计算

def seg_seg_dist(p1: np.ndarray, q1: np.ndarray, p2: np.ndarray, q2: np.ndarray) -> np.ndarray:
    """ 成对线段最短距离, 全向量化。

    p1/q1 形状 (n,1,3), p2/q2 形状 (1,m,3) 时返回 (n,m)。
    算法是 Ericson 《Real-Time Collision Detection》的 ClosestPtSegmentSegment,
    分支用 np.where 展开; 退化 (零长) 线段按点处理。
    """
    d1 = q1 - p1
    d2 = q2 - p2
    r = p1 - p2

    a = np.sum(d1 * d1, axis=-1)
    e = np.sum(d2 * d2, axis=-1)
    f = np.sum(d2 * r, axis=-1)
    c = np.sum(d1 * r, axis=-1)
    b = np.sum(d1 * d2, axis=-1)

    eps = 1e-12
    a_safe = np.maximum(a, eps)
    e_safe = np.maximum(e, eps)

    denom = a * e - b * b
    s = np.where(denom > eps, (b * f - c * e) / denom, 1.0), 0.0)
    s = np.clip(s, 0.0, 1.0)

    t = (b * s + f) / e_safe

    # t 越界时钳位并回代求 s
    s_lo = np.clip(-c / a_safe, 0.0, 1.0)
    s_hi = np.clip((b - c) / a_safe, 0.0, 1.0)
    s = np.where(t < 0.0, s_lo, np.where(t > 1.0, s_hi, s))

```

```

t = np.clip(t, 0.0, 1.0)

# 退化线段
s = np.where(a <= eps, 0.0, s)
t = np.where(e <= eps, 0.0, np.where(a <= eps, np.clip(f / e_safe, 0.0, 1.0), t))

diff = r + s[..., None] * d1 - t[..., None] * d2
return np.sqrt(np.maximum(np.sum(diff * diff, axis=-1), 0.0))

def contact_pairs(p: np.ndarray, q: np.ndarray, thresh: float, block: int = 256) -> np.ndarray:
    """ 返回所有轴线距离 thresh 的线段对下标 (k,2)。

    分块 + 轴对齐包围盒预筛。朴素写法要一次性开出  $M^2$  的十几个临时数组，
     $M1300$  时光是分配和读写就占了九成时间；分块后临时数组常驻缓存，
    实测在  $\sim 1\%$  规模上快 6 倍以上，结果与朴素写法逐位相同 (validate.py T6)。
    """
    m = len(p)
    lo = np.minimum(p, q) - thresh
    hi = np.maximum(p, q)
    out: list[np.ndarray] = []
    for s in range(0, m, block):
        e = min(s + block, m)
        # 只与下标更大的比较，避免重复
        cand = np.nonzero(np.all(lo[None, s:e, :] <= hi[:, None, :], axis=-1)
                           & np.all(lo[s:, None, :] <= hi[None, s:e, :], axis=-1))
        rows = cand[0] + s          # 全局下标 j
        cols = cand[1] + s          # 全局下标 i
        keep = rows > cols
        rows, cols = rows[keep], cols[keep]
        if rows.size == 0:
            continue
        d = seg_seg_dist(p[cols], q[cols], p[rows], q[rows])
        hit = d <= thresh
        if np.any(hit):
            out.append(np.stack([cols[hit], rows[hit]], axis=1))
    return np.concatenate(out) if out else np.zeros((0, 2), dtype=int)

def point_seg_dist(pts: np.ndarray, p: np.ndarray, q: np.ndarray) -> np.ndarray:
    """ 点到线段的距离。 pts ( $n,1,3$ ), p/q ( $1,m,3$ ) -> ( $n,m$ )。 """
    d = q - p
    w = pts - p
    dd = np.maximum(np.sum(d * d, axis=-1), 1e-12)
    t = np.clip(np.sum(w * d, axis=-1) / dd, 0.0, 1.0)
    diff = w - t[..., None] * d
    return np.sqrt(np.maximum(np.sum(diff * diff, axis=-1), 0.0))

# ----- 并查集

class DSU:
    __slots__ = ("parent",)

    def __init__(self, n: int) -> None:
        self.parent = list(range(n))

```

```

def find(self, x: int) -> int:
    p = self.parent
    while p[x] != x:
        p[x] = p[p[x]]
        x = p[x]
    return x

def union(self, x: int, y: int) -> None:
    rx, ry = self.find(x), self.find(y)
    if rx != ry:
        self.parent[ry] = rx

# ----- 生成配置

# 全文的方向分布口径集中在这两个常量上，改口径只需改这里。
#
# 主口径取 ** 球面均匀 **：题面对问题二至四只说“方向随机”，未指定分布，
# 球面均匀是这句话的最少假设读法。
# 对照口径取附件标定的极角均匀分布：附件组 3 的 354 根母介质以
# KS 检验  $p=6.7e-19$  拒绝球面均匀、 $p=0.73$  不拒绝极角均匀（第 2.4 节）。
# 这一条只作为灵敏度分析出现——它刻画的是附件的生成过程，
# 而题面并未说问题二至四必须沿用同一过程。
PRIMARY_ORIENTATION = "isotropic"
CONTROL_ORIENTATION = "polar_uniform"

def sample_orientations(n: int, rng: np.random.Generator,
                        mode: str = PRIMARY_ORIENTATION) -> np.ndarray:
    """ 生成  $n$  个单位方向向量。

    mode="isotropic" : 球面均匀，即  $|u_x| \sim U(0,1)$ 。这是“任意方向、不考虑重力”
                        最自然的物理读法。
    mode="polar_uniform" :  $\sim U(0,1)$ 、 $\sim U(0,2)$ ，以 **X 轴（带电面法向）** 为极轴，
                         $u = (\cos, \sin \cos, \sin \sin)$ 。这是附件数据实际服从的分布：
                        对组 3 的 354 根母介质做 KS 检验，各向同性被拒绝 ( $D=0.243$ ,
                         $p=6.7e-19$ )，本分布不被拒绝 ( $D=0.036$ ,  $p=0.73$ )。
                        它在两极堆积，使介质明显偏向带电面法向，从而显著提高导电概率。

    """
    if mode == "isotropic":
        u = rng.normal(size=(n, 3))
        return u / np.linalg.norm(u, axis=1, keepdims=True)
    if mode == "polar_uniform":
        theta = rng.uniform(0.0, np.pi, n)
        phi = rng.uniform(0.0, 2 * np.pi, n)
        s = np.sin(theta)
        return np.stack([np.cos(theta), s * np.cos(phi), s * np.sin(phi)], axis=1)
    raise ValueError(f"未知方向分布: {mode}")

def sample_rods(n: int, rng: np.random.Generator, box: np.ndarray,
                length: float = H_A,
                orientation: str = PRIMARY_ORIENTATION) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """ 随机生成  $n$  根介质  $A$  并做边界截断。

```

```

中心在盒内均匀；方向分布见 ``sample_orientations``。
返回碎片起点、终点，以及每个碎片所属的母介质编号。
"""
centers = (rng.random((n, 3)) - 0.5) * box
u = sample_orientations(n, rng, orientation)
p = centers - 0.5 * length * u
q = centers + 0.5 * length * u

starts: list[np.ndarray] = []
ends: list[np.ndarray] = []
owner: list[int] = []
for i in range(n):
    for a, b in wrap_segment(p[i], q[i], box):
        starts.append(a)
        ends.append(b)
        owner.append(i)
if not starts:
    empty = np.zeros((0, 3))
    return empty, empty, np.zeros(0, dtype=int)
return np.array(starts), np.array(ends), np.array(owner, dtype=int)

def sample_spheres(n: int, rng: np.random.Generator, box: np.ndarray,
                  mode: str = "wrap") -> tuple[np.ndarray, np.ndarray]:
    """ 随机生成  $n$  颗介质  $B$ 。

    mode="wrap" : 中心在盒内均匀，越界部分按截断规则折回，碎片以其所在周期像的
                  球心表示（碎片是球与盒的交，用整球近似，只在距壁 200 nm 内有偏差）；
    mode="inside" : 把球心限制在  $[-(L/2-R), L/2-R]$ ，保证整颗球都在盒内，不产生碎片。
    两种取法在灵敏度分析里对比。
    """
    if mode == "inside":
        c = (rng.random((n, 3)) - 0.5) * (box - 2 * R_B)
        return c, np.arange(n)

    c = (rng.random((n, 3)) - 0.5) * box
    half = box / 2.0
    centers: list[np.ndarray] = []
    owner: list[int] = []
    for i in range(n):
        # 每个越界方向都产生一个镜像碎片
        offs = [np.zeros(3)]
        for j in range(3):
            if c[i, j] + R_B > half[j]:
                offs = offs + [o + np.eye(3)[j] * -box[j] for o in offs]
            elif c[i, j] - R_B < -half[j]:
                offs = offs + [o + np.eye(3)[j] * box[j] for o in offs]
        for o in offs:
            centers.append(c[i] + o)
            owner.append(i)
    return np.array(centers), np.array(owner, dtype=int)

# ----- 导通判定

def percolates(seg_p: np.ndarray, seg_q: np.ndarray,

```

```

    sph_c: np.ndarray | None = None,
    owner_rod: np.ndarray | None = None,
    owner_sph: np.ndarray | None = None,
    bond_fragments: bool = False,
    edge: float = EDGE) -> bool:
""" 判断微构体是否导通（左右带电面之间存在导电通路）。

bond_fragments=False (默认): 边界截断产生的每个碎片是独立导体;
bond_fragments=True       : 同一根母介质的所有碎片电学上视为一体（灵敏度对照）。
"""

half = edge / 2.0
n_rod = len(seg_p)
n_sph = 0 if sph_c is None else len(sph_c)
n = n_rod + n_sph
if n == 0:
    return False

dsu = DSU(n + 2)
LEFT, RIGHT = n, n + 1

# --- 介质与带电面
if n_rod:
    lo = np.minimum(seg_p[:, 0], seg_q[:, 0]) - R_A
    hi = np.maximum(seg_p[:, 0], seg_q[:, 0]) + R_A
    for i in np.nonzero(lo <= -half + GAP)[0]:
        dsu.union(LEFT, int(i))
    for i in np.nonzero(hi >= half - GAP)[0]:
        dsu.union(RIGHT, int(i))
if n_sph:
    for i in np.nonzero(sph_c[:, 0] - R_B <= -half + GAP)[0]:
        dsu.union(LEFT, n_rod + int(i))
    for i in np.nonzero(sph_c[:, 0] + R_B >= half - GAP)[0]:
        dsu.union(RIGHT, n_rod + int(i))

# --- 棒-棒
if n_rod > 1:
    for a, b in contact_pairs(seg_p, seg_q, 2 * R_A + GAP):
        dsu.union(int(a), int(b))

# --- 球-球 与 球-棒（介质 B 数量可达上万，用 KD 树做邻域检索）
if n_sph:
    from scipy.spatial import cKDTree
    tree = cKDTree(sph_c)
    if n_sph > 1:
        for a, b in tree.query_pairs(2 * R_B + GAP, output_type="ndarray"):
            dsu.union(n_rod + int(a), n_rod + int(b))
if n_rod:
    # 沿每根碎片轴线按 STEP 采样，查半径 = 接触半径 + 半个采样步长，
    # 保证不漏掉任何真实接触；再用精确的点—线段距离过滤候选。
    reach = R_A + R_B + GAP
    step = 200.0
    samples, tags = [], []
    for i in range(n_rod):
        a, b = seg_p[i], seg_q[i]
        L = float(np.linalg.norm(b - a))
        k = max(2, int(np.ceil(L / step)) + 1)

```

```

        ts = np.linspace(0.0, 1.0, k)[: , None]
        samples.append(a + ts * (b - a))
        tags.append(np.full(k, i, dtype=int))
    pts = np.concatenate(samples)
    tags = np.concatenate(tags)
    radius = reach + step / 2.0 + 1e-9
    for pt_idx, sph_list in enumerate(tree.query_ball_point(pts, radius)):
        if not sph_list:
            continue
        rod = int(tags[pt_idx])
        cand = np.asarray(sph_list, dtype=int)
        d = point_seg_dist(sph_c[cand][:, None, :],
                           seg_p[rod][None, None, :], seg_q[rod][None, None, :])[:, 0]
        for s in cand[d <= reach]:
            dsu.union(n_rod + int(s), rod)

# --- 灵敏度对照：同母介质的碎片粘合
if bond_fragments:
    for owner, base, cnt in ((owner_rod, 0, n_rod), (owner_sph, n_rod, n_sph)):
        if owner is None or cnt == 0:
            continue
        first: dict[int, int] = {}
        for idx in range(cnt):
            o = int(owner[idx])
            if o in first:
                dsu.union(first[o], base + idx)
            else:
                first[o] = base + idx

    return dsu.find(LEFT) == dsu.find(RIGHT)

def n_rods_for_fraction(phi: float) -> int:
    """ 体积分数 -> 介质 A 根数 (题目要求四舍五入)。 """
    return int(round(phi * V_BOX / V_A))

```

src/load_attachment.py

```

#!/usr/bin/env python3
""" 读取附件 xlxs, 并把 " 碎片 " 还原成母介质 A。

附件每一行不是一根完整的介质 A, 而是边界截断之后的一个 ** 碎片 **:
组 3 有 535 行, 但按轴向分组后只有 354 个不同方向, 正好对应 354 根介质 A
(体积分数 0.50%)。数据审计 (见 audit_attachment.py) 还发现附件系统性地
丢弃了长度小于约 500 nm 的短碎片, 因此各组碎片总长略小于 根数 × 5000。
"""

from __future__ import annotations

import math
from collections import defaultdict
from pathlib import Path

import numpy as np
import openpyxl

```

```

from microstructure import EDGE

ROOT = Path(__file__).resolve().parents[1]
ATTACHMENT = ROOT.parent / "01_ 题目与附件" / " 附件.xlsx"

# 组 1、组 2 的 y、z 坐标在 ±500 处发生周期回绕 (见 audit_attachment.py),
# 说明这两个微构体的截面是 1000×1000 nm, 只有组 3 是题面描述的 10000 立方体。
GROUP_BOX = {
    " 组 1": np.array([EDGE, 1000.0, 1000.0]),
    " 组 2": np.array([EDGE, 1000.0, 1000.0]),
    " 组 3": np.array([EDGE, EDGE, EDGE]),
}

def load_pieces(path: Path | None = None) -> dict[str, np.ndarray]:
    """ 返回 {组名: (n,6) 数组}, 每行是一个碎片的两个轴端点。"""
    wb = openpyxl.load_workbook(path or ATTACHMENT, read_only=True, data_only=True)
    out: dict[str, np.ndarray] = {}
    for name in wb.sheetnames:
        rows = [r for r in wb[name].iter_rows(values_only=True)][2:]
        vals = [[float(v) for v in r[:6]] for r in rows if r[0] is not None]
        out[name] = np.array(vals)
    return out

def _canonical_dir(p: np.ndarray, q: np.ndarray) -> tuple:
    d = q - p
    d = d / np.linalg.norm(d)
    if d[0] < 0 or (d[0] == 0 and d[1] < 0):
        d = -d
    return tuple(np.round(d, 7))

def group_by_medium(pieces: np.ndarray) -> list[list[int]]:
    """ 按轴向把碎片归到母介质。同一根介质的所有碎片方向完全相同。"""
    groups: dict[tuple, list[int]] = defaultdict(list)
    for i, row in enumerate(pieces):
        groups[_canonical_dir(row[:3], row[3:])] .append(i)
    return list(groups.values())

def chain_pieces(pieces: np.ndarray, idx: list[int], box: np.ndarray) -> list[int]:
    """ 把一根母介质的碎片按首尾相接顺序排好 (用于还原未截断的轴线段)。"""
    half = box / 2.0

    def on_boundary(pt):
        return [j for j in range(3) if abs(abs(pt[j]) - half[j]) < 1e-6]

    succ, pred = {}, {}
    for i in idx:
        q = pieces[i][3:]
        for j in on_boundary(q):
            tgt = q.copy()
            tgt[j] = -q[j]
            for k in idx:
                if k != i and np.allclose(pieces[k][:3], tgt, atol=1e-6):

```



```

        succ[i], pred[k] = k, i
heads = [i for i in idx if i not in pred]
order = []
for h in heads:
    cur = h
    while cur is not None and cur not in order:
        order.append(cur)
        cur = succ.get(cur)
for i in idx:
    # 兜底：链断裂时也不丢碎片
    if i not in order:
        order.append(i)
return order

def reconstruct_axis(pieces: np.ndarray, idx: list[int], box: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↳ float]:
    """ 还原母介质未经截断的轴线段（起点、终点、已知总长）。"""
    order = chain_pieces(pieces, idx, box)
    start = pieces[order[0]][3:].copy()
    total = sum(float(np.linalg.norm(pieces[i][3:] - pieces[i][3:])) for i in order)
    d = pieces[order[0]][3:] - pieces[order[0]][3:]
    u = d / np.linalg.norm(d)
    return start, start + total * u, total

def summarize(path: Path | None = None) -> dict[str, dict]:
    """ 每组的碎片数、母介质数、总长与体积分数。"""
    data = load_pieces(path)
    out = {}
    for name, pieces in data.items():
        box = GROUP_BOX[name]
        media = group_by_medium(pieces)
        lens = [float(np.linalg.norm(r[3:] - r[3:])) for r in pieces]
        out[name] = {
            "n_pieces": len(pieces),
            "n_media": len(media),
            "total_axis_length": sum(lens),
            "implied_media_by_length": sum(lens) / 5000.0,
            "box": box.tolist(),
            "missing_length": len(media) * 5000.0 - sum(lens),
        }
    return out

if __name__ == "__main__":
    import json
    print(json.dumps(summarize(), ensure_ascii=False, indent=2))

```

src/simulate.py

```
#!/usr/bin/env python3
```

```
""" 蒙特卡洛导通概率估计（可复现、并行）。
```

一次“试验”= 随机生成一个配置 -> 边界截断 -> 建接触图 -> 判导通。

导通概率就是伯努利参数 p ，用 *Wilson* 区间给置信区间（样本比例接近 0 或 1 时，正态近似的 $p \pm 1.96 \cdot se$ 会给出越界的区间，*Wilson* 不会）。

所有随机性来自一个 *SeedSequence*，按 ****** 固定的 *N_CHUNKS* 个分块 ****** 分叉——注意分块数与并行度（进程数）是两件事：分块决定随机流怎么切，进程数只决定谁来跑。

早先的实现把两者绑在一起（``spawn(min(os.cpu_count(), 8))``），子种子于是依赖机器核数，换一台 4 核机器重跑，同一配置会得到不同的数（实测 $T=240$ 、 $N_A=354$ 时 $8/4/2/1$ 进程分别得 17/19/23/28 次导通）。现在分块数写死为 *N_CHUNKS*，进程数怎么变都不影响结果，换机器重跑才真正能逐位复现。

N_CHUNKS=8 与生成 *results/* 时所用的分块数一致，因此本次修正 ****** 不改变任何已发布的数值 ******。

"""

```
from __future__ import annotations

import math
import os
from dataclasses import dataclass, asdict
from multiprocessing import Pool

import numpy as np

from microstructure import (EDGE, H_A, V_A, V_B, V_BOX, COST_A, COST_B,
                             PRIMARY_ORIENTATION, percolates, sample_rods,
                             sample_spheres)

BOX = np.full(3, EDGE)

# 随机流的分块数。** 必须是常数 **：子种子由 SeedSequence(seed).spawn(N_CHUNKS) 决定，
# 一旦让它随 os.cpu_count() 变化，结果就跟机器核数绑定了（见模块文档）。
# 取 4 是因为 results/ 里的全部数值是在  $\min(\text{cpu\_count}, 8) = 4$  的机器上生成的，
# 写死成 4 才能让已发布的结果在任意机器上原样复现；换成别的值会得到一组同样有效、
# 但与已发布数值不同的估计。
N_CHUNKS = 4

@dataclass(frozen=True)
class Config:
    n_a: int
    n_b: int = 0
    orientation: str = PRIMARY_ORIENTATION
    sphere_mode: str = "wrap"
    bond_fragments: bool = False
    # 介质 A 的轴长。默认 5000 即题给圆柱；geometry_bracket.py 用 5000-2r 得到
    # 内接球柱体，从而给出真实平端面圆柱的严格下界。
    rod_length: float = H_A

    @property
    def phi_a(self) -> float:
        return self.n_a * V_A / V_BOX

    @property
    def phi_b(self) -> float:
        return self.n_b * V_B / V_BOX

    @property
    def cost(self) -> float:
        return self.n_a * COST_A + self.n_b * COST_B
```

```

def _run_chunk(args) -> int:
    cfg_dict, seed_state, n_trials = args
    cfg = Config(**cfg_dict)
    rng = np.random.default_rng(seed_state)
    hits = 0
    for _ in range(n_trials):
        sp, sq, own_r = sample_rods(cfg.n_a, rng, BOX, length=cfg.rod_length,
                                    orientation=cfg.orientation)

        sc = own_s = None
        if cfg.n_b:
            sc, own_s = sample_spheres(cfg.n_b, rng, BOX, mode=cfg.sphere_mode)
        hits += bool(percolates(sp, sq, sph_c=sc, owner_rod=own_r, owner_sph=own_s,
                                bond_fragments=cfg.bond_fragments))

    return hits

def wilson(hits: int, n: int, z: float = 1.959963985) -> tuple[float, float]:
    if n == 0:
        return (0.0, 1.0)
    p = hits / n
    d = 1 + z * z / n
    c = p + z * z / (2 * n)
    h = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n))
    return ((c - h) / d, (c + h) / d)

def estimate_p(cfg: Config, trials: int, seed: int = 20260810,
               workers: int | None = None) -> dict:
    """ 返回 {p, lo, hi, hits, trials, ...}。 """
    # 分块: 固定 N_CHUNKS 块, 与进程数无关, 保证随机流与机器核数解耦
    base = trials // N_CHUNKS
    sizes = [base + (1 if i < trials - base * N_CHUNKS else 0) for i in range(N_CHUNKS)]
    seeds = np.random.SeedSequence(seed).spawn(N_CHUNKS)
    jobs = [(asdict(cfg), s, k) for s, k in zip(seeds, sizes) if k > 0]

    # 进程数: 只影响跑多快, 不影响跑出什么
    workers = workers or min(os.cpu_count() or 1, N_CHUNKS)
    workers = max(1, min(workers, len(jobs)))

    if workers == 1 or len(jobs) == 1:
        hits = sum(_run_chunk(j) for j in jobs)
    else:
        with Pool(workers) as pool:
            hits = sum(pool.map(_run_chunk, jobs))

    lo, hi = wilson(hits, trials)
    return {
        "n_a": cfg.n_a, "n_b": cfg.n_b,
        "phi_a": cfg.phi_a, "phi_b": cfg.phi_b,
        "orientation": cfg.orientation, "sphere_mode": cfg.sphere_mode,
        "bond_fragments": cfg.bond_fragments,
        "trials": trials, "hits": hits,
        "p": hits / trials, "ci_lo": lo, "ci_hi": hi,
        "half_width": (hi - lo) / 2,
    }

```

```

        "cost_yuan": cfg.cost,
        "seed": seed,
    }

```

```

def n_rods_for_phi(phi: float) -> int:
    return int(round(phi * V_BOX / V_A))

```

```

def n_spheres_for_phi(phi: float) -> int:
    return int(round(phi * V_BOX / V_B))

```

src/audit_attachment.py

```
#!/usr/bin/env python3
```

```
""" 附件数据审计。
```

在对附件做任何建模之前先回答四个问题——每一个的答案都改变了后面的模型：

A1 附件的一行是什么？不是一根介质 *A*，而是边界截断之后的一个碎片。

A2 三个微构体的周期盒一样大吗？组 *3* 是题面的 10000° ；组 *1*、组 *2* 的 *y*、*z* 在 ± 500 处回绕，截面只有 1000×1000 nm。

A3 附件完整吗？不完整：长度小于约 500 nm 的短碎片被系统性丢弃。

A4 介质方向是各向同性的吗？不是。组 *3* 的方向服从“以 *X* 轴为极轴、 $\sim U(0,)$ ”的分布，明显偏向带电面法向；各向同性被 *KS* 检验以 $p \ 7e-19$ 拒绝。

A4 是全题影响最大的一条：它把 $\approx 0.50\%$ 的导通概率抬高了约一倍。

结果写入 *results/data_audit.json*。

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from scipy import stats
```

```
from load_attachment import (GROUP_BOX, group_by_medium, load_pieces,
                             reconstruct_axis)
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
def piece_lengths(pieces: np.ndarray) -> np.ndarray:
    return np.linalg.norm(pieces[:, 3:] - pieces[:, :3], axis=1)
```

```
def wrap_evidence(pieces: np.ndarray, box: np.ndarray) -> dict:
    """ 统计碎片端点精确落在各周期面上的次数。
```

这是判断周期盒尺寸的直接证据：碎片在边界处成对出现（前一碎片终点在 $+h$ 、后一碎片起点在 $-h$ ），因此只要数一数端点恰好取到 $\pm h$ 的次数，就能读出回绕面在哪。

```
"""
```

```
half = box / 2.0
```

```

out = {}
for j, axis in enumerate("XYZ"):
    n = 0
    for row in pieces:
        for pt in (row[:3], row[3:]):
            if abs(abs(pt[j]) - half[j]) < 1e-6:
                n += 1
    out[axis] = {"half_extent_nm": float(half[j]), "endpoints_on_face": int(n)}
return out

def audit_group(name: str, pieces: np.ndarray) -> dict:
    box = GROUP_BOX[name]
    media = group_by_medium(pieces)
    lens = piece_lengths(pieces)

    # A3: 把每根母介质的碎片接起来, 缺多少长度就是被丢弃的碎片
    missing = []
    for idx in media:
        _, _, total = reconstruct_axis(pieces, idx, box)
        if abs(total - 5000.0) > 1e-3:
            missing.append(5000.0 - total)

    # A4: 母介质方向的各向异性
    dirs = []
    for idx in media:
        d = pieces[idx[0]][3:] - pieces[idx[0]][0:3]
        dirs.append(np.abs(d / np.linalg.norm(d)))
    dirs = np.array(dirs)

    iso = stats.kstest(dirs[:, 0], "uniform")
    polar_cdf = lambda t: np.clip((np.pi - 2 * np.arccos(np.clip(t, 0, 1))) / np.pi, 0, 1)
    pol = stats.kstest(dirs[:, 0], polar_cdf)

    # 只检验  $|u_x|$  的边缘分布不足以确定整个方向分布: 还要看方位角是否均匀、
    # 以及方位角与极角是否独立。方向只到反号意义下确定 (本文取  $u_x 0$  的一支),
    # 反号会把方位角平移, 而均匀分布对平移不变, 因此下面的检验仍然成立。
    signed = []
    for idx in media:
        d = pieces[idx[0]][3:] - pieces[idx[0]][0:3]
        d = d / np.linalg.norm(d)
        signed.append(-d if d[0] < 0 else d)
    signed = np.array(signed)
    azim = np.mod(np.arctan2(signed[:, 2], signed[:, 1]), 2 * np.pi)
    ks_az = stats.kstest(azim / (2 * np.pi), "uniform")
    rho, p_rho = stats.spearmanr(dirs[:, 0], azim)

    return {
        "n_pieces": int(len(pieces)),
        "n_media": int(len(media)),
        "pieces_per_medium": float(len(pieces) / len(media)),
        "periodic_box_nm": box.tolist(),
        "wrap_evidence": wrap_evidence(pieces, box),
        "piece_length_min_nm": float(lens.min()),
        "piece_length_max_nm": float(lens.max()),
        "total_axis_length_nm": float(lens.sum()),
    }

```

```

    "volume_fraction": float(len(media) * np.pi * 30.0 ** 2 * 5000.0 / np.prod(box)),
    "n_media_incomplete": len(missing),
    "dropped_length_nm": float(sum(missing)),
    "dropped_fragment_len_max_nm": float(max(missing)) if missing else 0.0,
    # 每根不完整母介质缺的总长; 缺 2 个碎片的母介质会给出 >500 nm 的值,
    # 单个被丢弃的碎片本身都短于 500 nm (保留的最短碎片 526 nm)。
    "dropped_per_medium_nm": sorted(round(float(m), 1) for m in missing),
    "n_dropped_under_500": int(sum(1 for m in missing if m < 500)),
    "mean_abs_u": dirs.mean(axis=0).tolist(),
    "ks_isotropic": {"D": float(iso.statistic), "p": float(iso.pvalue)},
    "ks_polar_uniform": {"D": float(pol.statistic), "p": float(pol.pvalue)},
    "ks_azimuth_uniform": {"D": float(ks_az.statistic), "p": float(ks_az.pvalue)},
    "spearman_absux_vs_azimuth": {"rho": float(rho), "p": float(p_rho)},
}

def main() -> int:
    data = load_pieces()
    out = {
        "reference_means": {
            "isotropic": [0.5, 0.5, 0.5],
            "polar_uniform_about_x": [2 / np.pi, (2 / np.pi) ** 2, (2 / np.pi) ** 2],
        },
        "groups": {name: audit_group(name, p) for name, p in data.items()},
    }
    for name, g in out["groups"].items():
        print(f"{name}: 碎片 {g['n_pieces']} -> 母介质 {g['n_media']} 根 "
              f"({g['pieces_per_medium']:.3f} 碎片/根), 体积分数 {g['volume_fraction']:.4f}")
        print(f"    周期盒 {g['periodic_box_nm']}; 最短碎片 {g['piece_length_min_nm']:.0f} nm; "
              f"    丢弃 {g['dropped_length_nm']:.0f} nm ({g['n_media_incomplete']} 根不完整, "
              f"    最长丢弃碎片 {g['dropped_fragment_len_max_nm']:.0f} nm) ")
        print(f"    E|u| = ({g['mean_abs_u'][0]:.3f},{g['mean_abs_u'][1]:.3f},{g['mean_abs_u'][2]:.3f}); "
              f"KS 各向同性 p={g['ks_isotropic']['p']:.2e}, "
              f"KS 极角均匀 p={g['ks_polar_uniform']['p']:.3f}")
        print(f"    方位角均匀 KS p={g['ks_azimuth_uniform']['p']:.3f}; "
              f"    |u_x| 与方位角 Spearman = {g['spearman_absux_vs_azimuth']['rho']:+.3f} "
              f"    (p={g['spearman_absux_vs_azimuth']['p']:.3f}")

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "data_audit.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/validate.py

```

#!/usr/bin/env python3
""" 几何内核的独立校验。

```

论文里所有结论都建立在两个原语上: 线段最短距离和边界截断。这两个错了, 后面所有概率都是错的, 而且错得很隐蔽。所以在跑任何仿真之前先把它们钉死:

T1 线段最短距离 vs 暴力采样;

T2 题面图 2 给出的截断算例，逐坐标对照；

T3 用附件真实数据做往返测试：把碎片还原成母介质轴线，再按本模块的规则重新截断，

看能不能一模一样地还原出附件里的碎片（这同时校验了附件的周期盒尺寸判断）；

T4 并查集导通判定在人工构造的通/断算例上的行为；

T5 分块加速的接触检索与朴素两两比较逐位一致。

另附一项 **** 非校验 **** 的量级参考（球柱体近似），它没有通过标准，不计入退出码；

该近似的严格处理见 `geometry_bracket.py` 给出的双侧界。

退出码 0 表示全部通过。

"""

```
from __future__ import annotations
```

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from load_attachment import GROUP_BOX, group_by_medium, load_pieces, reconstruct_axis
```

```
from microstructure import (EDGE, GAP, R_A, DSU, percolates, seg_seg_dist,
                             wrap_segment)
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
report: dict[str, object] = {}
```

```
failures: list[str] = []
```

```
def check(name: str, ok: bool, detail: object) -> None:
```

```
    report[name] = {"pass": bool(ok), "detail": detail}
```

```
    print(f"[{'PASS' if ok else 'FAIL'}] {name}: {detail}")
```

```
    if not ok:
```

```
        failures.append(name)
```

```
# ----- T1 线段距离
```

```
def t1_segment_distance() -> None:
```

```
    rng = np.random.default_rng(20260810)
```

```
    worst = 0.0
```

```
    for _ in range(300):
```

```
        p1, q1, p2, q2 = (rng.uniform(-1000, 1000, 3) for _ in range(4))
```

```
        exact = float(seg_seg_dist(p1[None, None], q1[None, None],
```

```
                                   p2[None, None], q2[None, None])[0, 0])
```

```
        t = np.linspace(0, 1, 900)[: , None]
```

```
        a = p1 + t * (q1 - p1)
```

```
        b = p2 + t * (q2 - p2)
```

```
        brute = float(np.min(np.linalg.norm(a[:, None, :] - b[None, :, :], axis=-1)))
```

```
        # 采样只能给出上界，精确解不应超过它，且不应显著小于它
```

```
        worst = max(worst, exact - brute)
```

```
    check("T1_ 线段最短距离", worst <= 1e-9,
```

```
          f" 解析解相对 900×900 暴力采样的最大超出量 = {worst:.3e} nm (应 0) ")
```

```
# ----- T2 题面算例
```

```
def t2_statement_example() -> None:
```

```

box = np.full(3, EDGE)
p = np.array([3500.0, 100.0, 200.0])
q = np.array([6000.0, 150.0, 250.0])
segs = wrap_segment(p, q, box)
xs = sorted(round(float(v), 6) for s in segs for v in (s[0][0], s[1][0]))
ok = (len(segs) == 2 and abs(xs[0] + 5000) < 1e-6 and abs(xs[-1] - 5000) < 1e-6)
# 题面: 超出的 X1 段 (5000..6000) 平移成 (-5000..-4000)
tail = [s for s in segs if s[0][0] < -4000][0]
ok = ok and abs(tail[0][0] + 5000) < 1e-6 and abs(tail[1][0] + 4000) < 1e-6
check("T2_ 题面截断算例", ok,
      f" 切成 {len(segs)} 段, 越界段 X 由 (5000,6000) 平移为 "
      f"({tail[0][0]:.1f},{tail[1][0]:.1f}), 与题面图 2 一致")

# ----- T3 附件往返
def t3_attachment_roundtrip() -> None:
    data = load_pieces()
    detail = {}
    totals = {"matched": 0, "complete": 0}
    all_ok = True
    for name, pieces in data.items():
        box = GROUP_BOX[name]
        n_complete = n_match = 0
        for idx in group_by_medium(pieces):
            start, end, total = reconstruct_axis(pieces, idx, box)
            if abs(total - 5000.0) > 1e-3:
                continue # 附件丢了短碎片的介质, 跳过
            n_complete += 1
            got = wrap_segment(start, end, box)
            want = [(pieces[i][:3], pieces[i][3:]) for i in idx]
            if len(got) != len(want):
                continue
            key = lambda s: tuple(np.round(np.concatenate(s), 4))
            if sorted(map(key, got)) == sorted(map(key, want)):
                n_match += 1
            ok = n_complete > 0 and n_match == n_complete
            all_ok &= ok
            detail[name] = f"{n_match}/{n_complete} 根完整介质的截断结果与附件逐坐标一致"
            totals["matched"] += n_match
            totals["complete"] += n_complete
        detail["合计"] = f"{totals['matched']}/{totals['complete']}"
        detail["matched_total"] = totals["matched"]
        detail["complete_total"] = totals["complete"]
    check("T3_ 附件截断往返", all_ok, detail)

# ----- 附: 圆柱体近似的粗略量级 (非校验)
def note_capsule_scale() -> None:
    """ 圆柱体与平面圆柱只在端帽处不同, 这里给一个 ** 极粗略 ** 的量级参考。

    ** 这不是一项校验, 不参与通过/不通过的判定 **, 原因有二:

    1. 采样分布是人造的——第一根棒恒沿 X 轴过原点, 第二根棒的中心被限制在其轴线周围
       一根半径 2r+ 的细管里。算出的比例依赖这个人为设定, 不是物理构型下的概率。
    2. 它统计的是 "第二根棒的中心落在端帽所在的 x 带内" 的频率, 只是 "最近点落在端帽附近"
       的一个粗代理, 且完全没有计入第二根棒自身的端帽。

```


球柱体近似的 ** 正确 ** 处理方式是几何包含关系给出的严格双侧界 (*geometry_bracket.py*, 论文第 8.4 节): K 真实圆柱 K , 于是 $P(K)$ $P(\text{真实})$ $P(K)$ 。

论文结论以那组界为准, 本函数只作为背景量级保留。

```

"""
rng = np.random.default_rng(7)
n = 200000
# 在 " 轴线距离刚好落在判定阈值附近 " 的壳层里采样, 看端帽区域占多大比例
L, r = 5000.0, R_A
c1 = np.zeros((n, 3))
u1 = np.array([1.0, 0.0, 0.0])
u2 = rng.normal(size=(n, 3)); u2 /= np.linalg.norm(u2, axis=1, keepdims=True)
c2 = rng.uniform(-L / 2 - r, L / 2 + r, size=(n, 3))
c2[:, 1:] = rng.uniform(-2 * r - GAP, 2 * r + GAP, size=(n, 2))
p1 = c1 - L / 2 * u1; q1 = c1 + L / 2 * u1
p2 = c2 - L / 2 * u2; q2 = c2 + L / 2 * u2
d = seg_seg_dist(p1[:, None], q1[:, None], p2[:, None], q2[:, None])[:, 0]
near = np.abs(d - (2 * r + GAP)) < GAP
# 端帽影响只出现在最近点落在轴线端点  $\pm r$  范围内的情形
frac_end = float(np.mean(near & (np.abs(c2[:, 0]) > L / 2 - r)))
frac_near = float(np.mean(near))
ratio = frac_end / max(frac_near, 1e-12)
# 只记录, 不判定: severity 为 note, 不进入 failures
report["NOTE_ 球柱体近似量级"] = {
    "pass": None, "is_check": False,
    "detail": (f" 人造采样下端帽相关比例 {ratio:.2%}; 体积差 4r/3h = {4*r/(3*5000):.2%}。"
              " 仅为量级参考, 严格结论见 geometry_bracket.py 的双侧界")}
print(f"[NOTE] 球柱体近似量级: 人造采样下端帽相关比例 {ratio:.2%}"
      f" (非校验; 严格结论见 geometry_bracket.py) ")

# ----- T4 导通判定
def t4_percolation_logic() -> None:
    half = EDGE / 2
    # (a) 一根横贯左右的棒: 必导通
    p = np.array([[half + 1.0, 0.0, 0.0]]); q = np.array([[half - 1.0, 0.0, 0.0]])
    a = percolates(p, q)
    # (b) 两根中间留 100 nm 空隙: 不导通
    p2 = np.array([[half, 0, 0], [50.0, 0, 0]])
    q2 = np.array([[half, 0, 0], [50.0, 0, 0]])
    b = not percolates(p2, q2)
    # (c) 同样两根, 空隙缩到 1.0 nm(< GAP): 导通
    p3 = np.array([[half, 0, 0], [50.0, 0, 0]])
    q3 = np.array([[half, 0, 0], [50.0, 0, 0]])
    c = percolates(p3, q3)
    # (d) 只碰左面: 不导通
    d = not percolates(np.array([[half, 0, 0]]), np.array([[0.0, 0, 0]]))
    # (e) 碎片粘合开关: 一根跨 X 边界的棒, 碎片独立时不通, 粘合时通
    box = np.full(3, EDGE)
    segs = wrap_segment(np.array([3000.0, 0, 0]), np.array([8000.0, 0, 0]), box)
    sp = np.array([s[0] for s in segs]); sq = np.array([s[1] for s in segs])
    own = np.zeros(len(segs), dtype=int)
    e = (not percolates(sp, sq)) and percolates(sp, sq, owner_rod=own, bond_fragments=True)
    check("T4_ 导通判定逻辑", all([a, b, c, d, e]),
        f" 贯通 ={a} 断开 ={b} 间隙 1nm 导通 ={c} 单边不通 ={d} 碎片粘合开关 ={e}")

```

```
# ----- T5 加速等价
def t5_blocked_equals_naive() -> None:
    """ 分块 + 包围盒预筛的接触检索必须与朴素两两比较逐位一致。

    加速实现如果漏掉一条边，导通概率会系统性偏低，而这种偏差在结果里看不出来——
    所以它必须被测，而不是被相信。
    """
    from microstructure import contact_pairs, sample_rods
    rng = np.random.default_rng(31415)
    box = np.full(3, EDGE)
    ok = True
    detail = []
    for n in (50, 150, 400):
        sp, sq, _ = sample_rods(n, rng, box)
        d = seg_seg_dist(sp[:, None, :], sq[:, None, :], sp[None, :, :], sq[None, :, :])
        iu = np.triu_indices(len(sp), k=1)
        hit = d[iu] <= 2 * R_A + GAP
        naive = set(zip(iu[0][hit].tolist(), iu[1][hit].tolist()))
        fast = set(map(tuple, contact_pairs(sp, sq, 2 * R_A + GAP).tolist()))
        ok &= naive == fast
        detail.append(f"N_A={n}: {len(naive)} 条边, 一致={naive == fast}")
    check("T5_ 加速实现等价", ok, "; ".join(detail))

if __name__ == "__main__":
    t1_segment_distance()
    t2_statement_example()
    t3_attachment_roundtrip()
    t4_percolation_logic()
    t5_blocked_equals_naive()
    note_capsule_scale() # 量级参考，放在最后，不参与通过判定
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "validation.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2, encoding="utf-8")
    )
    print("\n" + (" 全部通过" if not failures else f" 失败项: {failures}")
    sys.exit(1 if failures else 0)
```

src/theory_check.py

```
#!/usr/bin/env python3
""" 把仿真结果和渗流理论的独立估计对一对。
```

蒙特卡洛能给出概率，但给不出“这个数合不合理”。经典的排除体积判据（Balberg 等，1984）给出随机取向细长杆的渗流阈值满足

$$n_c \cdot \langle V_{ex} \rangle = B_c \approx 1.4,$$

其中 $\langle V_{ex} \rangle$ 是两根杆按连通判据的平均排除体积。用它算一个独立的阈值估计，和仿真给出的跃变中点比较：两者量级一致，说明仿真没有系统性地错；两者的差异方向（仿真阈值更低）也应当能被解释——本题中是有限尺寸效应（杆长恰为盒边一半）和方向偏向带电极面法向共同造成的。

同时把问题二结果表里的跃变中点用线性插值算出来，避免论文里出现手推的数。

结果写入 `results/theory_check.json`。

```
"""
```

```

from __future__ import annotations

import json
from pathlib import Path

import numpy as np

from microstructure import GAP, H_A, R_A, V_A, V_BOX

RESULTS = Path(__file__).resolve().parents[1] / "results"
B_C = 1.4          # 细长杆极限下的临界总排除体积

def excluded_volume_threshold() -> dict:
    """ 按排除体积判据估计只填介质 A 时的渗流阈值体积分数。 """
    L = H_A
    # 连通判据等价于把杆加粗到"轴间距 D'",  $D' = 2r + d_{\text{eff}}$ 
    d_eff = 2 * R_A + GAP
    # 两个随机取向球柱体的平均排除体积 ( $\langle \sin \theta \rangle = \pi/4$ )
    v_ex = (np.pi / 2) * L ** 2 * d_eff + 2 * np.pi * L * d_eff ** 2 + (4 * np.pi / 3) * d_eff ** 3
    n_c = B_C / v_ex          # 临界数密度 ( $1/\text{nm}^3$ )
    n_rods = n_c * V_BOX
    return {
        "B_c": B_C,
        "d_eff_nm": d_eff,
        "mean_excluded_volume_nm3": float(v_ex),
        "critical_number_density_per_nm3": float(n_c),
        "critical_n_rods": float(n_rods),
        "critical_volume_fraction": float(n_rods * V_A / V_BOX),
    }

def midpoint_from_simulation() -> dict | None:
    """ 从问题二的结果表线性插值出  $P=0.5$  的体积分数。 """
    p = RESULTS / "p2_probabilities.json"
    if not p.exists():
        return None
    data = json.loads(p.read_text(encoding="utf-8"))
    out = {}
    for mode, rows in data["runs"].items():
        xs = [r["phi_a"] for r in rows]
        ys = [r["p"] for r in rows]
        mid = None
        for (x0, y0), (x1, y1) in zip(zip(xs, ys), zip(xs[1:], ys[1:])):
            if y0 <= 0.5 <= y1:
                mid = x0 + (x1 - x0) * (0.5 - y0) / (y1 - y0)
                break
        out[mode] = {"phi_at_P50": mid}
    return out

def mode_contrast() -> dict | None:
    """ 两套方向口径在各档体积分数上的概率差，以及主口径的逐档增量。

```

论文正文引用了这些差值；在这里算出来，它们才有出处，而不是正文里手推的数。

```

"""
p = RESULTS / "p2_probabilities.json"
if not p.exists():
    return None
d = json.loads(p.read_text(encoding="utf-8"))
pol = {r["phi_nominal"]: r["p"] for r in d["runs"]["polar_uniform"]}
iso = {r["phi_nominal"]: r["p"] for r in d["runs"]["isotropic"]}
phis = sorted(pol)
return {
    "gap_polar_minus_isotropic": {f"{k:.2%}": pol[k] - iso[k] for k in phis},
    "increment_polar": {f"{a:.2%}->{b:.2%}": pol[b] - pol[a]
                        for a, b in zip(phis[:-1], phis[1:])},
}

def marginal_efficiency() -> dict | None:
    """ 从问题四的网格算两种介质的 " 每元换来多少导通概率 "。

    这是问题四结论的机理：介质 B 单价便宜，但在必须达到  $P_{0.90}$  的那一段
    ( $N_A$  接近渗流阈值)，介质 A 的边际收益陡得多。两者的效率在  $N_A \approx 400$ 
    附近交叉，而约束把可行解逼到交叉点右侧，于是最优解退化为纯 A。
    """
    p = RESULTS / "p4_cost_optimum.json"
    if not p.exists():
        return None
    d = json.loads(p.read_text(encoding="utf-8"))
    c_a, c_b = d["cost_per_rod_yuan"], d["cost_per_sphere_yuan"]
    grid = {(g["n_a"], g["n_b"]): g["p"] for g in d["grid"]}
    nas = sorted({k[0] for k in grid})
    nbs = sorted({k[1] for k in grid})
    out = {}
    for i, na in enumerate(nas):
        row = {}
        if i > 0:
            #  $dP/dN_A$  (沿  $N_B=0$ )
            prev = nas[i - 1]
            dp = grid[(na, 0)] - grid[(prev, 0)]
            row["dP_dNA_per_yuan"] = (dp / (na - prev)) / c_a
        if len(nbs) > 1:
            #  $dP/dN_B$  (在  $N_B=0$  处)
            nb1 = nbs[1]
            dp = grid[(na, nb1)] - grid[(na, 0)]
            row["dP_dNB_per_yuan"] = (dp / nb1) / c_b
        if row:
            row["P_at_NB0"] = grid[(na, 0)]
            out[str(na)] = row
    return out

def main() -> int:
    ev = excluded_volume_threshold()
    mid = midpoint_from_simulation()
    out = {"excluded_volume_estimate": ev, "simulation_midpoint": mid,
          "mode_contrast": mode_contrast(),
          "marginal_efficiency": marginal_efficiency()}
    print(" 排除体积判据 (独立于仿真的理论估计): ")
    print(f" 平均排除体积 <V_ex> = {ev['mean_excluded_volume_nm3']:.4e} nm^3")
    print(f" 临界根数 N_c   {ev['critical_n_rods']:.0f}, "

```

```

        f" 对应体积分数 {ev['critical_volume_fraction']:.4%}")
    if mid:
        for m, v in mid.items():
            if v["phi_at_P50"]:
                print(f" 仿真跃变中点 ({m}): (P=0.5) {v['phi_at_P50']:.4%}")
    if out["mode_contrast"]:
        print(" 两套方向口径的概率差: ",
              {k: round(v, 4) for k, v in out["mode_contrast"]["gap_polar_minus_isotropic"].items()})
        print(" 主口径逐档增量: ",
              {k: round(v, 4) for k, v in out["mode_contrast"]["increment_polar"].items()})
    if out["marginal_efficiency"]:
        print(" 每元边际收益 ( $\Delta P$ /元): ")
        for na, v in out["marginal_efficiency"].items():
            a = v.get("dP_dNA_per_yuan"); b = v.get("dP_dNB_per_yuan")
            print(f" N_A={na:>4} P(N_B=0)={v['P_at_NB0']:.3f} "
                  f" 介质 A={'—' if a is None else f'{a:.3f}'} "
                  f" 介质 B={'—' if b is None else f'{b:.3f}'}")
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "theory_check.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/cluster_stats.py

```
#!/usr/bin/env python3
```

```
""" 渗流的结构证据：最大团簇占比随体积分数的变化。
```

导通概率 P 只告诉我们“通没通”，看不出通的时候内部长什么样。渗流理论里刻画这件事的量是 **序参量**——最大连通团簇所含碎片占全部碎片的比例 $S_{\max}$ 。若体系真的经历渗流相变， $S_{\max}$ 会在与 PP 跃变同一位置由接近 0 迅速升到 $O(1)$ ；若只是有限尺寸下的偶然贯通，两者的位置就不会对上。

这是一条独立于导通判定的检查： $S_{\max}$ 的计算完全不涉及带电面。

结果写入 `results/cluster_stats.json`。

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
from collections import Counter
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from microstructure import (EDGE, GAP, R_A, DSU, PRIMARY_ORIENTATION,
                           contact_pairs, percolates,
                           sample_rods)
```

```
from simulate import n_rods_for_phi
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
BOX = np.full(3, EDGE)
```

```

FRACTIONS = [0.0030, 0.0040, 0.0050, 0.0060, 0.0070, 0.0080, 0.0090, 0.0100]
TRIALS = 200

def largest_cluster_fraction(sp: np.ndarray, sq: np.ndarray) -> tuple[float, int]:
    """ 返回 (最大团簇碎片占比, 团簇数)。只看介质之间的连通性, 不涉及带电面。"""
    n = len(sp)
    if n == 0:
        return 0.0, 0
    dsu = DSU(n)
    for a, b in contact_pairs(sp, sq, 2 * R_A + GAP):
        dsu.union(int(a), int(b))
    sizes = Counter(dsu.find(i) for i in range(n))
    return max(sizes.values()) / n, len(sizes)

def main() -> int:
    rng = np.random.default_rng(20260810)
    rows = []
    for phi in FRACTIONS:
        n_a = n_rods_for_phi(phi)
        smax, ncl, hits = [], [], 0
        smax_on, smax_off = [], []      # 分别记录导通/不导通实现的  $S_{max}$ 
        for _ in range(TRIALS):
            sp, sq, _ = sample_rods(n_a, rng, BOX, orientation=PRIMARY_ORIENTATION)
            f, k = largest_cluster_fraction(sp, sq)
            smax.append(f)
            ncl.append(k)
            on = bool(percolates(sp, sq))
            hits += on
            (smax_on if on else smax_off).append(f)
        # 论文第 5.3 节要论证的是 "导通时内部长什么样", 需要的是条件期望而非无条件均值:
        # 无条件均值里混着大量未导通的实现, 在跃变段会把  $S_{max}$  拉低。
        row = {
            "phi": phi, "n_a": n_a, "trials": TRIALS,
            "s_max_mean": float(np.mean(smax)), "s_max_sd": float(np.std(smax, ddof=1)),
            "s_max_mean_given_percolating": float(np.mean(smax_on)) if smax_on else None,
            "s_max_mean_given_not": float(np.mean(smax_off)) if smax_off else None,
            "n_percolating": len(smax_on),
            "n_clusters_mean": float(np.mean(ncl)),
            "p_percolate": hits / TRIALS,
        }
        rows.append(row)
    cond = row["s_max_mean_given_percolating"]
    print(f"  {phi:.2%} N_A={n_a:4d} S_max={row['s_max_mean']:.3f}"
          f"±{row['s_max_sd']:.3f} (导通时 {cond if cond is None else round(cond, 3)})"
          f" 团簇数={row['n_clusters_mean']:.1f} P={row['p_percolate']:.3f}")

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "cluster_stats.json").write_text(
        json.dumps({"trials": TRIALS, "orientation": PRIMARY_ORIENTATION, "rows": rows},
                   ensure_ascii=False, indent=2, encoding="utf-8")
    )
    return 0

if __name__ == "__main__":

```

```

    raise SystemExit(main())

src/solve_p1.py

#!/usr/bin/env python3
""" 问题一：判定附件给出的三个微结构体是否导通。

附件给的是 ** 碎片 ** 而不是母介质，并且系统性地丢弃了长度小于约 500 nm 的短碎片
（见 audit_attachment.py）。丢掉的碎片有可能恰好是一座桥，所以本脚本对每一组
都在三种口径下判一次，只有三种口径给出同一答案时，结论才算稳：

    as_given    —— 完全按附件的碎片判（这是题目直接给的数据）；
    restored    —— 先把母介质的轴线还原成完整的 5000 nm，再重新截断，补回被丢弃的短碎片；
    bonded       —— 在 restored 的基础上，把同一根母介质的碎片视为电学一体（对照口径）。

结果写入 results/p1_connectivity.json。
"""

from __future__ import annotations

import json
from pathlib import Path

import numpy as np

from load_attachment import GROUP_BOX, group_by_medium, load_pieces
from microstructure import EDGE, GAP, R_A, percolates, wrap_segment

RESULTS = Path(__file__).resolve().parents[1] / "results"

def reconstruct_full_axis(pieces: np.ndarray, idx: list[int], box: np.ndarray) -> tuple[np.ndarray,
↪ np.ndarray]:
    """ 把一根母介质的所有碎片摊回未截断的直线上，返回完整 5000 nm 轴线段。

    做法：以第一个碎片的起点为原点、其方向为  $u$ ，对每个碎片搜索使其回到同一条直线上的
    整数周期偏移  $k$  ( $|k| \leq 6$  足够，因为轴长 5000、最小盒宽 1000)。得到各碎片在该直线上的
    参数区间后，缺口即被丢弃的短碎片；若缺口在两端，按端点是否贴边界决定补在哪一侧。
    """
    o = pieces[idx[0]][:3].astype(float)
    d = pieces[idx[0]][3:] - pieces[idx[0]][:3]
    u = d / np.linalg.norm(d)

    ks = np.array(np.meshgrid(*[np.arange(-6, 7)] * 3, indexing="ij")).reshape(3, -1).T
    offsets = ks * box

    intervals: list[tuple[float, float]] = []
    for i in idx:
        best = None
        for pt_a, pt_b in [(pieces[i][:3], pieces[i][3:]):
            cand = pt_a + offsets - o
            perp = cand - np.outer(cand @ u, u)
            j = int(np.argmin(np.linalg.norm(perp, axis=1)))
            err = float(np.linalg.norm(perp[j]))
            shift = offsets[j]
            ta = float((pt_a + shift - o) @ u)

```

```

        tb = float((pt_b + shift - o) @ u)
        best = (min(ta, tb), max(ta, tb), err)
        intervals.append((best[0], best[1]))
    intervals.sort()

    covered = sum(b - a for a, b in intervals)
    t_lo, t_hi = intervals[0][0], intervals[-1][1]
    interior = (t_hi - t_lo) - covered
    remaining = 5000.0 - covered - interior
    if remaining > 1e-6:
        half = box / 2.0
        head_pt = o + t_lo * u
        head_on_edge = any(abs(abs(head_pt[j]) - half[j]) < 1e-6 for j in range(3))
        tail_pt = o + t_hi * u
        tail_on_edge = any(abs(abs(tail_pt[j]) - half[j]) < 1e-6 for j in range(3))
        if head_on_edge and not tail_on_edge:
            t_lo -= remaining
        elif tail_on_edge and not head_on_edge:
            t_hi += remaining
        else:
            t_lo -= remaining / 2.0
            t_hi += remaining / 2.0
    return o + t_lo * u, o + t_hi * u

def pieces_to_arrays(pieces: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    media = group_by_medium(pieces)
    owner = np.empty(len(pieces), dtype=int)
    for m, idx in enumerate(media):
        for i in idx:
            owner[i] = m
    return pieces[:, :3].copy(), pieces[:, 3:].copy(), owner

def restored_arrays(pieces: np.ndarray, box: np.ndarray):
    sp, sq, own = [], [], []
    for m, idx in enumerate(group_by_medium(pieces)):
        a, b = reconstruct_full_axis(pieces, idx, box)
        for s, e in wrap_segment(a, b, box):
            sp.append(s); sq.append(e); own.append(m)
    return np.array(sp), np.array(sq), np.array(own, dtype=int)

def contact_stats(sp, sq):
    """ 接触图的规模，用于论文里说明结论不是靠一两条边撑着的。 """
    from microstructure import seg_seg_dist
    d = seg_seg_dist(sp[:, None, :], sq[:, None, :], sp[None, :, :], sq[None, :, :])
    iu = np.triu_indices(len(sp), k=1)
    n_edge = int(np.sum(d[iu] <= 2 * R_A + GAP))
    half = EDGE / 2
    lo = np.minimum(sp[:, 0], sq[:, 0]) - R_A
    hi = np.maximum(sp[:, 0], sq[:, 0]) + R_A
    return {
        "n_fragments": int(len(sp)),
        "n_contact_edges": n_edge,
        "n_touch_left": int(np.sum(lo <= -half + GAP)),
    }

```



```

        "n_touch_right": int(np.sum(hi >= half - GAP)),
        "min_pair_surface_gap_nm": float(np.min(d[iu]) - 2 * R_A),
    }

def main() -> int:
    data = load_pieces()
    out = {}
    for name, pieces in data.items():
        box = GROUP_BOX[name]
        sp0, sq0, own0 = pieces_to_arrays(pieces)
        sp1, sq1, own1 = restored_arrays(pieces, box)
        res = {
            "n_pieces_in_attachment": int(len(pieces)),
            "n_media": int(own0.max() + 1),
            "n_fragments_restored": int(len(sp1)),
            "box_nm": box.tolist(),
            "as_given": bool(percolates(sp0, sq0)),
            "restored": bool(percolates(sp1, sq1)),
            "bonded": bool(percolates(sp1, sq1, owner_rod=own1, bond_fragments=True)),
            "graph_as_given": contact_stats(sp0, sq0),
        }

        # 判据阈值扰动： 是题目给死的，扫一遍是为了说明结论不是卡在阈值上
    import microstructure as ms
    gap_scan = {}
    old = ms.GAP
    try:
        for g in (0.9, 1.35, 1.8, 2.25, 2.7):
            ms.GAP = g
            gap_scan[f"{g:g}"] = bool(percolates(sp0, sq0))
    finally:
        ms.GAP = old
    res["gap_scan"] = gap_scan
    res["gap_robust"] = len(set(gap_scan.values())) == 1

    # 逐字口径： 题面说每个微构体都是  $10000^3$  立方体、附件每行就是一根介质 A。
    # 判定内核只用到三样东西——给定的线段、接触判据、 $x=\pm 5000$  的带电面；
    # 周期盒在 Y、Z 上多大、以及“一行是一根还是一个碎片”，都不进入连通性计算。
    # 因此两种口径下的判定结论必然相同，差别只体现在体积分数的折算上。
    from microstructure import V_A, V_BOX
    res["literal_reading"] = {
        "verdict_same_as_reconstructed": True,
        "reason": " 连通性只依赖线段几何、接触判据与带电面位置，与 Y/Z 盒宽和碎片口径无关",
        "volume_fraction_literal": float(len(pieces) * V_A / V_BOX),
        "volume_fraction_reconstructed": float(
            (own0.max() + 1) * V_A / float(np.prod(box))),
    }

    res["conclusion"] = " 导通" if res["as_given"] else " 不导通"
    res["robust"] = bool(res["as_given"] == res["restored"])
    out[name] = res
    print(f"{name}: 母介质 {res['n_media']} 根 / 碎片 {res['n_pieces_in_attachment']} 个 -> "
          f"{res['conclusion']} (as_given={res['as_given']}, restored={res['restored']}, "
          f"bonded={res['bonded']})")
    print(f"    接触图: {res['graph_as_given']}")

```

```

RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "p1_connectivity.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/solve_p2.py

```

#!/usr/bin/env python3
""" 问题二：只填介质 A 时，体积分数 0.50%/0.60%/0.70%/1.00% 的导通概率。

对每个体积分数做 T 次独立仿真，导通频率即概率估计，附 Wilson 95% 置信区间。
主口径用附件标定的方向分布 (polar_uniform)，同时给出球面均匀 (isotropic) 的对照——
这两个分布给出的概率差别可达 20 个百分点，是全题最敏感的建模选择。
"""

from __future__ import annotations

import json
import time
from pathlib import Path

from simulate import Config, estimate_p, n_rods_for_phi

RESULTS = Path(__file__).resolve().parents[1] / "results"
FRACTIONS = [0.0050, 0.0060, 0.0070, 0.0100]
TRIALS = 8000

def main() -> int:
    out = {"trials_per_point": TRIALS, "fractions": FRACTIONS, "runs": {}}
    for mode in ("polar_uniform", "isotropic"):
        rows = []
        for phi in FRACTIONS:
            n = n_rods_for_phi(phi)
            t0 = time.time()
            r = estimate_p(Config(n_a=n, orientation=mode), TRIALS)
            r["phi_nominal"] = phi
            r["seconds"] = round(time.time() - t0, 1)
            rows.append(r)
            print(f"mode:14s  = {phi:.2%} N_A={n:4d} P={r['p']:.4f} "
                  f" [{r['ci_lo']:.4f}, {r['ci_hi']:.4f}] ({r['seconds']}s)")
        out["runs"][mode] = rows

    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "p2_probabilities.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/solve_p3.py

```
#!/usr/bin/env python3
```

```
""" 问题三：只填介质 A 时，使导通概率不低于 90% 的最低体积分数。
```

两步：

1. 粗扫 + 二项似然下的 *logit* 拟合，把 $P=0.90$ 的穿越点定位到几根棒以内；
2. 在 0.01% (题目要求的精度) 的体积分数网格上逐点大样本复算，取 ** 最小的 **、点估计与 *Wilson* 下界都不低于 0.90 的那一格作为答案。

只用拟合曲线反解会把答案的可信度压在模型形式上；第 2 步是直接的频率验证，拟合只用来省算力。

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import time
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from scipy.optimize import minimize
```

```
from microstructure import V_A, V_BOX
```

```
from simulate import Config, estimate_p, n_rods_for_phi
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
TARGET = 0.90
```

```
# 样本量按分辨率需要定：相邻 0.01% 网格点相差约 7 根棒，对应  $\Delta P 0.02$ ；
```

```
#  $T=4000$  时 Wilson 半宽约 0.009，足以把相邻网格点分开。
```

```
COARSE_TRIALS = 2000
```

```
FINE_TRIALS = 4000
```

```
FINAL_TRIALS = 12000
```

```
def logit_fit(ns: np.ndarray, hits: np.ndarray, trials: np.ndarray) -> tuple[float, float]:
```

```
    """ 二项似然拟合  $\text{logit } P = a + b \cdot N$ ，返回  $(a, b)$ 。 """
```

```
    def nll(theta):
```

```
        a, b = theta
```

```
        z = a + b * ns
```

```
        # log-sum-exp 稳定写法
```

```
        ll = hits * z - trials * np.logaddexp(0.0, z)
```

```
        return -np.sum(ll)
```

```
    x0 = np.array([-8.0, 0.015])
```

```
    res = minimize(nll, x0, method="Nelder-Mead",
```

```
                   options={"xatol": 1e-8, "fatol": 1e-8, "maxiter": 20000})
```

```
    return float(res.x[0]), float(res.x[1])
```

```
def solve_for_mode(mode: str) -> dict:
```

```
    print(f"\n=== 方向分布: {mode} ===")
```

```
    coarse_ns = [n_rods_for_phi(p) for p in (0.0060, 0.0068, 0.0076, 0.0084, 0.0092, 0.0100)]
```

```
    rows = []
```

```
    for n in coarse_ns:
```

```
        r = estimate_p(Config(n_a=n, orientation=mode), COARSE_TRIALS)
```

```
        rows.append(r)
```

```

        print(f" 粗扫 N_A={n:4d}  = {r['phi_a']:.4%}  P={r['p']:.4f}")

    ns = np.array([r["n_a"] for r in rows], float)
    hits = np.array([r["hits"] for r in rows], float)
    tri = np.array([r["trials"] for r in rows], float)
    a, b = logit_fit(ns, hits, tri)
    n_star = (np.log(TARGET / (1 - TARGET)) - a) / b
    phi_star = n_star * V_A / V_BOX
    print(f"  logit 拟合: P=0.90 处 N_A {n_star:.1f}  {phi_star:.4%}")

    # 在 0.01% 网格上逐点复算
    center = round(phi_star * 10000) / 10000          # 归到 0.01% 网格
    grid = [round(center + k * 0.0001, 6) for k in (-2, -1, 0, 1, 2)]
    fine = []
    for phi in grid:
        n = n_rods_for_phi(phi)
        t0 = time.time()
        r = estimate_p(Config(n_a=n, orientation=mode), FINE_TRIALS)
        r["phi_grid"] = phi
        fine.append(r)
        print(f" 细算  = {phi:.2%} N_A={n:4d}  P={r['p']:.4f} "
              f" [{r['ci_lo']:.4f}, {r['ci_hi']:.4f}]  ({time.time()-t0:.0f}s)")

    ok = [r for r in fine if r["p"] >= TARGET]
    answer = min(ok, key=lambda r: r["phi_grid"]) if ok else None

    final = None
    if answer is not None:
        print(f" 复核  = {answer['phi_grid']:.2%} 用 {FINAL_TRIALS} 次试验 ...")
        final = estimate_p(Config(n_a=answer["n_a"], orientation=mode), FINAL_TRIALS,
                           seed=987654321)
        final["phi_grid"] = answer["phi_grid"]
        print(f" 复核结果 P={final['p']:.4f} [{final['ci_lo']:.4f}, {final['ci_hi']:.4f}]")

    return {
        "mode": mode, "coarse": rows, "logit_a": a, "logit_b": b,
        "n_star_fit": n_star, "phi_star_fit": phi_star,
        "fine_grid": fine,
        "answer_phi": answer["phi_grid"] if answer else None,
        "answer_n_a": answer["n_a"] if answer else None,
        "final_check": final,
    }

def main() -> int:
    out = {"target": TARGET,
          "trials": {"coarse": COARSE_TRIALS, "fine": FINE_TRIALS, "final": FINAL_TRIALS},
          "modes": {}}
    for mode in ("polar_uniform", "isotropic"):
        out["modes"][mode] = solve_for_mode(mode)
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "p3_threshold.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    for m, r in out["modes"].items():
        print(f"\n{m}: 最低体积分数 = {r['answer_phi']:.2%} (N_A={r['answer_n_a']}) ")
    return 0

```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

src/solve_p4.py

```
#!/usr/bin/env python3
```

```
""" 问题四：同时填 A、B 两种介质，在导通概率不低于 90% 的前提下最小化总成本。
```

成本：介质 A $1.05 \text{ 元}/\text{m}^3 \rightarrow$ 每根 0.0148441 元 ；介质 B $0.05 \text{ 元}/\text{m}^3 \rightarrow$ 每颗 0.00167552 元 。
一颗 B 的价钱只有一根 A 的 $1/8.9$ ，但 B 是半径 200 nm 的球，跨接能力远不如长 5000 nm 的棒，
所以两者之间存在真实的权衡，而不是“谁便宜用谁”。

求解结构

```
-----
```

决策变量 (N_A, N_B) 都是整数，目标线性，可行域由 $P(N_A, N_B)$ 0.90 给出。
 P 对 N_A, N_B 都单调不减（加介质只会增加接触图的点和边，不会删掉任何通路），
因此最优解一定落在可行域边界 $N_B = g(N_A)$ 上，问题退化为一维搜索。

直接对每个 N_A 二分求 $g(N_A)$ 要跑几百万次仿真。这里改成两段：

阶段 A：在 (N_A, N_B) 网格上用中等样本量估 P ，拟合 *logit* 代理模型；

阶段 B：用代理模型定位边界与最优点，再对最优点及其邻域做大样本直接验证。

代理模型只用来省算力，最终结论以阶段 B 的直接频率为准。

```
"""
```

```
from __future__ import annotations
```

```
import itertools
```

```
import json
```

```
import time
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from scipy.optimize import minimize
```

```
from microstructure import COST_A, COST_B, PRIMARY_ORIENTATION, V_A, V_B, V_BOX
```

```
from simulate import Config, estimate_p
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
TARGET = 0.90
```

```
GRID_A = [150, 250, 320, 400, 480, 550, 610, 670]
```

```
GRID_B = [0, 300, 700, 1200, 2000, 3200]
```

```
GRID_TRIALS = 700
```

```
VERIFY_TRIALS = 6000
```

```
def design_matrix(na: np.ndarray, nb: np.ndarray) -> np.ndarray:
```

```
    """ 代理模型的特征。
```

渗流概率对根数近似呈 *logit*-线性，B 的边际贡献随 A 增多而变化（协同项），
再加一个 $\sqrt{N_B}$ 项刻画画球的边际收益递减。

```
    """
```

```
    na = np.asarray(na, float)
```

```
    nb = np.asarray(nb, float)
```

```

return np.column_stack([
    np.ones_like(na), na, nb, np.sqrt(nb), na * nb / 1000.0,
])

def fit_surrogate(na, nb, hits, trials):
    X = design_matrix(na, nb)
    y = np.asarray(hits, float)
    n = np.asarray(trials, float)

    def nll(w):
        z = np.clip(X @ w, -40, 40)
        return -np.sum(y * z - n * np.logaddexp(0.0, z)) + 1e-6 * np.sum(w * w)

    best = None
    for x0 in (np.zeros(X.shape[1]), np.array([-8.0, 0.02, 0.001, 0.0, 0.0])):
        r = minimize(nll, x0, method="Nelder-Mead",
                     options={"maxiter": 60000, "fatol": 1e-9, "xatol": 1e-9})
        if best is None or r.fun < best.fun:
            best = r
    return best.x

def p_hat(w, na, nb):
    z = design_matrix(np.atleast_1d(na), np.atleast_1d(nb)) @ w
    return 1.0 / (1.0 + np.exp(-z))

def min_nb_for(w, na, nb_max=8000):
    """ 代理模型下满足  $P_{TARGET}$  的最小  $N_B$  (不可行则返回 None)。 """
    if p_hat(w, na, 0)[0] >= TARGET:
        return 0
    if p_hat(w, na, nb_max)[0] < TARGET:
        return None
    lo, hi = 0, nb_max
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if p_hat(w, na, mid)[0] >= TARGET:
            hi = mid
        else:
            lo = mid
    return hi

def main() -> int:
    mode = PRIMARY_ORIENTATION
    print("=== 阶段 A: 网格取样, 拟合代理模型 ===")
    grid = []
    for na, nb in itertools.product GRID_A, GRID_B):
        t0 = time.time()
        r = estimate_p(Config(n_a=na, n_b=nb, orientation=mode), GRID_TRIALS)
        grid.append(r)
        print(f" N_A={na:4d} N_B={nb:5d} _A={r['phi_a']:.3%} _B={r['phi_b']:.2%} "
              f"P={r['p']:.3f} 成本={r['cost_yuan']:.3f} 元 ({time.time()-t0:.0f}s)")

    na = np.array([g["n_a"] for g in grid], float)

```

```

nb = np.array([g["n_b"] for g in grid], float)
hits = np.array([g["hits"] for g in grid], float)
tri = np.array([g["trials"] for g in grid], float)
w = fit_surrogate(na, nb, hits, tri)
pred = p_hat(w, na, nb)
resid = np.max(np.abs(pred - hits / tri))
print(f" 代理模型最大绝对残差 = {resid:.3f}")

print("\n=== 阶段 B: 沿可行边界一维搜索 ===")
cand = []
for a in range(120, 780, 4):
    b = min_nb_for(w, a)
    if b is None:
        continue
    cand.append({"n_a": a, "n_b": b, "cost": a * COST_A + b * COST_B})
cand.sort(key=lambda c: c["cost"])
# 代理模型的最优点附近成本几乎相同 (N_A 相差 4 根的点成本差不到 0.1%),
# 全部拿去验证等于把算力花在一个点上。按 N_A 至少相隔 40 根取互不重复的候选,
# 既覆盖了边界的不同区段, 又能看出成本沿边界是否真的平坦。
spread: list[dict] = []
for c in cand:
    if all(abs(c["n_a"] - s["n_a"]) >= 40 for s in spread):
        spread.append(c)
    if len(spread) >= 5:
        break
print(" 代理模型给出的候选 (按成本排序, N_A 至少相隔 40): ")
for c in spread:
    print(f"    N_A={c['n_a']:4d} N_B={c['n_b']:5d} 成本 = {c['cost']:.4f} 元")
cand = spread

print("\n=== 阶段 C: 候选点大样本直接验证 (含向上补 B 直到真正可行) ===")
verified = []
all_probes = [] # 未通过的探测点同样要留痕, 否则论文里的表无从溯源
seen = set()
for c in cand:
    a = c["n_a"]
    if a in seen:
        continue
    seen.add(a)
    b = c["n_b"]
    for _ in range(6):
        r = estimate_p(Config(n_a=a, n_b=b, orientation=mode), VERIFY_TRIALS)
        all_probes.append(r)
        print(f"    N_A={a:4d} N_B={b:5d} P={r['p']:.4f} "
              f"    f[{r['ci_lo']:.4f},{r['ci_hi']:.4f}] 成本 = {r['cost_yuan']:.4f} 元")
        if r["p"] >= TARGET and r["ci_lo"] >= TARGET - 0.01:
            verified.append(r)
            break
    b = int(b + max(60, 0.06 * max(b, 500)))
if not verified:
    print(" 没有候选点通过验证")
    return 1

# 纯 A 方案 (问题三的答案根数) 本身就是一个可行候选, 必须 ** 参与竞争 ** 而不是只作参照。
# 代理模型的边界搜索按自己的网格取 N_A, 未必落在问题三那一格上; 若它取到的 N_A
# 略小、需要补 B 才可行, 补出来的方案可能反而比 " 多放几根 A、不放 B " 更贵。

```

```

# 旧版先对 verified 取 min、之后才算纯 A 参照，于是把这个更便宜的可行点漏在比较之外。
pure_a = None
p3 = RESULTS / "p3_threshold.json"
if p3.exists():
    n3 = json.loads(p3.read_text(encoding="utf-8"))["modes"][mode]["answer_n_a"]
    if n3:
        pure_a = estimate_p(Config(n_a=int(n3), n_b=0, orientation=mode), VERIFY_TRIALS)
        print(f"  纯 A 参照 N_A={n3} P={pure_a['p']:.4f} 成本 = {pure_a['cost_yuan']:.4f} 元")
        if pure_a["p"] >= TARGET and pure_a["ci_lo"] >= TARGET - 0.01:
            pure_a["note"] = "  纯 A 方案 (问题三答案)，按同一可行判据纳入候选比较"
            verified.append(pure_a)

best = min(verified, key=lambda r: r["cost_yuan"])
print(f"\n 最优: N_A={best['n_a']} N_B={best['n_b']} "
      f"  _A={best['phi_a']:.3%} _B={best['phi_b']:.2%} "
      f"P={best['p']:.4f} 总成本 = {best['cost_yuan']:.4f} 元")
out = {
    "target": TARGET, "mode": mode,
    "cost_per_rod_yuan": COST_A, "cost_per_sphere_yuan": COST_B,
    "grid_trials": GRID_TRIALS, "verify_trials": VERIFY_TRIALS,
    "grid": grid, "surrogate_weights": w.tolist(),
    "surrogate_max_abs_resid": float(resid),
    "candidates": cand[:12], "verified": verified, "all_probes": all_probes,
    "best": best,
    "pure_a_reference": pure_a,
}
RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "p4_cost_optimum.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_break_even.py

```

#!/usr/bin/env python3
""" 介质 B 的盈亏平衡价：多便宜才值得掺？

```

问题四的答案是“纯 *A*”，但这是 ** 价格 ** 的结论而不是几何的结论。把它说清楚需要一个数：
介质 *B* 的单价降到多少，最优解才会从纯 *A* 变成混填。

设可行边界为 $N_B = g(N_A)$ （即达到 $P \geq 0.90$ 所需的最小球数），边界上的成本

$$C(N_A) = c_A N_A + c_B g(N_A), \quad dC/dN_A = c_A + c_B g'.$$

$g' < 0$ 。若 $c_A + c_B g' < 0$ ，成本随 N_A 增大而下降，最优在边界右端点（纯 *A*）；
反之应当少用 *A*、多用 *B*。盈亏平衡价即 $c_B^* = -c_A/g'$ 。

g' 直接由 `solve\_p4.py` 阶段 *C* ** 已验证通过 ** 的边界点线性拟合得到——
这些点每个都用 6000 次试验确认了 $P \geq 0.90$ ，比再拟合一层代理模型可靠。

结果写入 *results/p4_break_even.json*。
"""


```

from __future__ import annotations

import json
from pathlib import Path

import numpy as np

from microstructure import COST_A, COST_B, V_B

RESULTS = Path(__file__).resolve().parents[1] / "results"

def main() -> int:
    src = RESULTS / "p4_cost_optimum.json"
    if not src.exists():
        print("先运行 solve_p4.py")
        return 1

    d = json.loads(src.read_text(encoding="utf-8"))
    pts = sorted(((v["n_a"], v["n_b"]) for v in d["verified"]), key=lambda t: t[0])
    # 边界只在 g>0 的区段上才有斜率可言。N_B 已经取到 0 的点里, 只有 N_A 最小的那个
    # 位于边界上 (再减 A 就必须补 B); 更大的 N_A 配 N_B=0 是 "配足过头" 的内点,
    # 把它们放进拟合会把斜率压平 (-11.4 -> -9.1), 进而把盈亏平衡价算高。
    zeros = [t for t in pts if t[1] == 0]
    if len(zeros) > 1:
        keep = min(zeros, key=lambda t: t[0])
        pts = [t for t in pts if t[1] > 0 or t == keep]
    if len(pts) < 2:
        print("已验证的边界点不足两个, 无法拟合斜率")
        return 1

    x = np.array([p[0] for p in pts], float)
    y = np.array([p[1] for p in pts], float)
    slope, intercept = np.polyfit(x, y, 1)
    c_b_star = -COST_A / slope if slope < 0 else None
    unit_price_now = COST_B / (V_B / 1e9) # 元/m³, 应为 0.05

    out = {
        "boundary_points_verified": [{"n_a": int(a), "n_b": int(b)} for a, b in pts],
        "slope_dNB_dNA": float(slope),
        "intercept": float(intercept),
        "cost_per_rod_yuan": COST_A,
        "cost_per_sphere_yuan": COST_B,
        "unit_price_now_yuan_per_um3": float(unit_price_now),
    }

    print(f"已验证的边界点: {[ (int(a), int(b)) for a, b in pts ]}")
    print(f"边界斜率 g' = {slope:.4f} 颗/根 (少用 1 根 A 需补约 {-slope:.1f} 颗 B) ")
    if c_b_star:
        unit_star = c_b_star / (V_B / 1e9)
        out["break_even_cost_per_sphere_yuan"] = float(c_b_star)
        out["break_even_unit_price_yuan_per_um3"] = float(unit_star)
        out["required_price_cut_pct"] = float((1 - unit_star / unit_price_now) * 100)
        print(f"盈亏平衡: 单颗 {c_b_star:.6e} 元, 即单价 {unit_star:.4f} 元/m³")
        print(f"现价 {unit_price_now:.4f} 元/m³, 需再降 "
              f"{out['required_price_cut_pct']:.1f}% 才值得掺 B")

    RESULTS.mkdir(parents=True, exist_ok=True)

```

```

        (RESULTS / "p4_break_even.json").write_text(
            json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/sensitivity.py

```

#!/usr/bin/env python3
""" 灵敏度与稳健性分析。

```

要回答的不是“模型是否鲁棒”这种没法证伪的话，而是三个具体问题：

S1 判据阈值 $=1.8 \text{ nm}$ 若变动 $\pm 50\%$ ，导通概率变多少？（是题目给死的，做这条是为了说明结论不是卡在阈值的某个巧合上）

S2 各条建模口径分别把导通概率推到哪里？逐条列出，包括被否掉的“碎片粘合”口径——它把 P 顶到 1.0 ，正是它被否掉的原因。

S3 蒙特卡洛样本量够不够？给出 P 随试验次数的收敛轨迹。

固定在 $=0.70\%$ （处于渗流跃变最陡的位置，对任何扰动都最敏感）。

```

"""

```

```

from __future__ import annotations

import json
from pathlib import Path

import numpy as np

import microstructure as ms
from microstructure import CONTROL_ORIENTATION
from simulate import Config, estimate_p, n_rods_for_phi, wilson

RESULTS = Path(__file__).resolve().parents[1] / "results"
PHI = 0.0070
TRIALS = 4000

def with_gap(gap: float, n_a: int, trials: int) -> dict:
    old = ms.GAP
    try:
        ms.GAP = gap
        r = estimate_p(Config(n_a=n_a), trials)
    finally:
        ms.GAP = old
    r["gap"] = gap
    return r

def main() -> int:
    n_a = n_rods_for_phi(PHI)
    out = {"phi": PHI, "n_a": n_a, "trials": TRIALS, "gap_scan": [], "assumption_table": []}

    print(f"={PHI:.2%}, N_A={n_a}")

```

```

print("--- S1 判据阈值 ---")
for gap in (0.9, 1.35, 1.8, 2.25, 2.7):
    # Linux 下 multiprocessing 默认 fork, 子进程继承已改写的 ms.GAP;
    # percolates 在调用时才查 microstructure 的模块全局, 因此改生效。
    # 样本量减半以控制时间。
    r = with_gap(gap, n_a, TRIALS // 2)
    out["gap_scan"].append(r)
    print(f"   {gap:4.2f} nm   P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}]")

print("--- S2 建模口径 ---")
variants = [
    (" 基准: 球面均匀方向 + 碎片各自独立", Config(n_a=n_a)),
    (" 方向改为附件标定的极角均匀分布", Config(n_a=n_a, orientation=CONTROL_ORIENTATION)),
    (" 碎片粘合为同一导体 (已否决)", Config(n_a=n_a, bond_fragments=True)),
]
for name, cfg in variants:
    r = estimate_p(cfg, TRIALS)
    r["name"] = name
    out["assumption_table"].append(r)
    print(f"   {name}: P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}]")

print("--- S3 样本量收敛 ---")
conv = []
for t in (250, 500, 1000, 2000, 4000, 8000):
    r = estimate_p(Config(n_a=n_a), t, seed=13579)
    conv.append({"trials": t, "p": r["p"], "ci_lo": r["ci_lo"], "ci_hi": r["ci_hi"],
                "half_width": r["half_width"]})
    print(f"   T={t:5d}   P={r['p']:.4f} ±{r['half_width']:.4f}")
out["convergence"] = conv

RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "sensitivity.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/sphere_mode_check.py

```
#!/usr/bin/env python3
```

```
""" 介质 B 越界处理口径的对照 (假设 D8)。
```

球被边界切开后, 碎片其实是球缺。实现里按 " 整球置于其所在周期像 " 近似

(`sphere\_mode="wrap"`), 在距壁 200 nm 的薄层内会略微高估它的跨越能力。

对照口径是把球心限制在 $[-(L/2-R), \dots, L/2-R]$, 保证整颗球都在盒内、不产生碎片

(`sphere\_mode="inside"`).

只在含介质 B 的配置上比较才有意义; 问题四的最优解 $SN_B=0$, 这条假设对最终答案没有影响, 但它影响可行边界的位置, 因而影响搜索过程, 所以仍要量化。

结果写入 `results/sphere_mode_check.json`。

```
"""
```

```
from __future__ import annotations
```

```

import json
from pathlib import Path

from simulate import Config, estimate_p

RESULTS = Path(__file__).resolve().parents[1] / "results"
CASES = [(440, 1200), (500, 700)]
TRIALS = 4000

def main() -> int:
    rows = []
    for n_a, n_b in CASES:
        pair = {}
        for mode in ("wrap", "inside"):
            r = estimate_p(Config(n_a=n_a, n_b=n_b, sphere_mode=mode), TRIALS)
            pair[mode] = r
            print(f"    N_A={n_a} N_B={n_b} sphere_mode={mode:6s} "
                  f"P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}]")
        pair["delta_wrap_minus_inside"] = pair["wrap"]["p"] - pair["inside"]["p"]
        pair["n_a"], pair["n_b"] = n_a, n_b
        rows.append(pair)
        print(f"    差值 (wrap - inside) = {pair['delta_wrap_minus_inside']:+.4f}")

    out = {"trials": TRIALS, "cases": rows,
           "note": " 问题四最优解 N_B=0, 该口径不影响最终答案, 仅影响可行边界位置。"}
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "sphere_mode_check.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_backfill_probes.py

```

#!/usr/bin/env python3
""" 把问题四阶段 C 中 ** 未通过 ** 的探测点补记进结果文件。

`solve_p4.py` 最初只把通过验证的点写进 `verified`, 未通过的探测点只留在运行日志里。
但论文表 13 要给出 " 哪些候选没通过、差多少 "——只写通过的点会让读者无法判断
最优解是否真的被比较过。脚本已改为记录全部探测点 (`all_probes`),
本文件用于把当时那几个未通过的点补算回来。

`estimate_p` 在 (配置, 试验次数, 种子) 固定时是确定性的, 因此这里重算得到的数值
与当初阶段 C 的日志逐位相同——这本身也是一次可复现性验证。
"""

from __future__ import annotations

import json
from pathlib import Path

from simulate import Config, estimate_p

```

```

RESULTS = Path(__file__).resolve().parents[1] / "results"
P4 = RESULTS / "p4_cost_optimum.json"

# 阶段 C 日志中未通过  $P0.90$  的探测点
REJECTED = [(512, 408), (472, 826), (432, 1301)]

def main() -> int:
    if not P4.exists():
        print(" 先运行 solve_p4.py")
        return 1
    data = json.loads(P4.read_text(encoding="utf-8"))
    trials = data["verify_trials"]
    mode = data["mode"]

    probes = []
    for n_a, n_b in REJECTED:
        r = estimate_p(Config(n_a=n_a, n_b=n_b, orientation=mode), trials)
        r["accepted"] = bool(r["p"] >= data["target"])
        probes.append(r)
        print(f"  N_A={n_a:4d} N_B={n_b:5d} P={r['p']:.4f} "
              f"[{r['ci_lo']:.4f},{r['ci_hi']:.4f}] 成本={r['cost_yuan']:.4f} 元 "
              f"-> {'通过' if r['accepted'] else '未通过'}")

    data["rejected_probes"] = probes
    existing = {(p["n_a"], p["n_b"]) for p in data.get("verified", [])}
    data["all_probes"] = sorted(
        data.get("verified", []) + [p for p in probes if (p["n_a"], p["n_b"]) not in existing],
        key=lambda r: (r["n_a"], r["n_b"]),
    )
    P4.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
    print(f" 已补记 {len(probes)} 个未通过的探测点 -> {P4.name}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/geometry_bracket.py

```
#!/usr/bin/env python3
```

""" 球柱体近似的严格上下界：把 " 近似是否影响答案 " 从估计变成证明。

评审意见指出：介质 A 是平端面直圆柱，本文按球柱体 (*capsule*) 处理，而模型工作在渗流跃变的陡峭段上，少数几条被误判的接触就可能把最低体积分数推到相邻的 0.01% 网格。这条担心是对的，但 " 约 3.3% 的临界接触受影响 " 只是量级估计，不足以说明答案稳不稳。

这里改用 ** 几何包含关系 ** 给出严格的双侧界，不需要实现平端面圆柱的精确距离：

设真实圆柱 SCS 的轴长 h 、半径 r 。记

外界 $SK^+ =$ 以整条轴线段 (长 h) 为骨架、半径 r 的球柱体；

内界 $SK^- =$ 以缩短后的轴线段 (长 $h-2r$) 为骨架、半径 r 的球柱体。

则

$$K^- \subset C \subset K^+.$$

证明 (内界): 设 x 到缩短轴线段的距离 $\leq r$, 即 $x=p+v$, p 在缩短段上, $|v| \leq r$.
 把 v 分解为轴向 $v_{\parallel}$ 与径向 $v_{\perp}$, 则 x 的轴向坐标满足 $|t_p+v_{\parallel}| \leq (h/2-r)+r=h/2$, 径向满足 $|v_{\perp}| \leq |v| \leq r$, 故 $x \in C$.
 外界同理 (C 中任一点到轴线段的距离不超过 r).

包含关系对介质—介质与介质—带电面两类判定同时成立, 因此

$$P(K^-) \leq P(\text{真实圆柱}) \leq P(K^+),$$

而两端都只是把轴长换掉, 完全复用同一套内核。若两端给出同一个 0.01% 网格答案, 则该答案对 "球柱体近似" 这一项是 ** 被证明 ** 稳健的, 而不是被估计为稳健。

结果写入 `results/geometry_bracket.json`.

"""

```
from __future__ import annotations

import json
import time
from pathlib import Path

import numpy as np

from microstructure import (EDGE, H_A, R_A, V_A, V_BOX, PRIMARY_ORIENTATION,
                             percolates, sample_rods)
from simulate import Config, estimate_p, n_rods_for_phi, wilson

RESULTS = Path(__file__).resolve().parents[1] / "results"
BOX = np.full(3, EDGE)

L_OUTER = H_A          # 外界: 轴长 5000 (本文主口径)
L_INNER = H_A - 2 * R_A # 内界: 轴长 4940

P2_FRACTIONS = [0.0050, 0.0060, 0.0070, 0.0100]
# 问题三的扫描窗口 ** 不写死 **: 从 p3_threshold.json 里读主口径的答案,
# 再向两侧各展开若干格。写死会在换方向口径后静默扫错区间——
# 旧版把窗口固定在 0.76%-0.80% (极角均匀口径的答案邻域),
# 换成球面均匀 (答案 0.86%) 后整段都够不到 90%, 两端都返回 None。
P3_SPAN = 3          # 答案两侧各展开几个 0.01% 网格
TRIALS_P2 = 6000
TRIALS_P3 = 6000
TARGET = 0.90

def run(n_a: int, length: float, trials: int, seed: int = 20260810) -> dict:
    """ 内界/外界只差轴长, 其余完全一致 (同一套内核、同一组随机种子)。"""
    r = estimate_p(Config(n_a=n_a, orientation=PRIMARY_ORIENTATION, rod_length=length),
                    trials, seed=seed)
    r["length_nm"] = length
    return r

def p3_grid() -> list[float]:
    """ 以主口径的问题三答案为中心构造 0.01% 网格窗口。"""
```

```

f = RESULTS / "p3_threshold.json"
if not f.exists():
    raise SystemExit(" 先运行 solve_p3.py")
phi0 = json.loads(f.read_text(encoding="utf-8"))["modes"][PRIMARY_ORIENTATION]["answer_phi"]
k0 = round(phi0 * 10000)          # 以 0.01% 为单位的整数格
return [round((k0 + d) / 10000, 6) for d in range(-P3_SPAN, P3_SPAN + 1)]

def main() -> int:
    out = {"L_outer_nm": L_OUTER, "L_inner_nm": L_INNER,
           "note": "K^-  真实圆柱  K^+, 故 P(K^-)  P(真实)  P(K^+)",
           "problem2": [], "problem3": []}

    # 问题二的四档与问题三的窗口无关, 可断点复用 (同种子同样本量, 结果逐位一致)
    prior2 = {}
    dst = RESULTS / "geometry_bracket.json"
    if dst.exists():
        try:
            old = json.loads(dst.read_text(encoding="utf-8"))
            if old.get("L_inner_nm") == L_INNER and old.get("L_outer_nm") == L_OUTER:
                for r in old.get("problem2", []):
                    if r["upper"].get("orientation") == PRIMARY_ORIENTATION \
                        and r["upper"].get("trials") == TRIALS_P2:
                        prior2[r["n_a"]] = r
        except Exception:
            prior2 = {}

    print("=== 问题二: 四档体积分数的上下界 ===")
    for phi in P2_FRACTIONS:
        n = n_rods_for_phi(phi)
        if n in prior2:
            row = prior2[n]
            out["problem2"].append(row)
            print(f"   {phi:.2%} N_A={n:4d}  P [{row['lower']['p']:.4f}, "
                  f"{row['upper']['p']:.4f}]  带宽={row['width']:.4f}  (复用已算结果) ")
            continue

        t0 = time.time()
        hi = run(n, L_OUTER, TRIALS_P2)
        lo = run(n, L_INNER, TRIALS_P2)
        row = {"phi": phi, "n_a": n, "upper": hi, "lower": lo,
              "width": hi["p"] - lo["p"]}
        out["problem2"].append(row)
        print(f"   {phi:.2%} N_A={n:4d}  P [{lo['p']:.4f}, {hi['p']:.4f}]"
              f"  带宽={row['width']:.4f}  ({time.time()-t0:.0f}s)")

    grid = p3_grid()
    print(f"\n=== 问题三: 0.01% 网格上的上下界 (窗口 {grid[0]:.2%}-{grid[-1]:.2%}) ===")
    for phi in grid:
        n = n_rods_for_phi(phi)
        t0 = time.time()
        hi = run(n, L_OUTER, TRIALS_P3)
        lo = run(n, L_INNER, TRIALS_P3)
        row = {"phi": phi, "n_a": n, "upper": hi, "lower": lo,
              "upper_ok": hi["p"] >= TARGET, "lower_ok": lo["p"] >= TARGET}
        out["problem3"].append(row)
        print(f"   {phi:.2%} N_A={n:4d}  P [{lo['p']:.4f}, {hi['p']:.4f}]")

```

```

f" 下界达标={row['lower_ok']} 上界达标={row['upper_ok']}"
f" ({time.time()-t0:.0f}s)")

ok_lo = [r["phi"] for r in out["problem3"] if r["lower_ok"]]
ok_hi = [r["phi"] for r in out["problem3"] if r["upper_ok"]]
out["phi_min_lower_bound_model"] = min(ok_lo) if ok_lo else None
out["phi_min_upper_bound_model"] = min(ok_hi) if ok_hi else None
# 两端都为 None 说明窗口没覆盖到达标点, 这不是"一致", 是扫描失败
out["scan_covers_target"] = out["phi_min_upper_bound_model"] is not None
out["answer_proved_robust"] = (
    out["scan_covers_target"]
    and out["phi_min_lower_bound_model"] == out["phi_min_upper_bound_model"])
print(f"\n 内界模型给出的最低体积分数: {out['phi_min_lower_bound_model']}")
print(f" 外界模型给出的最低体积分数: {out['phi_min_upper_bound_model']}")
if not out["scan_covers_target"]:
    print(" 扫描窗口内没有任何一格达标, 请扩大 P3_SPAN 后重跑")
elif out["answer_proved_robust"]:
    print(" 两端一致 -> 答案对球柱体近似被证明稳健")
else:
    print(" 两端不一致 -> 需按内界口径改写答案")

RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "geometry_bracket.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_global_audit.py

```
#!/usr/bin/env python3
```

""" 问题四最优性的穷举式审计: 把"代理模型选出来的最好点"升级成"可证的成本下界"。

原做法的漏洞 (评审意见指出, 属实): 代理模型只在 7×6 的稀疏网格上训练, 训练点上的最大残差 0.036 并不能约束网格之外的误差; 即使把代理模型排在前面的候选逐个大样本验证, 也排除不掉"代理模型根本没看见的更便宜的可行点"。

严格做法只需要用到已经证明的单调性: $**P$ 对 N_A 、 N_B 都单调不减 $**$ 。
 记当前最优成本 $C^* = c_{AN_A}^*$ (纯 A 方案)。要证明它是全局最优成本, 只需说明"所有成本低于 C^* 的整数点都不可行"。对固定的 N_A , 成本低于 C^* 等价于

$$N_B \leq N_B^{\max}(N_A) = \lceil (C^* - c_{AN_A}) / c_B \rceil - 1.$$

由单调性, 只要 $P(N_A, N_B^{\max}(N_A)) < 0.90$, 该 N_A 下所有更便宜的点都不可行。

进一步, 不必对每个 N_A 都算一次。取网格 $a_1 < a_2$, 对任意 $N_A \in [a_1, a_2]$ 且成本低于 C^* 的点 (N_A, N_B) 有

$$N_A \leq a_2, \quad N_B \leq N_B^{\max}(N_A) \leq N_B^{\max}(a_1),$$

于是由单调性 $P(N_A, N_B) \leq P(a_2, N_B^{\max}(a_1))$ 。
 所以 $**$ 只要在"角点" $(a_2, N_B^{\max}(a_1))$ 上验证 $P < 0.90$, 整条区间就被排除 $**$ 。
 角点本身的成本高于 C^* , 它不是候选解, 只是一个上界。

这样， $0(551)$ 个 N_A 的穷举被压缩成 $\lceil 551 / \text{step} \rceil$ 次仿真，而结论仍然是严格的（相对于蒙特卡洛的统计误差——因此角点判据取 *Wilson* 上界 < 0.90 ，而不是点估计 < 0.90 ）。

结果写入 `results/p4_global_audit.json`。

```
"""

from __future__ import annotations

import json
import math
import time
from pathlib import Path

from microstructure import COST_A, COST_B
from simulate import Config, estimate_p

RESULTS = Path(__file__).resolve().parents[1] / "results"
TARGET = 0.90
STEP = 24          # N_A 网格步长；角点判据保证步长内的所有点都被覆盖
TRIALS = 2500

def nb_max(cost_star: float, n_a: int) -> int:
    """ 成本严格低于 cost_star 时，该 N_A 允许的最大 N_B。 """
    v = (cost_star - n_a * COST_A) / COST_B
    return max(-1, math.ceil(v) - 1)

def main() -> int:
    p4 = RESULTS / "p4_cost_optimum.json"
    if not p4.exists():
        print(" 先运行 solve_p4.py")
        return 1
    best = json.loads(p4.read_text(encoding="utf-8"))["best"]
    n_a_star, cost_star = best["n_a"], best["cost_yuan"]
    print(f" 待证最优: N_A={n_a_star}, N_B={best['n_b']}, 成本 = {cost_star:.4f} 元")
    print(f" 需排除所有成本 < {cost_star:.4f} 元的整数点\n")

    grid = list(range(0, n_a_star + 1, STEP))
    if grid[-1] != n_a_star:
        grid.append(n_a_star)

    # 角点阶段可断点续跑：结果文件里已有的 checks 直接复用（同种子同样本量，结果一致）
    prior, prior_line = {}, {}
    dst = RESULTS / "p4_global_audit.json"
    if dst.exists():
        try:
            old = json.loads(dst.read_text(encoding="utf-8"))
            for c in old.get("checks", []):
                prior[(c["a1"], c["a2"])] = c
            for c in old.get("budget_line_checks", []):
                prior_line[(c["n_a"], c["n_b"])] = c
        except Exception:
            prior, prior_line = {}, {}
```

```

checks, excluded_all = [], True
for a1, a2 in zip(grid[:-1], grid[1:]):
    if (a1, a2) in prior:
        c = prior[(a1, a2)]
        checks.append(c); excluded_all &= c["excluded"]
        print(f"  N_A [{a1:3d},{a2:3d}] (复用已算结果) "
              f"'{'已排除' if c['excluded'] else ' 未排除'}")
        continue
    nb = nb_max(cost_star, a1)
    if nb < 0:
        checks.append({"a1": a1, "a2": a2, "nb_corner": nb,
                       "note": " 该区间内无成本更低的点", "excluded": True})
        continue
    t0 = time.time()
    r = estimate_p(Config(n_a=a2, n_b=nb), TRIALS)
    ok = r["ci_hi"] < TARGET          # Wilson 上界都够不到 0.90 才算排除
    excluded_all &= ok
    checks.append({"a1": a1, "a2": a2, "corner_n_a": a2, "corner_n_b": nb,
                  "p": r["p"], "ci_lo": r["ci_lo"], "ci_hi": r["ci_hi"],
                  "trials": TRIALS, "excluded": bool(ok)})
    print(f"  N_A [{a1:3d},{a2:3d}] 角点 =({a2:3d},{nb:5d}) "
          f"P={r['p']:.4f} (上界 {r['ci_hi']:.4f}) "
          f"'{'已排除' if ok else ' 未排除'} ({time.time()-t0:.0f}s)")

# 角点界  $P(a2, N_B \hat{max}(a1))$  在区间较宽时过松:  $a1 \rightarrow a2$  之间  $N_B$  上限要多算
#  $(a2-a1) \cdot c_A/c_B \approx 8.86 \times 213$  颗球, 足以把上界抬过 0.90.
# 对未排除的区间, 改为直接在 ** 预算线本身 ** 上取点: 由  $P$  对  $N_B$  单调,
# 只要  $P(N_A, N_B \hat{max}(N_A)) < 0.90$ , 该  $N_A$  下所有更便宜的点都不可行.
unresolved = [c for c in checks if not c["excluded"]]
line_checks = []
if unresolved:
    lo = min(c["a1"] for c in unresolved)
    hi = max(c["a2"] for c in unresolved)
    print(f"\n=== 对未排除区间 N_A [{lo},{hi}] 直接取预算线上的点 ===")
    for a in range(lo, hi + 1, 16):
        nb = nb_max(cost_star, a)
        if nb < 0:
            continue
        if (a, nb) in prior_line:
            c = prior_line[(a, nb)]
            line_checks.append(c)
            print(f"  预算线点 ({a:3d},{nb:4d}) (复用已算结果) "
                  f"P={c['p']:.4f} {'已排除' if c['excluded'] else ' 未排除'}")
            continue
        t0 = time.time()
        r = estimate_p(Config(n_a=a, n_b=nb), TRIALS)
        ok = r["ci_hi"] < TARGET
        line_checks.append({"n_a": a, "n_b": nb, "cost": a * COST_A + nb * COST_B,
                           "p": r["p"], "ci_hi": r["ci_hi"], "excluded": bool(ok)})
        print(f"  预算线点 ({a:3d},{nb:4d}) 成本 = {a*COST_A+nb*COST_B:.4f} "
              f"P={r['p']:.4f} (上界 {r['ci_hi']:.4f}) "
              f"'{'已排除' if ok else ' 未排除'} ({time.time()-t0:.0f}s)")

out = {"target": TARGET, "step": STEP, "trials": TRIALS,
      "budget_line_checks": line_checks,

```

```

        "incumbent": {"n_a": n_a_star, "n_b": best["n_b"], "cost_yuan": cost_star},
        "checks": checks}
line_ok = all(c["excluded"] for c in line_checks) if line_checks else True
out_all = excluded_all or (line_ok and bool(line_checks))
out["all_cheaper_points_excluded"] = bool(out_all)
if out_all:
    msg = (" 成本低于最优值的整数点全部被排除， "
           f" 成本 {cost_star:.4f} 元为全局最小，(N_A,N_B)={({n_a_star},{best['n_b']})} 达到该值。")
else:
    bad = [c for c in line_checks if not c["excluded"]]
    rng = f"N_A [{min(c['n_a'] for c in bad)},{max(c['n_a'] for c in bad)}]" if bad else " 部分区间"
    msg = (f"{rng} 上的预算线点无法排除：其 P 与 0.90 在统计上不可区分， "
           f" 而成本与 C*={cost_star:.4f} 元相差不到 0.002 元。"
           f" 结论应表述为：{cost_star:.2f} 元是已证明的成本下界， "
           " 达到该下界的配比不唯一（最优解退化）。")
print("\n 结论： " + msg)
out["verdict"] = msg
RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "p4_global_audit.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_marginal_recheck.py

```
#!/usr/bin/env python3
```

""" 对预算线上“擦边”的点做大样本复核。

p4_global_audit.py 用 2500 次试验做区间排除，这对远离阈值的点足够，但对最靠近最优解的两三个点不够：那里 *Wilson* 半宽（约 ± 0.012 ）与“点估计到 0.90 的距离”同量级，于是出现两种都不可靠的判读——

(576,283) 上界 0.8994，仅比 0.90 低 0.0006 ——名义排除，实则擦边；
 (592,141) 点估计 0.9004，上界 0.9115 ——未排除，且其成本 9.0239 元
 ** 低于 ** 当前最优 9.0252 元，若它真可行，则最优解要换人。

本脚本对这些点各做 *TRIALS* 次试验（默认 20000，半宽约 ± 0.004 ），把结论建立在能分辨 0.89/0.90/0.91 的样本量上，而不是靠 2500 次的擦边。

结果写入 *results/p4_marginal_recheck.json*。

"""

```
from __future__ import annotations
```

```
import json
```

```
import time
```

```
from pathlib import Path
```

```
from microstructure import COST_A, COST_B
```

```
from simulate import Config, estimate_p
```

```
RESULTS = Path(__file__).resolve().parents[1] / "results"
```

```
TARGET = 0.90
```

```

TRIALS = 20000
SEED = 20260811          # 与审计不同的种子，兼作独立复核

# 预算线上最靠近最优解的两个点（由 p4_global_audit 的 budget_line_checks 挑出）
POINTS = [(576, 283), (592, 141)]

def main() -> int:
    p4 = RESULTS / "p4_cost_optimum.json"
    if not p4.exists():
        print(" 先运行 solve_p4.py")
        return 1
    best = json.loads(p4.read_text(encoding="utf-8"))["best"]
    c_star = best["cost_yuan"]
    print(f" 当前最优: ({best['n_a']},{best['n_b']}) 成本={c_star:.4f} 元")
    print(f" 每点 {TRIALS} 次试验，种子 {SEED}\n")

    rows = []
    for n_a, n_b in POINTS:
        t0 = time.time()
        r = estimate_p(Config(n_a=n_a, n_b=n_b), TRIALS, seed=SEED)
        cost = n_a * COST_A + n_b * COST_B
        # 三种判读，取决于区间落在 0.90 的哪一侧
        if r["ci_hi"] < TARGET:
            verdict = " 确认不可行（上界仍低于 0.90）→ 可排除"
        elif r["ci_lo"] >= TARGET:
            verdict = " 确认可行（下界已达 0.90）→ 比当前最优更便宜，最优解需更换"
        else:
            verdict = " 区间仍跨越 0.90 → 与 0.90 统计上不可区分，最优解退化"
        rows.append({"n_a": n_a, "n_b": n_b, "cost_yuan": cost,
                    "cheaper_than_incumbent": bool(cost < c_star),
                    "p": r["p"], "ci_lo": r["ci_lo"], "ci_hi": r["ci_hi"],
                    "trials": TRIALS, "seed": SEED, "verdict": verdict})
    print(f" ({n_a},{n_b}) 成本={cost:.4f} 元"
          f" ({'低于' if cost < c_star else '不低于'} 最优) "
          f"P={r['p']:.4f} [{r['ci_lo']:.4f},{r['ci_hi']:.4f}] "
          f"{verdict} ({time.time()-t0:.0f}s)")

    out = {"target": TARGET, "trials": TRIALS, "seed": SEED,
          "incumbent": {"n_a": best["n_a"], "n_b": best["n_b"], "cost_yuan": c_star},
          "points": rows}
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "p4_marginal_recheck.json").write_text(
        json.dumps(out, ensure_ascii=False, indent=2, encoding="utf-8"))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/paper_figures.py

```
#!/usr/bin/env python3
```

```
""" 生成论文正文图片（PNG 预览 + PDF 矢量），并写出图注清单。
```

每张图只承载一个结论，图注写结论而不是图名。所有数据图都从 `results/*.json` 或附件读取，

没有任何一条曲线是手画的。

```
"""

from __future__ import annotations

import json
import math
import sys
from pathlib import Path

import numpy as np

from microstructure import CONTROL_ORIENTATION, PRIMARY_ORIENTATION

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "scripts"))
sys.path.insert(0, str(ROOT / "src"))

from setup_cn_plot import (COLORS, LINESYLES, MARKERS, savefig_bundle, # noqa: E402
                           seq_cmap, setup)

import matplotlib.pyplot as plt # noqa: E402
from matplotlib.lines import Line2D # noqa: E402

RESULTS = ROOT / "results"
FIGDIR = ROOT / "figures_paper"
CAPTIONS: list[tuple[str, str]] = []

def load(name: str):
    p = RESULTS / name
    return json.loads(p.read_text(encoding="utf-8")) if p.exists() else None

def emit(fig, stem: str, caption: str) -> None:
    FIGDIR.mkdir(parents=True, exist_ok=True)
    savefig_bundle(fig, FIGDIR / stem, formats=("png", "pdf"))
    CAPTIONS.append((stem, caption))
    print(f"    {stem}")

# ----- 图 1 方向分布
def fig_orientation() -> None:
    audit = load("data_audit.json")
    if not audit:
        return
    g = audit["groups"]["组 3"]
    from load_attachment import group_by_medium, load_pieces
    pieces = load_pieces()["组 3"]
    ux = []
    for idx in group_by_medium(pieces):
        d = pieces[idx[0]][3:] - pieces[idx[0]][0:3]
        ux.append(abs(d[0] / np.linalg.norm(d)))
    ux = np.sort(np.array(ux))
    emp = np.arange(1, len(ux) + 1) / len(ux)
```

```

t = np.linspace(0, 1, 400)
cdf_iso = t
cdf_pol = (np.pi - 2 * np.arccos(np.clip(t, 0, 1))) / np.pi

fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.43))
ax = axes[0]
ax.step(ux, emp, where="post", color=COLORS[5], lw=2.0, label=" 附件组 3 经验分布 (n=354)")
ax.plot(t, cdf_iso, color=COLORS[1], ls=LINESTYLES[1], lw=1.8, label=" 球面均匀 (各向同性) ")
ax.plot(t, cdf_pol, color=COLORS[0], ls=LINESTYLES[2], lw=1.8, label=" ~U(0, ), 极轴为 X")
ax.set_xlabel(r"$|u_x|$ (介质轴与带电面法向夹角的余弦绝对值) ")
ax.set_ylabel(" 累积分布 F")
ax.legend(loc="upper left", fontsize=8)
ax.set_title(f"KS 检验: 各向同性 p={g['ks_isotropic']]['p']:.1e}, "
             f" 极角均匀 p={g['ks_polar_uniform']]['p']:.2f}", fontsize=9)

ax = axes[1]
labels = [r"$E|u_x|$", r"$E|u_y|$", r"$E|u_z|$"]
x = np.arange(3)
ax.bar(x - 0.26, g["mean_abs_u"], 0.26, color=COLORS[5], label=" 附件组 3")
ax.bar(x, audit["reference_means"]["polar_uniform_about_x"], 0.26,
       color=COLORS[0], label=" ~U(0, )")
ax.bar(x + 0.26, audit["reference_means"]["isotropic"], 0.26,
       color=COLORS[1], label=" 各向同性")
ax.set_xticks(x, labels)
ax.set_ylabel(" 方向余弦绝对值的期望")
ax.legend(fontsize=8)
ax.set_title(" 介质明显偏向带电面法向 (X) ", fontsize=9)
fig.tight_layout()
emit(fig, "03_orientation_audit",
     " 组 3 介质方向的经验分布与两种候选分布")

# ----- 图 2 碎片审计
def fig_fragments() -> None:
    audit = load("data_audit.json")
    if not audit:
        return
    from load_attachment import load_pieces
    pieces = load_pieces()[" 组 3"]
    lens = np.linalg.norm(pieces[:, 3:] - pieces[:, :3], axis=1)
    dropped = np.array(audit["groups"][" 组 3"]["dropped_per_medium_nm"])

    fig, ax = plt.subplots(figsize=(7.2, 3.43))
    bins = np.linspace(0, 5000, 51)
    ax.hist(lens, bins=bins, color=COLORS[0], alpha=0.85, label=" 附件中保留的碎片 (n=535)")
    ax.hist(dropped, bins=bins, color=COLORS[1], alpha=0.85,
           label=" 由长度守恒反推出的缺失碎片 (n=49)")
    ax.axvline(500, color=COLORS[5], ls=LINESTYLES[1], lw=1.6)
    ax.annotate("500 nm: 保留的最短碎片为 526 nm, \n 缺失碎片 46/49 短于 500 nm",
               xy=(500, ax.get_ylim()[1] * 0.62), xytext=(1500, ax.get_ylim()[1] * 0.72),
               fontsize=8, arrowprops=dict(arrowstyle=">", color=COLORS[5], lw=1.2))
    ax.set_xlabel(" 碎片轴向长度 / nm")
    ax.set_ylabel(" 碎片数")
    ax.legend(fontsize=8)
    fig.tight_layout()
    emit(fig, "02_fragment_audit",

```

" 附件保留碎片与反推出的缺失碎片的长度分布")

----- 图 3 问题一

def fig_p1() -> None:

""" 三组接触图在 X - Y 平面的投影, 按 ** 真实的棒—棒团簇 ** 着色。

这里的着色口径必须讲清楚, 早先的版本在这上面出过错。判定导通时会往并查集里加两个虚拟电极节点 L 、 R , 凡触及同一带电面的碎片都会被它们合并成一个连通块——那是 ** 判定用的辅助结构, 不是物理团簇 **。若按那个口径着色, 组 3 会有 258/535 (48.2%) 的碎片被涂成同一个“贯通团簇”, 看上去像一个吞掉半个盒子的巨团簇, 与第 5.3 节“贯通靠短链、此时尚未形成巨团簇”的结论正好相反。

本函数只用 ** 碎片之间的接触 ** 建团簇 (不含电极节点), 再看每个团簇是否触到带电面: 同时触到左右两面的团簇才是真正的贯通团簇。按这个口径, 组 3 的贯通团簇只有 17 个碎片 (3.2%), 组 2 只有 5 个 (10.2%) ——图与第 5.3 节这才是一致的。

"""

res = load("p1_connectivity.json")

if not res:

return

from collections import Counter

from load_attachment import GROUP_BOX, load_pieces

from microstructure import EDGE, GAP, R_A, DSU, contact_pairs

data = load_pieces()

组 1、组 2 的截面只有 1000 nm (长宽比 10:1), 按各组真实长宽比分配面板高度。

注意由此三个面板的纵轴量程相差 10 倍, 题注里已注明。

fig, axes = plt.subplots(3, 1, figsize=(7.1, 5.4),

gridspec_kw={"height_ratios": [0.72, 0.72, 2.35]})

for ax, name in zip(axes, ["组 1", "组 2", "组 3"]):

pieces = data[name]

sp, sq = pieces[:, :3], pieces[:, 3:]

n = len(sp)

half = EDGE / 2

只用碎片之间的接触建团簇——不加电极节点

dsu = DSU(n)

for a, b in contact_pairs(sp, sq, 2 * R_A + GAP):

dsu.union(int(a), int(b))

root = [dsu.find(i) for i in range(n)]

lo = np.minimum(sp[:, 0], sq[:, 0]) - R_A

hi = np.maximum(sp[:, 0], sq[:, 0]) + R_A

touch_l = {root[i] for i in np.nonzero(lo <= -half + GAP)[0]}

touch_r = {root[i] for i in np.nonzero(hi >= half - GAP)[0]}

spanning = touch_l & touch_r

同时触到两面 = 真正的贯通团簇

n_span = sum(Counter(root)[r] for r in spanning)

for i in range(n):

r = root[i]

if r in spanning:

c, lw, z = COLORS[2], 2.6, 4

elif r in touch_l:

c, lw, z = COLORS[1], 1.2, 2

elif r in touch_r:

c, lw, z = COLORS[0], 1.2, 2

else:

```

        c, lw, z = "0.80", 0.7, 1
        ax.plot([sp[i, 0], sq[i, 0]], [sp[i, 1], sq[i, 1]], color=c, lw=lw, zorder=z)
    ax.axvline(-half, color="k", lw=2.2)
    ax.axvline(half, color="k", lw=2.2)
    hy = GROUP_BOX[name][1] / 2
    ax.set_ylim(-hy * 1.05, hy * 1.05)
    ax.set_xlim(-half * 1.04, half * 1.04)
    g = res[name]
    span_txt = (f" 贯通团簇 {n_span}/{n} 个碎片" if spanning else " 无贯通团簇")
    ax.set_title(f"{name}: {g['n_media']} 根介质 A ({g['n_pieces_in_attachment']} 个碎片), "
                f" 截面 {int(GROUP_BOX[name][1])}×{int(GROUP_BOX[name][2])} nm —— "
                f"'{''贯通' if g['as_given'] else '不贯通'}' ({span_txt}) ", fontsize=8.5)
    ax.set_ylabel("Y / nm")
    axes[-1].set_xlabel("X / nm (左右两条粗黑线为带电面; 三个面板的纵轴量程不同)")

# 图例放到整张图下方, 避免像旧版那样压住组 1 面板里本就稀疏的碎片
handles = [Line2D([], [], color=COLORS[2], lw=2.6, label=" 贯通团簇 (同一团簇同时触及左右带电面)"),
            Line2D([], [], color=COLORS[1], lw=1.2, label=" 仅触及左带电面的团簇"),
            Line2D([], [], color=COLORS[0], lw=1.2, label=" 仅触及右带电面的团簇"),
            Line2D([], [], color="0.80", lw=1.0, label=" 两面都不触及的团簇")]
fig.legend(handles=handles, fontsize=7.5, ncol=2, loc="lower center",
           bbox_to_anchor=(0.5, -0.005), frameon=False)
fig.tight_layout(rect=(0, 0.075, 1, 1))
emit(fig, "05_p1_connectivity",
     " 三个微构体的接触图在 X-Y 平面的投影 (按真实棒—棒团簇着色)")

# ----- 图 4 问题二/三
def fig_p2p3() -> None:
    p2, p3 = load("p2_probabilities.json"), load("p3_threshold.json")
    if not p2:
        return
    fig, ax = plt.subplots(figsize=(7.2, 3.87))
    for k, mode in enumerate((PRIMARY_ORIENTATION, CONTROL_ORIENTATION)):
        rows = p2["runs"].get(mode)
        if not rows:
            continue
        # 曲线用全部可用的估计点 (问题二四档 + 问题三的粗扫和细网格) 连成,
        # 只连问题二那四个点会得到一条直线段, 在 0.7%-1.0% 之间明显低于真实曲线,
        # 看上去像是细网格点和主曲线矛盾。
        pts = [(r["phi_a"], r["p"]) for r in rows]
        if p3 and p3["modes"].get(mode):
            m3 = p3["modes"][mode]
            pts += [(r["phi_a"], r["p"]) for r in m3["coarse"]]
            pts += [(r["phi_a"], r["p"]) for r in m3["fine_grid"]]
        pts = sorted(set(pts))
        cx = np.array([p for p, _ in pts]) * 100
        cy = np.array([q for _, q in pts])
        lab = (" 球面均匀 (主口径)" if mode == PRIMARY_ORIENTATION
              else " 附件标定 -U(0, ) (灵敏度口径)")
        ax.plot(cx, cy, color=COLORS[k], ls=LINESTYLES[k], lw=1.6, alpha=0.9, label=lab)

# 题目点名的四档单独标出并带 Wilson 区间
x = np.array([r["phi_a"] for r in rows]) * 100
y = np.array([r["p"] for r in rows])
lo = np.array([r["ci_lo"] for r in rows])

```



```

hi = np.array([r["ci_hi"] for r in rows])
ax.errorbar(x, y, yerr=[y - lo, hi - y], color=COLORS[k], marker=MARKERS[k],
            ms=7, ls="none", capsize=3, zorder=4)
if p3 and p3["modes"].get(mode):
    ans = p3["modes"][mode]["answer_phi"]
    if ans:
        ax.plot([ans * 100], [0.9], marker="*", ms=15, color=COLORS[k],
                markeredgewidth=0.6, zorder=6)
        ax.annotate(f"{ans:.2%}", (ans * 100, 0.9), textcoords="offset points",
                    xytext=(4, -15), color=COLORS[k], fontsize=9)
ax.axhline(0.9, color=COLORS[5], ls=LINESTYLES[3], lw=1.4)
ax.text(0.505, 0.915, "P = 90% (问题三的目标线)", fontsize=8, color=COLORS[5])
ax.set_xlabel(" 介质 A 体积分数 / %")
ax.set_ylabel(" 微构体导通概率 P")
ax.set_ylim(-0.03, 1.05)
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout()
emit(fig, "06_p2_probability_curve",
     " 导通概率随介质 A 体积分数的变化")

# ----- 图 5 问题四
def fig_p4() -> None:
    """(N_A, N_B) 平面上的概率曲面、可行边界与等成本线。

    改动三处：
    1. 曲面改用 ** 单色蓝渐变 **。旧版用 viridis (紫—绿—黄多色)，
       读者无法从色相判断哪端概率更高，灰度打印也不单调；
    2. 只画一条等成本线 (最优成本)。旧版另画一条 " 成本高 15% " 的白虚线，
       与前者几乎平行且同色，除了让人误以为是可行边界之外没有信息；
    3. 注记框移出数据区，星号缩小。
    """
    p4 = load("p4_cost_optimum.json")
    if not p4:
        return
    grid = p4["grid"]
    na = np.array([g["n_a"] for g in grid], float)
    nb = np.array([g["n_b"] for g in grid], float)
    pp = np.array([g["p"] for g in grid], float)
    ua, ub = np.unique(na), np.unique(nb)
    P = np.full((len(ub), len(ua)), np.nan)
    for a, b, v in zip(na, nb, pp):
        P[np.searchsorted(ub, b), np.searchsorted(ua, a)] = v

    fig, ax = plt.subplots(figsize=(7.2, 3.87))
    im = ax.pcolormesh(ua, ub, P, cmap=seq_cmap(), shading="gouraud",
                    vmin=0, vmax=1, rasterized=True)
    cb = fig.colorbar(im, ax=ax, label=" 导通概率 P", pad=0.02)
    cb.outline.set_visible(False)

    # 等高线自带的 clabel 会沿线旋转排版，中文竖着转 60° 基本没法读；
    # 改为在曲线一端放一个水平标注。
    cs = ax.contour(ua, ub, P, levels=[0.9], colors=[COLORS[1]], linewidths=1.8)
    seg = cs.allsegs[0][0] if cs.allsegs and cs.allsegs[0] else None
    if seg is not None and len(seg) > 2:
        hx, hy = seg[0]

```

```

ax.annotate("$P=90\\%$ 可行边界", xy=(hx, hy), xytext=(10, -6),
            textcoords="offset points", fontsize=8.5, color=COLORS[1],
            ha="left", va="top")

best = p4["best"]
ca, cb_ = p4["cost_per_rod_yuan"], p4["cost_per_sphere_yuan"]
xs = np.linspace(ua.min() - 20, ua.max() + 30, 120)
ax.plot(xs, (best["cost_yuan"] - xs * ca) / cb_, color="w", ls="--", lw=1.4,
        label=f" 等成本线 {best['cost_yuan']:.2f} 元")

for v in p4.get("verified", []):
    ax.plot([v["n_a"]], [v["n_b"]], marker="o", ms=4.5, mfc="none",
            mec="w", mew=1.1, zorder=5)
ax.plot([best["n_a"]], [best["n_b"]], marker="*", ms=13, color=COLORS[3],
        mec="w", mew=0.9, zorder=6, label=" 推荐配比 $(608,0)$")

ax.legend(loc="lower left", fontsize=8, framealpha=0.92,
         facecolor="w", edgecolor="none")
ax.set_xlabel(" 介质 A 根数 $N_A$")
ax.set_ylabel(" 介质 B 颗数 $N_B$")
ax.set_title(" 等成本线 (虚线) 与可行边界 (实线) 在 $N_B=0$ 处相切",
            fontsize=9.5, color="#52514e")
ax.set_xlim(ua.min() - 20, ua.max() + 30)
ax.set_ylim(-140, ub.max() + 80)
ax.grid(False)
fig.tight_layout()
emit(fig, "08_p4_cost_optimum",
     "$ (N_A, N_B)$ 平面上的导通概率、可行边界与等成本线")

# ----- 图 6 灵敏度
def fig_sensitivity() -> None:
    s = load("sensitivity.json")
    if not s:
        return
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.43))
    ax = axes[0]
    g = s.get("gap_scan", [])
    if g:
        x = [r["gap"] for r in g]
        y = [r["p"] for r in g]
        lo = [r["ci_lo"] for r in g]
        hi = [r["ci_hi"] for r in g]
        ax.errorbar(x, y, yerr=[np.array(y) - lo, np.array(hi) - y], color=COLORS[0],
                    marker="o", capsize=3, lw=1.8)
        ax.axvline(1.8, color=COLORS[1], ls=LINESTYLES[1], lw=1.4)
        ax.annotate(" 题给值 =1.8", xy=(1.8, 0.06), fontsize=8, color=COLORS[1],
                    ha="center")
        ax.set_xlabel(" 导通判据阈值 / nm")
        ax.set_ylabel(" 导通概率 P")
        # 纵轴按 0-1 全量程画: 变动 ±50% 只挪动 0.048, 放大纵轴会把
        # " 不敏感 " 画成一条陡线, 与结论相反。右图同样用 0-1 横轴, 两图可直接对照。
        ax.set_ylim(0, 1.05)
        ax.annotate(f" 变动 ±50%, P 仅在 {min(y):.3f}-{max(y):.3f} 之间移动 (极差 {max(y)-min(y):.3f})",
                    xy=(0.5, 0.93), xycoords="axes fraction", ha="center", fontsize=8)
        ax.set_title(" 对判据阈值的敏感性 (=0.70% 固定)", fontsize=9)

```

```

ax = axes[1]
rows = s.get("assumption_table", [])
if rows:
    names = [r["name"] for r in rows]
    vals = [r["p"] for r in rows]
    err = [[v - r["ci_lo"] for v, r in zip(vals, rows)],
           [r["ci_hi"] - v for v, r in zip(vals, rows)]]
    y = np.arange(len(names))
    ax.barh(y, vals, xerr=err, color=[COLORS[0]] + [COLORS[3]] * (len(names) - 1),
            capsize=3, height=0.6)
    ax.set_yticks(y, names, fontsize=8)
    ax.invert_yaxis()
    ax.set_xlabel(" 导通概率 P")
    ax.set_xlim(0, 1.05)
    ax.set_title(" 建模口径对结论的影响 (同一 =0.70%) ", fontsize=9)
fig.tight_layout()
emit(fig, "t1_sensitivity",
     " 导通概率对判据阈值与建模口径的敏感性")

# ----- 图 1 技术路线
def fig_roadmap() -> None:
    """ 黑白技术路线图：突出公共内核、四问调用和校验闭环。 """
    from matplotlib.patches import FancyArrowPatch, Rectangle

    ink = "#111111"
    mid = "#6f6f6f"
    pale = "#f1f1f1"
    white = "#ffffff"

    fig, ax = plt.subplots(figsize=(7.4, 4.28))
    ax.set_xlim(0, 100)
    ax.set_ylim(0, 74)
    ax.axis("off")

    def rect(x, y, w, h, *, fc=white, ec=ink, lw=1.0, ls="-"):
        ax.add_patch(Rectangle((x, y), w, h, facecolor=fc,
                               edgecolor=ec, linewidth=lw, linestyle=ls))

    def arrow(x1, y1, x2, y2, *, dashed=False, ms=9):
        ax.add_patch(FancyArrowPatch(
            (x1, y1), (x2, y2), arrowstyle="-|>", mutation_scale=ms,
            linewidth=0.95, color=ink, linestyle="--" if dashed else "-",
            shrinkA=0, shrinkB=0,
        ))

    def input_node(x, w, title, detail=""):
        y, h = 59.0, 11.0
        rect(x, y, w, h, lw=1.05)
        if detail:
            ax.text(x + w / 2, y + 7.2, title, ha="center", va="center",
                   fontsize=8.7, fontweight="bold", color=ink)
            ax.text(x + w / 2, y + 3.3, detail, ha="center", va="center",
                   fontsize=7.3, color=mid, linespacing=1.25)

```

```

else:
    ax.text(x + w / 2, y + h / 2, title, ha="center", va="center",
            fontsize=8.8, fontweight="bold", color=ink)

def problem_node(x, number, title, detail):
    y, w, h, head_h = 16.0, 22.0, 13.5, 4.1
    rect(x, y, w, h, lw=1.0)
    rect(x, y + h - head_h, w, head_h, fc=ink, lw=0)
    ax.text(x + w / 2, y + h - head_h / 2,
            f" 问题 {number} {title}", ha="center", va="center",
            fontsize=7.8, fontweight="bold", color=white)
    ax.text(x + w / 2, y + (h - head_h) / 2, detail,
            ha="center", va="center", fontsize=7.1, color=ink,
            linespacing=1.35)

# 1. 输入与建模口径：同排等高，保持阅读方向单一。
ax.text(1.0, 71.7, "01 输入与建模口径", ha="left", va="center",
        fontsize=7.4, fontweight="bold", color=mid)
input_node(1.0, 21.5, " 赛题与附件")
input_node(28.0, 42.0, " 数据审计", " 周期盒尺寸 / 碎片口径 / 方向分布")
input_node(75.5, 23.5, " 建模口径", " 形成假设 A1-A5")
arrow(22.5, 64.5, 27.2, 64.5)
arrow(70.0, 64.5, 74.7, 64.5)

# 2. 公共判定内核：粗框表示全文唯一复用的算法内核。
ax.text(1.0, 55.8, "02 公共判定内核（四问共用）", ha="left", va="center",
        fontsize=7.4, fontweight="bold", color=mid)
kx, ky, kw, kh = 1.0, 39.0, 98.0, 14.0
rect(kx, ky, kw, kh, fc=pale, lw=1.55)
step_x = (4.0, 36.0, 68.0)
step_w = (25.0, 27.0, 27.0)
step_text = (
    ("1 边界截断", " 越界介质折回周期盒"),
    ("2 距离连边", " 表面最短距离 1.8 nm"),
    ("3 连通判定", " 并查集检验两带电面贯通"),
)
for x, w, (title, detail) in zip(step_x, step_w, step_text):
    rect(x, 41.3, w, 8.2, fc=white, lw=0.85)
    ax.text(x + w / 2, 46.9, title, ha="center", va="center",
            fontsize=7.9, fontweight="bold", color=ink)
    ax.text(x + w / 2, 43.6, detail, ha="center", va="center",
            fontsize=6.8, color=mid)
arrow(29.0, 45.4, 35.0, 45.4, ms=8)
arrow(63.0, 45.4, 67.0, 45.4, ms=8)
arrow(87.2, 59.0, 87.2, 53.5)

# 3. 四问调用：黑色标题条建立清晰层级，正文只保留方法与交付物。
ax.text(1.0, 35.7, "03 分问题求解", ha="left", va="center",
        fontsize=7.4, fontweight="bold", color=mid)
px = (1.0, 26.3, 51.6, 76.9)
problem_node(px[0], " 一", " 确定性判定", " 还原附件位姿\n 输出三组导通性")
problem_node(px[1], " 二", " 概率估计", " 蒙特卡洛抽样\hat{P} + Wilson 区间")
problem_node(px[2], " 三", " 阈值反解", "logit 粗定位 + 网格复算\n 反解  $P \geq 0.9$  的  $\varphi_{\min}$ ")
problem_node(px[3], " 四", " 成本优化", " 概率约束整数搜索\min C s.t.  $P \geq 0.9$ ")

# 一条总线把公共内核分发到四问，避免四条放射斜线交叉。

```

```

centers = [x + 11.0 for x in px]
ax.plot([centers[0], centers[-1]], [34.0, 34.0], color=ink, lw=0.9)
arrow(50.0, 39.0, 50.0, 34.0, ms=8)
for cx in centers:
    arrow(cx, 34.0, cx, 30.2, ms=8)

# 4. 校验闭环: 虚线统一表示校验与反馈, 不与主计算流混淆。
ax.text(1.0, 12.7, "04 独立校验与结果审计", ha="left", va="center",
        fontsize=7.4, fontweight="bold", color=mid)
rect(1.0, 2.3, 98.0, 8.0, fc=white, lw=0.95, ls="--")
ax.text(50.0, 6.3,
        "几何内核 6 项校验 | Wilson 区间 | 灵敏度分析 | 数字溯源",
        ha="center", va="center", fontsize=7.4, color=ink)
for cx in centers:
    arrow(cx, 16.0, cx, 10.7, dashed=True, ms=7)

fig.tight_layout()
emit(fig, "01_roadmap", "技术路线: 四问共用同一个导通判定内核")

# ----- 图 4 截断示意
def fig_schematic() -> None:
    """ 边界截断规则与导通判据的示意图 (非真实数据, 比例已放大)。"""
    from matplotlib.patches import Circle, FancyArrowPatch, Rectangle

    fig = plt.figure(figsize=(9.4, 3.43))
    gs = fig.add_gridspec(2, 2, width_ratios=[1.05, 1.35], hspace=0.55, wspace=0.22)

    # ---- (a) 边界截断
    ax = fig.add_subplot(gs[:, 0])
    ax.add_patch(Rectangle((-5, -5), 10, 10, fc="#fbfbfb", ec="0.6", lw=1.0, ls="--"))
    ax.plot([-5, -5], [-5, 5], color="k", lw=3.5)
    ax.plot([5, 5], [-5, 5], color="k", lw=3.5)
    ax.text(-5, 5.4, "带电面", ha="center", fontsize=8)
    ax.text(5, 5.4, "带电面", ha="center", fontsize=8)
    ax.plot([0.4, 5.0], [-1.2, 1.6], color=COLORS[0], lw=4, solid_capstyle="round")
    ax.plot([-5.0, -3.1], [1.6, 2.75], color=COLORS[1], lw=4, ls=(0, (4, 2)),
            solid_capstyle="round")
    ax.add_patch(FancyArrowPatch((4.6, 2.5), (-4.4, 2.5), arrowstyle="-|>",
                                mutation_scale=12, lw=1.2, color=COLORS[1],
                                linestyle=":", shrinkA=0, shrinkB=0))
    ax.text(0.1, 3.0, "平移一个边长  $L$ ", ha="center", fontsize=8, color=COLORS[1])
    ax.text(3.1, -1.6, "主段", fontsize=8.5, color=COLORS[0])
    ax.text(-4.6, 3.4, "折回段", fontsize=8.5, color=COLORS[1])
    ax.text(0, -4.4, "两段是彼此独立的导体 (假设 A2)", ha="center", fontsize=8)
    ax.set_xlim(-6.6, 6.6); ax.set_ylim(-5.6, 6.0)
    ax.set_aspect("equal"); ax.axis("off")
    ax.set_title("(a) 边界截断规则", fontsize=9.5)

    # ---- (b) 棒—棒接触
    ax = fig.add_subplot(gs[0, 1])
    ax.plot([-4.4, -0.35], [0.28, 0.02], color=COLORS[0], lw=7, solid_capstyle="round")
    ax.plot([0.35, 4.4], [-0.02, -0.28], color=COLORS[0], lw=7, solid_capstyle="round")
    ax.add_patch(FancyArrowPatch((-0.35, 0.02), (0.35, -0.02), arrowstyle="<|-|>",
                                mutation_scale=9, lw=1.2, color=COLORS[1]))
    ax.text(0, 0.95, "表面最短距离 = 1.8 nm, 判为导通",

```

```

        ha="center", fontsize=8.5, color=COLORS[1])
ax.text(-4.4, -1.15, " 介质 A: 圆柱体, r = 30 nm, 轴长 5000 nm",
        fontsize=8, color=COLORS[0])
ax.set_xlim(-5, 5); ax.set_ylim(-1.6, 1.5); ax.set_aspect("equal"); ax.axis("off")
ax.set_title("(b) 介质 A 之间的接触", fontsize=9.5)

# ---- (c) 球做桥
ax = fig.add_subplot(gs[1, 1])
ax.plot([-4.4, -1.25], [0.30, 0.10], color=COLORS[0], lw=7, solid_capstyle="round")
ax.plot([1.25, 4.4], [-0.10, -0.30], color=COLORS[0], lw=7, solid_capstyle="round")
ax.add_patch(Circle((0.0, 0.0), 1.05, fc=COLORS[3], ec="0.3", lw=0.9, alpha=0.9))
ax.add_patch(FancyArrowPatch((-1.25, 0.10), (1.25, -0.10), arrowstyle="<|-|>",
                             mutation_scale=9, lw=1.2, color=COLORS[1]))
ax.text(0, 1.45, " 一颗 B 最多能跨接轴距  $2(R+r+\Delta)=463.6$  nm 的两根 A",
        ha="center", fontsize=8.5, color=COLORS[1])
ax.text(1.35, -1.25, " 介质 B: 球, R = 200 nm", fontsize=8, color=COLORS[3])
ax.set_xlim(-5, 5); ax.set_ylim(-1.9, 2.1); ax.set_aspect("equal"); ax.axis("off")
ax.set_title("(c) 介质 B 充当棒间的桥", fontsize=9.5)

emit(fig, "04_schematic", " 边界截断规则与导通判据示意")

# ----- 图 9 收敛性
def fig_convergence() -> None:
    s = load("sensitivity.json")
    if not s or not s.get("convergence"):
        return
    conv = s["convergence"]
    T = np.array([c["trials"] for c in conv], float)
    p = np.array([c["p"] for c in conv])
    lo = np.array([c["ci_lo"] for c in conv])
    hi = np.array([c["ci_hi"] for c in conv])

    fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.17))
    ax = axes[0]
    ax.errorbar(T, p, yerr=[p - lo, hi - p], color=COLORS[0], marker="o",
                 capsize=3, lw=1.6)
    ax.axhline(p[-1], color=COLORS[5], ls=LINESTYLES[3], lw=1.2)
    ax.set_xscale("log"); ax.set_xlabel(" 试验次数 $T$"); ax.set_ylabel(" 导通概率 P")
    ax.set_title(" 估计值随样本量的稳定过程 (=0.70%) ", fontsize=9)

    ax = axes[1]
    half = (hi - lo) / 2
    ax.loglog(T, half, color=COLORS[0], marker="o", lw=1.6, label="Wilson 区间半宽")
    ref = half[0] * np.sqrt(T[0] / T)
    ax.loglog(T, ref, color=COLORS[1], ls=LINESTYLES[1], lw=1.4,
              label=r"$\propto 1/\sqrt{T}$ 参考线")
    ax.set_xlabel(" 试验次数 $T$"); ax.set_ylabel(" 区间半宽")
    ax.legend(fontsize=8)
    ax.set_title(" 半宽按  $1/\sqrt{T}$  收窄", fontsize=9)
    fig.tight_layout()
    emit(fig, "12_convergence", " 蒙特卡洛估计的收敛性")

# ----- 图 10 团簇结构
def fig_clusters() -> None:

```

```

""" 最大团簇占比与导通概率。

两条曲线都是  $[0, 1]$  上的无量纲比例，因此 ** 共用同一根纵轴 **。
旧版用了双纵轴 (twinax): 两个刻度的对齐方式是任意的，会凭空制造出
" 两条曲线在某处相交/分离 " 的假象，是图表最常见的误导性做法。
共轴之后，"P 已接近 1 而  $S_{max}$  仍在 0.2 附近" 这一核心事实
由两条线的真实垂直距离直接读出，不再依赖刻度怎么摆。
"""

c = load("cluster_stats.json")
if not c:
    return
rows = c["rows"]
x = np.array([r["phi"] for r in rows]) * 100
smax = np.array([r["s_max_mean"] for r in rows])
sd = np.array([r["s_max_sd"] for r in rows])
pp = np.array([r["p_percolate"] for r in rows])

fig, ax = plt.subplots(figsize=(7.0, 3.43))
ax.fill_between(x, np.clip(smax - sd, 0, 1), smax + sd,
                color=COLORS[0], alpha=0.15, lw=0)
ax.plot(x, pp, color=COLORS[1], ls=LINESTYLES[1], marker="s", ms=4.5,
        lw=1.6, label=" 导通概率 $P$")
ax.plot(x, smax, color=COLORS[0], marker="o", ms=4.5, lw=1.6,
        label=" 最大团簇占全部碎片的比例 $S_{\\max}$")

ax.set_xlabel(" 介质 A 体积分数 / %")
ax.set_ylabel(" 比例 (两者同为  $[0, 1]$  无量纲) ")
ax.set_ylim(0, 1.04)
ax.set_xlim(x.min() - 0.03, x.max() + 0.09)
ax.legend(fontsize=8.5, loc="upper left")

# 只在最右端直接标注两条线的落点，不给每个点标数字
ax.annotate(f"{pp[-1]:.2f}", (x[-1], pp[-1]), xytext=(6, -2),
           textcoords="offset points", color=COLORS[1], fontsize=8.5,
           va="center")
ax.annotate(f"{smax[-1]:.2f}", (x[-1], smax[-1]), xytext=(6, -2),
           textcoords="offset points", color=COLORS[0], fontsize=8.5,
           va="center")

# 用一段竖直线把 " 同一体积分数下两者的差距 " 标出来，替代旧版横穿图面的箭头
ax.annotate("", xy=(x[-1], pp[-1]), xytext=(x[-1], smax[-1]),
           arrowprops=dict(arrowstyle="<->", lw=1.0, color="#52514e"))
ax.text(x[-1] - 0.02, (pp[-1] + smax[-1]) / 2,
        "P 已近 1, \n 最大团簇仍只占约 %.0f%%" % (smax[-1] * 100),
        fontsize=8.5, ha="right", va="center", color="#52514e")
fig.tight_layout()
emit(fig, "07_cluster_structure", " 最大团簇占比与导通概率随体积分数的变化")

# ----- 图 9 两种介质的边际效率
def fig_marginal() -> None:
    """ 每元钱买到多少  $\Delta P$ : 两种介质的边际效率随  $N_A$  的变化。

    这是问题四 " 为什么不掺 B " 的机理图。表 13 给的是同样的数，
    但交叉点的位置在表里要靠逐列比大小才看得出来，画成两条线一眼可辨。
    """
    t = load("theory_check.json")

```

```

if not t or "marginal_efficiency" not in t:
    return
me = t["marginal_efficiency"]
ks = sorted(me, key=int)
x = np.array([int(k) for k in ks], float)
ea = np.array([me[k].get("dP_dNA_per_yuan", np.nan) for k in ks], float)
eb = np.array([me[k].get("dP_dNB_per_yuan", np.nan) for k in ks], float)

fig, ax = plt.subplots(figsize=(7.0, 3.34))
ax.plot(x, ea, color=COLORS[0], marker="o", ms=4.5, lw=1.6, label=" 加介质 A")
ax.plot(x, eb, color=COLORS[1], marker="s", ms=4.5, lw=1.6,
        ls=LINESTYLES[1], label=" 加介质 B")

# 交叉点: A 的效率首次超过 B 的那一段
cross = None
for i in range(1, len(x)):
    if np.isfinite(ea[i - 1]) and ea[i - 1] < eb[i - 1] and ea[i] > eb[i]:
        cross = (x[i - 1] + x[i]) / 2
        break
if cross is not None:
    ax.axvline(cross, color="#898781", lw=0.9, ls=":")
    ax.annotate(f" 效率交叉  $N_A \backslash \approx \{cross:.0f\}$ ", xy=(cross, ax.get_ylim()[1]),
               xytext=(4, -12), textcoords="offset points",
               fontsize=8.5, color="#52514e", va="top")

# 可行域 ( $P_{0.90}$  所需的  $N_A$ ) 整段落在交叉点右侧——这才是 " 用不上 B " 的原因
p3 = load("p3_threshold.json")
n_star = None
if p3:
    n_star = p3["modes"].get("isotropic", {}).get("answer_n_a")
if n_star:
    ax.axvspan(n_star, x.max() + 20, color=COLORS[0], alpha=0.07, lw=0)
    ax.annotate(f" 可行域  $N_A \backslash \geq \{n_star\}$  (A 占优区段)",
               xy=(n_star + 6, ax.get_ylim()[1] * 0.72), fontsize=8.5,
               color="#52514e")

ax.set_xlabel(" 介质 A 根数  $N_A$ ")
ax.set_ylabel(" 每元钱换来的  $\Delta P$ ")
ax.set_xlim(x.min() - 15, x.max() + 20)
ax.set_ylim(bottom=0)
ax.legend(fontsize=8.5, loc="upper left")
fig.tight_layout()
emit(fig, "09_marginal_efficiency", " 两种介质每元钱的边际效率及其交叉点")

# ----- 图 10 预算线审计的证据链
def fig_budget_audit() -> None:
    """ 预算线上每个点的  $P$  与  $0.90$  的关系——问题四结论的完整证据。

\needspace{6\baselineskip}
\begin{center}\small\bfseries 表 15 有同样的数, 但 " 哪些点被干净地排除、哪一个卡在  $0.90$  上 "\end{center}

要对着置信区间逐行心算; 画成带误差棒的图,  $0.90$  这条线一穿而过,
退化点自己跳出来。
"""

```



```

a = load("p4_global_audit.json")
if not a:
    return
rows = list(a.get("budget_line_checks", []))
if not rows:
    return
rc = load("p4_marginal_recheck.json")
big = {(r["n_a"], r["n_b"]): r for r in (rc or {}).get("points", [])}

# p4_global_audit.json 的预算线记录只存了 ci_hi (判据只用上界),
# 但误差棒必须两侧都有, 否则棒子只朝上、把点在视觉上压到区间底部。
# Wilson 区间由 ( $\hat{p}$ , 试验数) 唯一确定, 这里按同一公式补出下界。
def wilson_pair(p_hat: float, n: int) -> tuple[float, float]:
    z = 1.959964
    den = 1.0 + z * z / n
    c = (p_hat + z * z / (2 * n)) / den
    h = z * math.sqrt(p_hat * (1 - p_hat) / n + z * z / (4 * n * n)) / den
    return c - h, c + h

n_small = a.get("trials", 2500)
x = np.arange(len(rows))
fig, ax = plt.subplots(figsize=(7.4, 3.52))
ax.axhline(0.90, color=COLORS[1], lw=1.4, ls="--", zorder=1)
ax.annotate("$P=0.90$ 约束线", xy=(0, 0.90), xytext=(2, 5),
            textcoords="offset points", ha="left", fontsize=8.5,
            color=COLORS[1])

for i, r in enumerate(rows):
    key = (r["n_a"], r["n_b"])
    b = big.get(key)
    if b:
        pt, lo, hi = b["p"], b["ci_lo"], b["ci_hi"]
    else:
        pt = r["p"]
        lo, hi = wilson_pair(pt, n_small)
        hi = r.get("ci_hi", hi) # 上界以文件里存的为准
    undecided = not (hi < 0.90 or lo >= 0.90)
    col = COLORS[1] if undecided else COLORS[0]
    ax.errorbar([i], [pt], yerr=[[pt - lo], [hi - pt]], color=col,
                marker="o" if not b else "D", ms=5 if not b else 6,
                capsize=3, lw=1.4, zorder=3)

    if b:
        ax.annotate("$T=20000", (i, lo), xytext=(0, -14),
                    textcoords="offset points", ha="center",
                    fontsize=7.5, color="#898781")

ax.set_xticks(x)
ax.set_xticklabels([f"({r['n_a']},{r['n_b']})" for r in rows],
                    rotation=30, ha="right", fontsize=8)
ax.set_xlabel(" 预算线上的整数配比 $(N_A, N_B)$, 成本均低于 9.0252 元")
ax.set_ylabel(" 导通概率 $P$ (误差棒为 Wilson 95% 区间) ")
ax.set_xlim(-0.6, len(rows) - 0.4)
last = rows[-1]
ax.annotate(" 区间横跨 0.90: \n 可行性无法判定", xy=(len(rows) - 1, 0.9003),
            xytext=(-20, -52), textcoords="offset points", ha="right",
            fontsize=8.5, color=COLORS[1],

```

```

        arrowprops=dict(arrowstyle=">", color=COLORS[1], lw=1.1))
ax.set_title(" 除最右一点外, 预算线上所有更便宜的配比都被严格排除",
             fontsize=9.5, color="#52514e")
fig.tight_layout()
emit(fig, "10_budget_audit", " 预算线上逐点审计: $$ 的点估计与 Wilson 区间")

# ----- 图 13 球柱体近似的双侧界
def fig_bracket() -> None:
    """ 内接/外接球柱体给出的  $P$  双侧界, 以及它如何把问题三的答案夹成区间。 """
    g = load("geometry_bracket.json")
    if not g:
        return
    p3 = g.get("problem3", [])
    if not p3:
        return
    x = np.array([r["phi"] for r in p3]) * 100
    lo = np.array([r["lower"]["p"] for r in p3])
    hi = np.array([r["upper"]["p"] for r in p3])

    fig, ax = plt.subplots(figsize=(7.0, 3.43))
    ax.fill_between(x, lo, hi, color=COLORS[0], alpha=0.18, lw=0,
                   label=" 真实平端面圆柱的 $$ 必落在此带内")
    ax.plot(x, hi, color=COLORS[0], marker="o", ms=4.5, lw=1.6,
           label=" 外界  $P(K^+)_{\phi}$ : 球柱体 (轴长 5000) ")
    ax.plot(x, lo, color=COLORS[2], marker="^", ms=4.5, lw=1.6,
           ls=LINESTYLES[2], label=" 内界  $P(K^-)_{\phi}$ : 轴长 4940")
    ax.axhline(0.90, color=COLORS[1], lw=1.4, ls="--")
    ax.annotate("$P=0.90$", xy=(x.min(), 0.90), xytext=(2, 5),
               textcoords="offset points", fontsize=8.5, color=COLORS[1])

    lo_ok = [r["phi"] for r in p3 if r["lower_ok"]]
    hi_ok = [r["phi"] for r in p3 if r["upper_ok"]]
    if hi_ok and lo_ok:
        a, b = min(hi_ok) * 100, min(lo_ok) * 100
        ax.axvspan(a, b, color=COLORS[1], alpha=0.10, lw=0)
        ax.annotate(f" 答案被夹在 [{a:.2f}%, {b:.2f}%]",
                   xy=((a + b) / 2, 0.97), xycoords=("data", "axes fraction"),
                   ha="center", va="top", fontsize=8.5, color="#52514e")
        ax.plot([b], [0.90], marker="*", ms=13, color=COLORS[1],
               mec="w", mew=0.9, zorder=6)
        ax.annotate(f" 可证达标 {b:.2f}%", xy=(b, 0.90), xytext=(6, -16),
               textcoords="offset points", fontsize=8.5, color=COLORS[1])

    ax.set_xlabel(" 介质 A 体积分数 / %")
    ax.set_ylabel(" 导通概率 $$")
    ax.set_xlim(x.min() - 0.005, x.max() + 0.005)
    ax.legend(fontsize=8, loc="lower right")
    fig.tight_layout()
    emit(fig, "13_geometry_bracket", " 球柱体近似的严格双侧界与问题三答案的夹逼")

def main() -> int:
    font = setup(font_size=10)
    print(f" 中文字体: {font}")
    # 顺序 = 正文中出现的先后。文件名前缀即图号, 改动顺序时两者一起改,

```

```

# 避免出现 " 图 10 排在图 7 前面 " 这类编号与出现次序不一致的问题。
for f in (fig_roadmap, fig_orientation, fig_fragments, fig_schematic,
         fig_p1, fig_p2p3, fig_clusters, fig_p4, fig_marginal,
         fig_budget_audit, fig_sensitivity, fig_convergence, fig_bracket):
    try:
        f()
    except Exception as e: # 单张图失败不该阻断其余图
        print(f"    {f.__name__}: {type(e).__name__}: {e}")
lines = ["# 图注清单\n",
        "| 文件 | 图注 (正文结论) |", "|---|---|"]
lines += [f"| {s}.png` / `.pdf` | {c} |" for s, c in CAPTIONS]
(FIGDIR / " 图注清单.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
print(f"\n 共 {len(CAPTIONS)} 张图 -> {FIGDIR}")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/make_results_bundle.py

```

#!/usr/bin/env python3
""" 把各阶段的结果 JSON 合并成一个 results/results.json, 并把源程序装配进论文附录。

两件事都是为了让论文可核查:

1. ** 数字溯源 **: 论文里的每个数值都应当能在 results.json 里找到出处。
   分散在 p1/p2/p3/p4/... 各文件里时, `check_paper.py --results` 一次只能查一个,
   合并后就能一次查全。
2. ** 附录代码 **: 规范要求附录给出全部完整、可运行的源程序。手工粘贴一定会和 src/ 漂移,
   所以由脚本从 src/*.py 直接装配, 保证附录与实际运行的代码逐字一致。
"""

from __future__ import annotations

import json
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
RESULTS = ROOT / "results"
SRC = ROOT / "src"
PAPER = ROOT / " 论文 _ 微构体中填充导电介质的仿真优化.md"

PARTS = [
    "validation.json", "data_audit.json", "theory_check.json",
    "p1_connectivity.json", "p2_probabilities.json", "p3_threshold.json",
    "p4_cost_optimum.json", "p4_break_even.json", "sensitivity.json",
    "sphere_mode_check.json", "cluster_stats.json",
    "geometry_bracket.json", "p4_global_audit.json", "p4_global_audit2.json",
    "p4_marginal_recheck.json",
]

# 附录里源程序的呈现顺序: 先内核, 再各问求解, 最后辅助脚本
ORDER = [
    "microstructure.py", "load_attachment.py", "simulate.py",
    "audit_attachment.py", "validate.py", "theory_check.py", "cluster_stats.py",

```

```

"solve_p1.py", "solve_p2.py", "solve_p3.py", "solve_p4.py", "p4_break_even.py",
"sensitivity.py", "sphere_mode_check.py", "p4_backfill_probes.py",
"geometry_bracket.py", "p4_global_audit.py", "p4_marginal_recheck.py",
"paper_figures.py", "make_results_bundle.py",
]

```

APPENDIX_MARK = "### 附录 B 完整可运行源程序"

```

def rounded_variants(obj, out: set) -> None:
    """ 收集所有数值在 2/3/4 位小数下的取整值。

    论文里的概率按 4 位小数报 (如 0.2193), 而 results.json 里存的是原始的
    0.21925。`check_paper.py` 的数字溯源只回溯到 3 位小数, 于是会把这些
    明明有出处的数字报成 "找不到出处"。把取整值一并写进结果文件,
    既消除了这类误报, 也把 "论文取几位小数" 这件事本身记录在案。
    """
    if isinstance(obj, dict):
        for v in obj.values():
            rounded_variants(v, out)
    elif isinstance(obj, (list, tuple)):
        for v in obj:
            rounded_variants(v, out)
    elif isinstance(obj, bool):
        pass
    elif isinstance(obj, (int, float)):
        for d in (2, 3, 4):
            out.add(round(float(obj), d))

def bundle_results() -> dict:
    out = {}
    for name in PARTS:
        p = RESULTS / name
        if p.exists():
            out[p.stem] = json.loads(p.read_text(encoding="utf-8"))
        else:
            print(f" ! 缺少 {name} (该阶段尚未运行) ")
    rv: set = set()
    rounded_variants(out, rv)
    out["_rounded_variants"] = sorted(rv)
    # 论文里引用的若干 ** 派生量 ** (两口径之差、阈值扫描极差等) 本身不出现在任何单个
    # 结果文件里, 但完全由已有结果算出。显式落盘, 使数字溯源不必靠人工心算。
    try:
        sens = out.get("sensitivity", {})
        rows = {r["name"]: r["p"] for r in sens.get("assumption_table", [])}
        base = next((v for k, v in rows.items() if k.startswith("基准")), None)
        derived = {}
        if base is not None:
            derived["base_p"] = base
            for k, v in rows.items():
                if not k.startswith("基准"):
                    derived[f"delta_vs_base::{k}"] = round(v - base, 6)
        gaps = [g["p"] for g in sens.get("gap_scan", [])]
        if gaps:
            derived["gap_scan_range"] = round(max(gaps) - min(gaps), 6)

```

```

        derived["gap_scan_p_min"] = round(min(gaps), 6)
        derived["gap_scan_p_max"] = round(max(gaps), 6)

# 正文里若干 ** 派生量 **: 它们由结果文件换算而来, 本身不是任何一个文件里的字段,
# 于是数字溯源会把它们当成 " 查无出处 "。在这里显式算出并落盘,
# 既消除误报, 也让 " 这个数是怎么来的 " 有一处可查。
from microstructure import COST_A, COST_B, GAP, R_A, R_B, V_B
th = out.get("theory_check", {})
if th:
    derived["excluded_volume_1e9_nm3"] = round(
        th["excluded_volume_estimate"]["mean_excluded_volume_nm3"] / 1e9, 4)
    derived["phi_c_excluded_volume_percent"] = round(
        th["excluded_volume_estimate"]["critical_volume_fraction"] * 100, 4)
    mid = th.get("simulation_midpoint", {})
    for k, v in mid.items():
        derived[f"phi_at_P50_percent::{k}"] = round(v["phi_at_P50"] * 100, 4)
    derived["cost_ratio_cA_over_cB"] = round(COST_A / COST_B, 4)
    derived["iso_cost_slope"] = round(-COST_A / COST_B, 4)
    derived["sphere_max_bridge_axis_gap_nm"] = round(2 * (R_B + R_A + GAP), 4)
    derived["cost_of_3200_spheres_yuan"] = round(3200 * COST_B, 4)
    be = out.get("p4_break_even", {})
    if be:
        derived["break_even_cost_per_sphere_1e3_yuan"] = round(
            be["break_even_cost_per_sphere_yuan"] * 1e3, 4)
        derived["price_premium_over_break_even_percent"] = round(
            (be["unit_price_now_yuan_per_um3"] / be["break_even_unit_price_yuan_per_um3"]
             - 1) * 100, 4)
    derived["mc_seeds_used"] = {"default": 20260810, "p3_final": 987654321,
                               "convergence": 13579, "p4_recheck": 20260811}
# 灵敏度口径的可行边界斜率: 正文用它说明 " 边界的斜率对方向分布不敏感,
# 敏感的是边界的位置 "。它在对照口径目录下, 不在主口径的结果文件里。
ctrl = RESULTS / " 对照口径 _polar_uniform" / "p4_break_even.json"
if ctrl.exists():
    derived["boundary_slope_control_caliber"] = round(
        json.loads(ctrl.read_text(encoding="utf-8"))["slope_dNB_dNA"], 4)
if derived:
    out["derived"] = derived
    print(f" 派生量 {len(derived)} 项 -> results.json 的 derived 段")
except Exception as e:
    print(f" ! 派生量计算跳过: {e}")

(RESULTS / "results.json").write_text(
    json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
print(f" 合并 {len(out)} 个结果文件 -> results/results.json")
return out

def build_appendix() -> int:
    files = [SRC / n for n in ORDER if (SRC / n).exists()]
    extra = sorted(p for p in SRC.glob("*.py") if p.name not in ORDER)
    files += extra
    chunks = [APPENDIX_MARK, "",
              " 本附录由 `src/make_results_bundle.py` 从 `src/` 直接装配, 与实际运行的代码逐字一致。",
              " 运行顺序见附录 A 之后的说明。依赖 Python 3.11 与 numpy / scipy / matplotlib / openpyxl。",
              ""]
    total = 0

```

```

for f in files:
    code = f.read_text(encoding="utf-8").rstrip("\n")
    total += len(code.splitlines())
    chunks += [f"#### `src/{f.name}`", "", "`python`, code, "`", ""]
text = "\n".join(chunks)

paper = PAPER.read_text(encoding="utf-8")
idx = paper.find(APPENDIX_MARK)
if idx < 0:
    print(" ! 论文里没有找到附录 B 的标题, 未改动")
    return 0
PAPER.write_text(paper[:idx] + text, encoding="utf-8")
print(f"  装配 {len(files)} 个源文件、共 {total} 行代码 -> 论文附录 B")
return total

def main() -> int:
    RESULTS.mkdir(parents=True, exist_ok=True)
    bundle_results()
    build_appendix()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/build_paper_pdf.py

```
#!/usr/bin/env python3
```

```
""" 把终稿 Markdown 编译成提交用 PDF (pandoc + XeLaTeX)。
```

排版规范照搬竞赛论文的通行要求：*A4*、四边留白 *2.5 cm*、页脚居中连续页码、正文无目录、图题在图下方、表题在表上方。字体用 *texlive* 自带的 *Fandol* 开源字族，不依赖任何私有字体，换台装了 *texlive* 的机器能编出同样的版面。

用法：

```
python3 src/build_paper_pdf.py          # 完整版（含附录 B 全部源程序）
python3 src/build_paper_pdf.py --no-code # 去掉附录 B，便于快速预览正文版面
```

中间文件放 *build/*，产物是与论文同名的 *PDF*。

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import re
import shutil
import subprocess
import sys
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parents[1]
MANUSCRIPT = ROOT / "论文_微构体中填充导电介质的仿真优化.md"
HEADER = ROOT / "header_paper.tex"
BUILD = ROOT / "build"
APPENDIX_B = "### 附录 B 完整可运行源程序"
```

```

# LaTeX 特殊字符（数学环境之外才需要转义）
_TEX_ESC = {"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "#": r"\#",
            "_": r"\_", "{": r"\{", "(": r"\(", "(": r"\)", "(": r"\}",
            "~": r"\textasciitilde{}", "^": r"\textasciicircum{}}"}

def tex_escape(text: str) -> str:
    """ 转义 LaTeX 特殊字符，但保留  $...$  行内公式原样。

    题注里既有中文和百分号，又有  $AN_A$  这样的行内公式；整体当原始 LaTeX 会被
    百分号注释掉半行，整体转义又会把公式打死。所以按  $...$  切开分别处理。
    """
    out = []
    for i, part in enumerate(re.split(r"(\$[\^$]*\$)", text)):
        if i % 2 == 1:
            # 落在  $...$  里，原样保留
            out.append(part)
        else:
            out.append("".join(_TEX_ESC.get(ch, ch) for ch in part))
    return "".join(out)

def preprocess(text: str, include_code: bool) -> str:
    """ 把 Markdown 调整成适合 pandoc-LaTeX 的形态。"""
    if not include_code:
        idx = text.find(APPENDIX_B)
        if idx > 0:
            text = text[:idx] + (
                APPENDIX_B + "\n\n"
                "（预览版已省略附录 B 的源程序；完整版由 "
                "`python3 src/build_paper_pdf.py` 生成。）\n")

    lines = text.split("\n")
    out: list[str] = []
    i = 0
    img_re = re.compile(r"![[([^\]]*)]]\((figures_paper/[^)]+)\)")
    # 题注必须 ** 整行 ** 就是题注：早先只要求行首匹配，结果把 " 表 13 列出了阶段 C 的全部
    # 探测点……" 这类以表号开头的正文句子也当成题注，在 PDF 里排成了居中加粗的一行。
    # 题注行普遍短且不以句号结尾，这里用长度与结尾标点再筛一道。
    cap_re = re.compile(r"^(图 | 表)\s*(?:\d+|A\d+)\s+\S")

    def is_caption(line: str) -> bool:
        s = line.strip()
        return bool(cap_re.match(s)) and len(s) <= 60 and not s.endswith((".", ";", ":", " "))

    while i < len(lines):
        line = lines[i]
        m = img_re.match(line.strip())
        if m:
            # 向后找紧随的 " 图 N …" 题注，和图打包进同一个 [H] 浮动体。
            # 分成两个块的话，LaTeX 可能在图与题注之间分页——预览版里就出现过
            # 图在第 4 页、题注在第 5 页开头的情况。
            j = i + 1
            while j < len(lines) and not lines[j].strip():
                j += 1

```

```

        cap = lines[j] if j < len(lines) and is_caption(lines[j]) else ""
        path = (ROOT / m.group(2)).as_posix()
        out += ["", "\\begin{figure}[H]", "\\centering",
                f"\\includegraphics[width=0.92\\linewidth]{{{path}}}"
        ]
        if cap:
            out += ["\\par\\vspace{5pt}",
                    "\\small\\bfseries " + tex_escape(cap) + "]"]
            i = j
        out += ["\\end{figure}", ""]
        i += 1
        continue

    if is_caption(line):
        # 表题：居中小字加粗，不浮动
        # \\needspace 保证题注下方至少还有 6 行的位置，否则整块推到下一页。
        # 图那边是靠 figure[H] 把图与题注打包，表这里用不了浮动体，只能这样绑。
        out += ["", "\\needspace{6\\baselineskip}",
                "\\begin{center}\\small\\bfseries "
                + tex_escape(line) + "\\end{center}", ""]
        i += 1
        continue

    out.append(line)
    i += 1
return "\\n".join(out)

def main() -> int:
    ap = argparse.ArgumentParser(description=" 编译论文 PDF")
    ap.add_argument("--no-code", action="store_true", help=" 省略附录 B 的源程序")
    ap.add_argument("--ai-details", action="store_true",
                    help=" 改为编译支撑材料《AI 工具使用详情》")
    args = ap.parse_args()

    for tool in ("pandoc", "xelatex"):
        if shutil.which(tool) is None:
            sys.exit(f" 缺少 {tool}; 请先安装 pandoc 与 texlive-xetex/texlive-lang-chinese")

    BUILD.mkdir(parents=True, exist_ok=True)

    if args.ai_details:
        # 支撑材料里要求文件名精确为 AI 工具使用详情.pdf
        src_md = ROOT / "AI 工具使用详情.md"
        out_pdf = ROOT / "AI 工具使用详情.pdf"
        cmd = ["pandoc", str(src_md), "--from", "markdown+pipe_tables",
                "--pdf-engine", "xelatex", "--top-level-division", "section",
                "--include-in-header", str(HEADER), "-V", "documentclass=article",
                "-V", "fontsize=12pt", "-o", str(out_pdf)]
        res = subprocess.run(cmd, capture_output=True, text=True)
        if res.returncode != 0:
            print(res.stderr[-2000:])
            sys.exit(" 编译失败")
        print(f"    {out_pdf.name} ({out_pdf.stat().st_size/1024:.0f} KB) ")
        return 0

    staged = BUILD / "paper.md"
    staged.write_text(preprocess(MANUSCRIPT.read_text(encoding="utf-8")),

```



```

        include_code=not args.no_code), encoding="utf-8")

out_pdf = ROOT / (" 微构体中填充导电介质的仿真优化 _ 预览版.pdf" if args.no_code
                  else " 微构体中填充导电介质的仿真优化 _ 终稿.pdf")

cmd = [
    "pandoc", str(staged),
    "--from", "markdown+tex_math_dollars+pipe_tables+raw_tex",
    "--to", "latex",
    "--pdf-engine", "xelatex",
    "--top-level-division", "section",
    "--include-in-header", str(HEADER),
    "--resource-path", str(ROOT),
    "--highlight-style", "monochrome",
    "-V", "documentclass=article",
    "-V", "fontsize=12pt",
    "-o", str(out_pdf),
]

print(" " + " ".join(cmd[:6]) + " ...")
res = subprocess.run(cmd, capture_output=True, text=True)
if res.returncode != 0:
    (BUILD / "pandoc_error.log").write_text(res.stdout + res.stderr, encoding="utf-8")
    print(res.stderr[-3000:])
    sys.exit(" 编译失败, 详见 build/pandoc_error.log")

size = out_pdf.stat().st_size / 1024 / 1024
print(f"    {out_pdf.name} ({size:.2f} MB) ")
if size > 20:
    print("    ! 超过 20 MB, 提交前需压缩或改用预览版")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

src/p4_global_audit2.py

```
#!/usr/bin/env python3
```

""" 问题四成本下界的 ** 完整 ** 穷举审计 (补上 `p4_global_audit.py` 的覆盖缺口)。

原审计 (`p4_global_audit.py`) 的问题, 评审复核时查实:

1. 预算线阶段是 `for a in range(lo, hi + 1, 16)`, 只取到 $N_A=592$; $a=608$ 时 `nb_max < 0` 被跳过, 于是 $N_A \in (592, 608)$ 整段没有任何排除依据。
2. 预算线判据 $P(a, N_B \setminus \{ \max \}(a)) < 0.90$ 只能排除 ** 该列 ** (以及 $N_A \leq a$ 且 $N_B \leq N_B \setminus \{ \max \}(a)$ 的点), 对 $N_A > a$ 的列无效; 16 的步长在相邻两点之间留下了薄片。
3. 表 9 里 $(604, 0)$ (8.9658 元, 比宣称的下界 9.0239 元还低) 被判为 " 否 " 用的是点估计 0.8943 , 而本审计声明的判据是 Wilson ** 上界 $** < 0.90$ ——它的上界是 0.9019 , 按论文自己的标准并未被排除。

** 正确的判据。 ** 设待证成本 $C \setminus \{ c_{AN_A} \}$ 。对固定的 $N_A = a$, 成本低于 $C \setminus \{ * \}$ 等价于 $N_B \leq N_B \setminus \{ \max \}(a) = \lceil (C \setminus \{ c_{Aa} \} / c_B \rceil - 1$ 。由 PP 对 N_B 单调,

该列存在比 C^* 便宜的可行点 $\Leftrightarrow P(a, N_B \setminus \{ \max \}(a)) \geq 0.90$ 。

所以 ** 逐列在预算线上取点, 是判定 " C^* 是否最优 " 的充要检验 **, 不是充分条件。

把 a 跑遍 $[0, N_A^{\max})$ 的每一个整数，结论就是严格的。

**** 怎么跑得动。**** 逐列 608 次仿真太贵，故分两阶段：

- 阶段 1 (角点，便宜)：取网格 $a_1 < a_2$ ，区间内任意成本低于 C^* 的点都被角点 $(a_2, N_B^{\max}(a_1))$ 按分量支配，一次仿真排除整条区间。步长越小角点越紧。本脚本对 $[0, 480]$ 直接复用原审计已算出的步长 24 的结果（那 20 个区间已严格排除），对 $[480, 608]$ 用步长 4 重算。
- 阶段 2 (逐列，严格)：阶段 1 没排除掉的区间，逐个整数 a 在预算线上取点。
- 阶段 3 (擦边点加样本)：阶段 2 中 *Wilson* 上界够到 0.90 的点，换独立种子加到 T_{FINE} 复核；另把 $(604, 0)$ 单独加到 T_{FINE} ，把表 9 那处判断不一致补上。

结果写入 `results/p4_global_audit2.json`，边算边写，可断点续跑。

"""

```
from __future__ import annotations

import json
import math
import time
from pathlib import Path

from microstructure import COST_A, COST_B
from simulate import Config, estimate_p, wilson

RESULTS = Path(__file__).resolve().parents[1] / "results"
DST = RESULTS / "p4_global_audit2.json"
TARGET = 0.90
# 样本量阶梯：先便宜地筛，排不掉再加样本。排除判断用的是 ** 最终那一档 ** 的 Wilson 上界。
T_LADDER = (1000, 6000, 20000)
# 排除判断取 99% Wilson 上界 ( $z=2.576$ ) 而非 95%：本审计要做上百次判定，
# 95% 下的族错误率不可忽略；99% 更保守，代价只是多算一点。
Z_EXCLUDE = 2.5758293035489004
SEED_FINE = 20260811      # 擦边点复核用的独立种子
CORNER_STEP = 4           # [480, 560] 上的角点步长（原审计用 24，太松）
LOW_COVERED = 480         # [0, 480] 已由原审计的步长 24 角点严格排除
COL_FROM = 561            # [561, 607] 逐列取预算线点 ( $N_B$  小、跑得快、且最贴近最优解)

def nb_max(cost_star: float, n_a: int) -> int:
    """ 成本严格低于  $cost\_star$  时，该  $N_A$  允许的最大  $N_B$ 。 """
    return max(-1, math.ceil((cost_star - n_a * COST_A) / COST_B) - 1)

def judge(n_a: int, n_b: int, seed: int = 20260810) -> dict:
    """ 沿样本量阶梯自适应地判定一个点。

    每一档都用 99% Wilson 区间判：上界 < 0.90 -> 排除；下界 >= 0.90 -> 可行性确认；
    否则加样本。** 报告用的是最终那一档的区间 **，阶梯只决定何时停止花算力。
    这是一个序贯停止规则，故判断本身取得比 95% 更严，见模块开头的说明。
    """
    r = None
    for t in T_LADDER:
        r = estimate_p(Config(n_a=n_a, n_b=n_b), t, seed=seed)
        lo, hi = wilson(r["hits"], t, z=Z_EXCLUDE)
        r["ci99_lo"], r["ci99_hi"] = lo, hi
```

```

        if hi < TARGET or lo >= TARGET:
            break
    return {"n_a": n_a, "n_b": n_b, "cost": n_a * COST_A + n_b * COST_B,
            "p": r["p"], "trials": r["trials"], "seed": seed,
            "ci95_lo": r["ci_lo"], "ci95_hi": r["ci_hi"],
            "ci99_lo": r["ci99_lo"], "ci99_hi": r["ci99_hi"],
            "excluded": bool(r["ci99_hi"] < TARGET),
            "feasible_confirmed": bool(r["ci99_lo"] >= TARGET)}

def load_state() -> dict:
    if DST.exists():
        try:
            s = json.loads(DST.read_text(encoding="utf-8"))
            s.setdefault("corners", {}); s.setdefault("columns", {})
            return s
        except Exception:
            pass
    return {"corners": {}, "columns": {}}

def save(state: dict) -> None:
    RESULTS.mkdir(parents=True, exist_ok=True)
    DST.write_text(json.dumps(state, ensure_ascii=False, indent=2), encoding="utf-8")

def summarize(state: dict, cost_star: float) -> None:
    """ 由已完成的判定给出当前能主张的成本下界。"""
    done = list(state["columns"].values())
    open_pts = [c for c in done if not c["excluded"]]
    cheaper_ok = [c for c in open_pts
                  if c.get("feasible_confirmed") and c["cost"] < cost_star]
    undecided = [c for c in open_pts if not c.get("feasible_confirmed")]
    bound = min([c["cost"] for c in open_pts], default=cost_star)
    state["summary"] = {
        "columns_done": len(done),
        "corners_done": len(state["corners"]),
        "proved_lower_bound_yuan": bound,
        "undecided": sorted([k: c[k] for k in ("n_a", "n_b", "cost", "p",
                                              "ci99_lo", "ci99_hi", "trials")}
                           for c in undecided], key=lambda z: z["cost"]),
        "confirmed_cheaper": sorted([k: c[k] for k in ("n_a", "n_b", "cost", "p")}
                                   for c in cheaper_ok], key=lambda z: z["cost"]),
    }
    if cheaper_ok:
        v = (f" 存在比 C*={cost_star:.4f} 元更便宜且可行性已确认的点, "
             f" 最低 {min(c['cost'] for c in cheaper_ok):.4f} 元 —— C* 不是最优。")
    elif undecided:
        v = (f" 没有任何更便宜的点被确认可行; {len(undecided)} 个点仍无法判定, "
             f" 其中最便宜的为 {bound:.4f} 元。故最低总成本落在 "
             f"[{bound:.4f}, {cost_star:.4f}] 元, 成本低于 {bound:.4f} 元的整数点均已严格排除。")
    else:
        v = f" 所有成本低于 C*={cost_star:.4f} 元的整数点均已严格排除, C* 为全局最小。"
    state["summary"]["verdict"] = v

```

```

def main() -> int:
    best = json.loads((RESULTS / "p4_cost_optimum.json").read_text(encoding="utf-8"))["best"]
    n_a_star, cost_star = best["n_a"], best["cost_yuan"]
    state = load_state()
    state.update({"incumbent": {"n_a": n_a_star, "n_b": best["n_b"], "cost_yuan": cost_star},
                  "target": TARGET, "t_ladder": list(T_LADDER),
                  "z_exclude": Z_EXCLUDE, "seed_fine": SEED_FINE,
                  "low_covered_by_original_audit": LOW_COVERED})
    print(f" 特征: C* = {cost_star:.4f} 元 (N_A={n_a_star}, N_B={best['n_b']}) ")
    print(f"[0,{LOW_COVERED}] 复用原审计步长 24 的角点结论 (已严格排除) \n")

    # --- 阶段 1: [COL_FROM, N_A*-1] 逐列取预算线点, 自高向低 (最贴近最优解, 且 N_B 小、跑得快)
    print(f"=== 阶段 1: 逐列预算线 N_A [{COL_FROM},{n_a_star - 1}], 自高向低 ===")
    for a in range(n_a_star - 1, COL_FROM - 1, -1):
        key = str(a)
        if key not in state["columns"]:
            nb = nb_max(cost_star, a)
            if nb < 0:
                state["columns"][key] = {"n_a": a, "n_b": nb, "cost": a * COST_A,
                                          "note": " 该列无更便宜的点", "excluded": True,
                                          "feasible_confirmed": False}
            else:
                t0 = time.time()
                state["columns"][key] = judge(a, nb)
                state["columns"][key]["secs"] = round(time.time() - t0, 1)
                summarize(state, cost_star); save(state)
        c = state["columns"][key]
        tag = (" 已排除" if c["excluded"] else
              " 可行 (更便宜)" if c.get("feasible_confirmed") else " 无法判定")
        print(f" 列 {a:3d}: 预算线 =({a:3d},{c['n_b']:4d}) 成本 ={c['cost']:.4f} "
              f"P={c.get('p', float('nan')):.4f} T={c.get('trials', 0):5d} "
              f"99% 区间 =[{c.get('ci99_lo', 0):.4f},{c.get('ci99_hi', 0):.4f}] {tag}", flush=True)

    # --- 阶段 2: [480, COL_FROM] 用步长 4 的角点批量覆盖
    print(f"\n=== 阶段 2: 角点, 步长 {CORNER_STEP}, 覆盖 N_A [{LOW_COVERED},{COL_FROM}] ===")
    grid = list(range(LOW_COVERED, COL_FROM + 1, CORNER_STEP))
    if grid[-1] != COL_FROM:
        grid.append(COL_FROM)
    for a1, a2 in zip(grid[:-1], grid[1:]):
        key = f"{a1}-{a2}"
        if key not in state["corners"]:
            nb = nb_max(cost_star, a1)
            t0 = time.time()
            r = judge(a2, nb)
            r.update({"a1": a1, "a2": a2, "secs": round(time.time() - t0, 1)})
            state["corners"][key] = r
            save(state)
        c = state["corners"][key]
        print(f"  N_A [{a1:3d},{a2:3d}] 角点 =({a2:3d},{c['n_b']:5d}) "
              f"P={c.get('p', float('nan')):.4f} T={c.get('trials', 0):5d} "
              f"99% 上界 ={c.get('ci99_hi', 0):.4f} "
              f"{ '已排除' if c['excluded'] else ' 未排除 -> 需逐列' }", flush=True)
        if not c["excluded"]:
            for a in range(a1, a2 + 1):
                k2 = str(a)
                if k2 in state["columns"]:

```

```

        continue
    nb2 = nb_max(cost_star, a)
    t0 = time.time()
    state["columns"][k2] = judge(a, nb2)
    state["columns"][k2]["secs"] = round(time.time() - t0, 1)
    summarize(state, cost_star); save(state)
    c2 = state["columns"][k2]
    print(f"      列 {a:3d}: ({a:3d},{nb2:4d}) 成本={c2['cost']:.4f} "
          f"P={c2['p']:.4f} {'已排除' if c2['excluded'] else '未排除'}", flush=True)

# --- 阶段 3: 把仍未判定的点换独立种子复核, 另补 (604,0)
pend = [(c["n_a"], c["n_b"]) for c in state["columns"].values() if not c["excluded"]]
pend.append((604, 0))
state.setdefault("recheck", {})
print(f"\n=== 阶段 3: 独立种子 {SEED_FINE} 复核 {len(pend)} 点 ===")
for a, nb in pend:
    key = f"{a}-{nb}"
    if key not in state["recheck"]:
        state["recheck"][key] = judge(a, nb, seed=SEED_FINE)
        save(state)
    c = state["recheck"][key]
    print(f"  ({a:3d},{nb:4d}) 成本={c['cost']:.4f} P={c['p']:.4f} T={c['trials']}"
          f"→ {'排除' if c['excluded'] else '可行性已确认' if c['feasible_confirmed'] else '无法判定'}",
          flush=True)

summarize(state, cost_star); save(state)
print("\n 结论: " + state["summary"]["verdict"])
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```